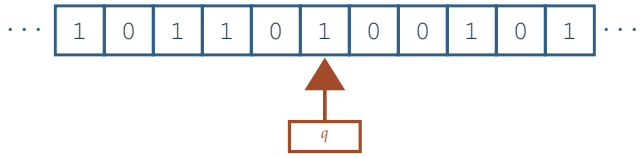
Informatik von Grund auf — Band 2

Die Grenze der Maschine

Eine vollständige Einführung in Berechenbarkeit und formale Sprachen



Von endlichen Automaten bis zum Halteproblem
Jeder Schritt erklärt. Mit vollständig durchgerechneten Übungsaufgaben.

Stand der Forschung: August 2026

Informatik von Grund auf — Band 2
Die Grenze der Maschine · Berechenbarkeit und formale Sprachen

Herausgeber und Urheber: Jürgen Maier
Entstanden 2026 in Zusammenarbeit mit Claude (Anthropic); Konzeption, Vorgaben, Prüfung und Herausgeberschaft: Jürgen Maier.

Lizenz
Dieses Werk ist lizenziert unter einer Creative-Commons-Lizenz **Namensnennung – Weitergabe unter gleichen Bedingungen 4.0 International (CC BY-SA 4.0)**. Sie dürfen das Werk vervielfältigen, verbreiten, bearbeiten und auch kommerziell nutzen, sofern Sie den Urheber nennen und Bearbeitungen unter derselben Lizenz weitergeben.
Lizenztext: `https://creativecommons.org/licenses/by-sa/4.0/legalcode.de`

Quellen und Fassungen
Quelltexte, Fehlermeldungen und aktuelle Fassungen aller Bände:
`https://github.com/juergenmaier2026/informatik-von-grund-auf`
Dauerhaft archiviert bei Zenodo: `https://doi.org/10.5281/zenodo.23773283`
Schwesterreihe: *Mathematik von Grund auf*, `https://doi.org/10.5281/zenodo.23698547`

Zitiervorschlag
Maier, J. (2026). *Informatik von Grund auf, Band 2: Die Grenze der Maschine*. Fassung 1.0.1.
`https://doi.org/10.5281/zenodo.23773283`

Satz: LaTeX mit Palatino. Fassung 1.0.1, Oktober 2026. 176 Seiten.

Inhaltsverzeichnis

Vorwort **1**

I Sprachen und Automaten **9**

1 Wörter, Sprachen und das Abzählen **11**
1.1 Warum fast alles unentscheidbar ist 12

2 Endliche Automaten **19**
2.1 Nichtdeterminismus 20

3 Reguläre Ausdrücke und ihre Grenze **27**
3.1 Das Pumping-Lemma 29

4 Kontextfreie Sprachen **37**
4.1 Kellerautomaten und Mehrdeutigkeit 38

II Die Maschine **47**

5 Die Turingmaschine **49**
5.1 Robustheit des Modells 51

6 Die universelle Maschine **59**
6.1 Kodierung 59

7 Die Church-Turing-These **67**
7.1 Warum man sie trotzdem glaubt 67

III Die Grenze **77**

8 Das Halteproblem **79**
8.1 Der Beweis 79

9 Reduktionen und der Satz von Rice **87**
9.1 Ein durchgeführtes Beispiel 88
9.2 Der Satz von Rice 89

10 Aufzählbarkeit **97**

11 Grade der Unlösbarkeit **105**
11.1 Das Postsche Problem 107
11.2 Die arithmetische Hierarchie 108

IV Jenseits der Informatik **115**

12 Unentscheidbares in der Mathematik **117**
12.1 Das Postsche Korrespondenzproblem 117
12.2 Hilberts zehntes Problem 120

13 Gödels Unvollständigkeitssätze **129**

14 Kolmogorow-Komplexität **139**

V Aktueller Stand **149**

15 Wo die Theorie heute steht **151**
15.1 Warum die Komplexitätstheorie stecken bleibt 151
15.2 Was sich seit 2000 getan hat 152

VI Anhang **159**

A Übersicht und Formelsammlung **161**
A.1 Die große Tabelle 161
A.2 Die Beweistechniken 163
A.3 Die Landkarte des Bandes 165
A.4 Glossar 166

B Kommentierte Literatur **169**

Vorwort: Was keine Maschine kann

Dieser Band handelt von einem einzigen Satz und seinen Folgen. *Der Satz ist über neunzig Jahre alt, sein Beweis passt auf eine halbe Seite, und er hat mehr Konsequenzen als irgendein anderer Satz der Informatik.*

Der Satz

> ***Es gibt kein Programm, das für beliebige Programme entscheidet, ob sie anhalten.***

Das ist keine Aussage über heutige Rechner. *Kein schnellerer Prozessor, kein größerer Speicher, kein Quantenrechner und kein Verfahren, das noch niemand erfunden hat, ändert daran etwas.*

> *Der Satz gilt für* ***jede*** *Maschine, die man bauen kann* – **und für alle Zeiten.**

Warum das kein technisches Problem ist

Man könnte einwenden: Vielleicht ist das Problem nur schwer und nicht unmöglich.

> *Der Unterschied ist scharf.* **„Schwer" heißt: Man kennt kein schnelles Verfahren** – *so steht es bei* P *gegen* NP (Band 3). **„Unmöglich" heißt: Es ist *bewiesen*, dass kein Verfahren existiert.**

Das Halteproblem gehört zur zweiten Sorte. **Es ist keine offene Frage, sondern eine geschlossene** – *eine der wenigen der Informatik.*

Warum das jeden angeht, der programmiert

> *Die Unentscheidbarkeit klingt nach einer Kuriosität für Logiker.* ***Sie ist der Grund für die Beschaffenheit fast aller Werkzeuge, mit denen man täglich arbeitet.***

Frage	Antwort
Warum warnt der Übersetzer so oft?	Weil er nicht entscheiden kann, ob die Variable wirklich uninitialisiert ist – also warnt er vorsichtshalber.
Warum findet der Virenscanner nicht alles?	Weil „verhält sich bösartig“ eine Eigenschaft des *Verhaltens* ist – und die sind unentscheidbar.
Warum weist der Typprüfer korrekte Programme zurück?	Weil er konservativ sein **muss**: lieber ein korrektes Programm ablehnen als ein fehlerhaftes durchlassen.
Warum gibt es keine perfekte Optimierung?	Weil das Erkennen aller unerreichbaren Zweige das Halteproblem lösen würde.

Jede dieser Eigenschaften ist keine Nachlässigkeit der Entwickler, sondern eine mathematische Notwendigkeit. Wer den Satz von Rice kennt (Kapitel 9), weiß, dass es nicht besser geht – und hört auf, sich darüber zu ärgern.

Und die praktische Folgerung ist nicht Resignation, sondern Handwerk:

Ausweg 1	Die Antwort „weiß nicht“ zulassen
Ausweg 2	Nur eine Teilklasse von Programmen betrachten
Ausweg 3	Fehler in **eine** Richtung erlauben
Ausweg 4	Eine Zeitschranke setzen

Die gesamte statische Programmanalyse lebt von diesen vier Zeilen.

Was dieses Buch will

Erstens: Die Begriffe präzise machen.

Band 0 hat das Halteproblem in vier Zeilen bewiesen und dabei den Begriff „Programm“ umgangssprachlich benutzt. **Das war zulässig und ist unbefriedigend.** *Wer beweisen will, dass* ***kein*** *Verfahren existiert, muss vorher sagen, was ein Verfahren ist.*

Kapitel 5 definiert die Turingmaschine formal – *nicht weil sie ein brauchbares Rechenmodell wäre (sie ist entsetzlich unhandlich), sondern weil sie einfach genug ist, um über sie* Sätze *zu beweisen.*

Zweitens: Den Aufstieg über die Sprachklassen gehen.

Sprachklasse	Maschine	Gedächtnis
regulär	endlicher Automat	**keines**
kontextfrei	Kellerautomat	**Stapel**
kontextsensitiv	linear beschränkt	begrenztes Band
aufzählbar	**Turingmaschine**	unbegrenztes Band

Die rechte Spalte ist die eigentliche Aussage: *Die Hierarchie entsteht allein daraus, wie viel und wie frei die Maschine sich merken darf.*

Drittens: Zeigen, dass die Grenze eine *Struktur* hat.

Es gibt nicht bloß „entscheidbar“ und „unentscheidbar“. **Jenseits der Grenze liegt eine unendliche Hierarchie von Schwierigkeitsgraden**, *und Kapitel 11 steigt ein Stück hinein.*

Drei Warnungen

Erstens: Dieser Band verlangt mehr Geduld als die beiden vorigen.
Die Beweise sind nicht schwerer, aber sie sind ***Beweise*** *– und die Gegenstände sind abstrakt.*
Ich führe jeden Schritt aus und lasse nichts als „offensichtlich“ stehen.
Zweitens: Turingmaschinen sind ein Beweismittel, kein Programmiermodell.

> *Niemand programmiert Turingmaschinen, so wie niemand mit Peano-Axiomen addiert.* **Ihr Zweck ist, dass man über *alle* Programme reden kann, ohne sich auf eine Sprache festzulegen.**

Drittens: Unentscheidbar heißt nicht unlösbar *in der Praxis*.
Für konkrete Programme lässt sich Terminierung oft sehr wohl beweisen – Terminierungsprüfer sind heute erstaunlich stark (Kapitel 15). **Der Satz sagt nur, dass kein Verfahren *alle* Fälle richtig entscheidet.**

> *Der Unterschied zwischen „alle“ und „die, die vorkommen“ ist der ganze Spielraum, in dem sich die Praxis bewegt.*

Zum Aufbau

- **Teil I** (Kap. 1–4): Wörter und Sprachen; endliche Automaten, reguläre Ausdrücke, kontextfreie Grammatiken – und die Grenzen jeder dieser Klassen.
- **Teil II** (Kap. 5–7): Die Turingmaschine, die universelle Maschine und die Church-Turing-These.
- **Teil III** (Kap. 8–11): Das Halteproblem, Reduktionen, der Satz von Rice, Aufzählbarkeit und die Grade der Unlösbarkeit.
- **Teil IV** (Kap. 12–14): Unentscheidbares außerhalb der Informatik, Gödels Unvollständigkeitssätze und die Kolmogorow-Komplexität.
- **Teil V** (Kap. 15): Aktueller Stand, August 2026.

Ein Wettlauf im Jahr 1936

Im Frühjahr 1935 hörte Alan Turing in Cambridge eine Vorlesung von Max Newman über Grundlagen der Mathematik. *Newman erwähnte* Hilberts ***Entscheidungsproblem*** *von 1928: Gibt es ein „mechanisches Verfahren“, das für jede mathematische Aussage entscheidet, ob sie beweisbar ist?*

> *Turing war 22 und hatte das Problem gelöst, bevor er wusste, dass andere daran arbeiteten.* **Sein Beitrag war nicht die Antwort – die lautete „nein“ –, sondern die Frage, die er vorher stellte:** *„Was heißt eigentlich* mechanisches Verfahren*?“*

Seine Antwort ist die berühmteste Gedankenanalyse der Informatik. *Er betrachtete einen* ***Menschen****, der mit Bleistift und Papier rechnet, und fragte, welche Fähigkeiten dieser dabei wirklich braucht:*

1. Er sieht nur endlich viele Symbole auf einmal an.
2. Er kann sich nur endlich viel merken.
3. Er handelt nach endlich vielen festen Regeln.
4. Er kann Papier lesen, beschreiben und sich darauf bewegen.

> *Daraus baute er seine Maschine* – **und die Behauptung war nicht, dass sie ein guter Rechner sei, sondern dass sie *alles* kann, was ein Mensch mit Papier und Regeln kann.**

Turing reichte seine Arbeit im Mai 1936 ein. *Wenige Wochen zuvor, im April, war in Princeton eine Arbeit von* ALONZO CHURCH *erschienen, die dasselbe Ergebnis mit ganz anderen Mitteln bewies* – **mit dem Lambda-Kalkül.**

Newman musste Turing mitteilen, dass er zu spät war. **Die Arbeit erschien trotzdem**, *mit einem angehängten Beweis, dass beide Begriffe dasselbe bedeuten.*

Turing ging danach zu Church nach Princeton und promovierte bei ihm.

Die Nachwelt hat entschieden, wessen Zugang der überzeugendere war. Churchs Beweis ist ein formales Ergebnis über ein Kalkül. **Turings Beweis enthält eine *Begründung*, warum sein Modell der richtige Begriff von Berechenbarkeit ist** – *und das ist der Grund, warum wir heute von Turingmaschinen sprechen und nicht von Church-Termen.*

GÖDEL, der beide Arbeiten kannte, sagte es 1946 so: *Mit Turings Begriff habe man zum ersten Mal einen Begriff der Berechenbarkeit, der* ***nicht*** *von der gewählten Formalisierung abhänge* – **ein „absoluter" Begriff, wie er es nannte.**

Neunzig Jahre Berechenbarkeit

1900	Hilbert	23 Probleme; darunter das zehnte
1928	Hilbert, Ackermann	**Entscheidungsproblem** gestellt
1931	**Gödel**	**Unvollständigkeitssätze**
1933	Gödel, Herbrand	allgemein rekursive Funktionen
1936	**Church**	**Lambda-Kalkül; Unlösbarkeit**
1936	**Turing**	**Maschine, Universalität, Halteproblem**
1936	Post	unabhängig ein ähnliches Modell
1937	Turing	Äquivalenz von λ-Kalkül und Maschine
1938	Kleene	**Rekursionstheorem**
1943	Kleene	Normalformsatz; μ-Rekursion
1944	**Post**	**Grade der Unlösbarkeit; Postsches Problem**
1946	Post	**Postsches Korrespondenzproblem**
1947	Post, Markow	Wortproblem für Halbgruppen unlösbar
1951	Kleene	Arithmetische Hierarchie
1953	**Rice**	**Satz von Rice**
1955	Novikow	Wortproblem für Gruppen unlösbar
1956	Chomsky	**Sprachhierarchie**

1957	Friedberg, Muchnik	Postsches Problem gelöst (Priorität)
1959	Rabin, Scott	**nichtdeterministische Automaten**
1961	Myhill, Nerode	Charakterisierung regulärer Sprachen
1964/65	Solomonoff, Kolmogorow	**algorithmische Komplexität**
1966	Chaitin	unabhängig dasselbe
1970	Matijassewitsch	**Hilberts zehntes Problem unlösbar**
1975	Chaitin	Unvollständigkeit aus Informationstheorie
1975	Baker, Gill, Solovay	Relativierungsschranke
2016	Yedidia, Aaronson	Turingmaschine, die ZFC entscheidet
2020	Ji u. a.	$\mathrm{MIP}^* = \mathrm{RE}$; Connes-Vermutung falsch
2024	**bbchallenge**	$\Sigma(5) = 4098$ bewiesen und verifiziert

Zwei Zeilen verdienen Hervorhebung.
1944, Post. EMIL POST *fragte, ob es zwischen den entscheidbaren Problemen und dem Halteproblem noch etwas gibt – Schwierigkeitsgrade* dazwischen. **Die Frage blieb dreizehn Jahre offen und führte zu einer eigenen Beweistechnik** (Kapitel 11).
2020, Ji und Mitarbeiter. *Ein Ergebnis über Quantenverschränkung entschied nebenbei eine Vermutung aus der Operatoralgebra von 1976* – **über den Umweg der Berechenbarkeitstheorie.** *Ein Beleg dafür, dass dieses Gebiet noch lebt.*

Wir können nur eine kurze Strecke voraussehen,
aber wir sehen dort viel, was getan werden muss.

— ALAN TURING (1912–1954).
Derselbe Satz stand am Ende von Band 0.
Er steht hier wieder, weil dieser Band
das ausführt, was dort begann.

Part I

Sprachen und Automaten

1 Wörter, Sprachen und das Abzählen

Alles ist ein Wort

*Um über **alle** Probleme reden zu können, braucht man eine einheitliche Form.*
Sie lautet: Ein Problem ist eine Menge von Wörtern.

Definition 1.1. Ein **Alphabet** Σ ist eine endliche, nichtleere Menge von **Zeichen**. Ein **Wort** über Σ ist eine endliche Folge von Zeichen; das **leere Wort** heißt ε. Die Menge aller Wörter heißt Σ^*. Eine **Sprache** ist eine beliebige Teilmenge $L \subseteq \Sigma^*$.

Zeichen	Bedeutung	Beispiel
Σ	Alphabet	$\{0,1\}$, $\{a,\dots,z\}$
w	Wort	0110
$\lvert w\rvert$	Länge	$\lvert 0110\rvert = 4$
ε	leeres Wort	$\lvert\varepsilon\rvert = 0$
Σ^*	alle Wörter	$\{\varepsilon, 0, 1, 00, 01, \dots\}$
L	Sprache	$\{w :$ w hat gerade viele $1\}$

Σ^* ist unendlich, jedes Wort ist endlich

Diese beiden Aussagen zusammen sind der Ausgangspunkt des ganzen Bandes.

*Σ^* enthält unendlich viele Wörter – aber **kein einziges unendliches Wort**.*
Es gibt kein Wort $000\dots$ **ohne Ende.**

*Der Unterschied wird gleich entscheidend: Σ^* ist **abzählbar**, die Menge aller **Sprachen** über Σ ist es nicht.*

Warum jedes Problem so darstellbar ist.

Frage	**Ist n eine Primzahl?**
als Sprache	$L =$ {Binärdarstellungen von Primzahlen}
Frage	**Ist dieser Graph zusammenhängend?**
als Sprache	$L =$ {Kodierungen zusammenhängender Graphen}
Frage	**Hält dieses Programm?**
als Sprache	$L = \{\langle P \rangle\, w : P \text{ hält bei Eingabe } w\}$

Die Beschränkung auf Ja-Nein-Fragen ist keine echte Einschränkung. Wer einen Wert berechnen will, kann statt dessen fragen: „Ist das i-te Bit des Ergebnisses eine 1?“ – **und den Wert Bit für Bit erfragen.**

1.1 Warum fast alles unentscheidbar ist

Ein Abzählargument, das alles vorwegnimmt

*Bevor wir eine einzige Maschine definieren, lässt sich schon zeigen, dass es unentscheidbare Sprachen **geben muss** –* **und zwar überwältigend viele.**

Satz 1.2. *Die Menge aller Programme ist abzählbar. Die Menge aller Sprachen über $\{0,1\}$ ist überabzählbar. Also gibt es Sprachen, die kein Programm entscheidet.*

Schritt 1 – Σ^* ist abzählbar.
Man ordnet die Wörter zuerst nach Länge und innerhalb gleicher Länge lexikographisch (**längenlexikographische Ordnung**):

Länge 0	ε
Länge 1	0, 1
Länge 2	00, 01, 10, 11
Länge 3	000, 001, 010, 011, 100, …

*Diese Liste erfasst **jedes** Wort nach endlich vielen Schritten –* **denn zu jeder Länge gibt es nur endlich viele Wörter.** ✓

Die rein lexikographische Ordnung versagt

Ordnet man nur lexikographisch – $\varepsilon, 0, 00, 000, \ldots$ **–, so kommt man bei der 1 nie an.**

Vor ihr stehen unendlich viele Wörter. **Die Abzählung bricht ab, bevor sie das zweite Zeichen erreicht.**

Deshalb ist die ***Längenschichtung*** *wesentlich, nicht bloß bequem.* **Dasselbe Muster begegnet uns bei jeder Abzählung unendlicher Mengen** (Mathematik-Band 10, Kapitel 5).

Schritt 2 – Die Menge der Programme ist abzählbar.

Ein Programm ist ein ***endlicher Text*** *über einem endlichen Zeichensatz –* **also ein Wort über einem Alphabet.** *Nach Schritt 1 sind das abzählbar viele.* ✓

Das gilt für jede Programmiersprache: C, Python, Turingmaschinen, was auch immer. **Solange ein Programm ein endlicher Text ist, gibt es nur abzählbar viele.**

Schritt 3 – Die Menge der Sprachen ist überabzählbar.
Eine Sprache über $\{0,1\}$ *ist eine* ***Teilmenge*** *von* Σ^*. *Da* Σ^* *abzählbar unendlich ist, entspricht jede Teilmenge einer unendlichen Bitfolge:*

Wort	ε	0	1	00	$\cdots$
in L?	1	0	1	1	$\cdots$

Also entsprechen die Sprachen genau den unendlichen Bitfolgen, und deren Menge ist nach CANTORS *Diagonalargument überabzählbar* (Mathematik-Band 10, Kapitel 5).
Der Beweis, ausgeführt.
Angenommen, man könnte alle Sprachen als $L_1, L_2, L_3, \ldots$ *auflisten. Man bilde*

$$D := \{w_i : w_i \notin L_i\},$$

wobei w_i *das i-te Wort in der Abzählung aus Schritt 1 ist.*

D unterscheidet sich von ***jedem*** L_i *an der Stelle* w_i*: Ist* $w_i \in L_i$*, so ist* $w_i \notin D$*, und umgekehrt.* **Also steht** D **in keiner Zeile der Liste** *– Widerspruch.*

Schritt 4 – Der Schluss.

	Anzahl	**Mächtigkeit**
Programme	abzählbar	$\aleph_0$
Sprachen	**überabzählbar**	$2^{\aleph_0}$

Es gibt überabzählbar viele Sprachen, die kein Programm entscheidet.

Genauer: Die entscheidbaren Sprachen sind eine* verschwindend kleine *Teilmenge aller Sprachen. Wählte man eine Sprache zufällig, so wäre sie mit Wahrscheinlichkeit 1 unentscheidbar.

Was dieses Argument leistet – und was nicht

Es beweist die *Existenz* unentscheidbarer Sprachen. *Es nennt keine einzige.*

> *Der Beweis ist **nichtkonstruktiv**: Er zeigt, dass es mehr Sprachen als Programme gibt, aber er zeigt nicht auf eine Sprache und sagt „diese hier".*

Und die Sprachen, die er liefert, wären auch uninteressant – *die diagonal konstruierte Sprache D hängt von einer willkürlich gewählten Abzählung ab und beschreibt kein Problem, das jemanden beschäftigt.*

*Was man braucht, ist eine **konkrete, natürliche** unentscheidbare Sprache* – **und die liefert Kapitel 8 mit dem Halteproblem.**

> *Dasselbe Muster gibt es in der Analysis: Cantor zeigte 1874, dass fast alle reellen Zahlen transzendent sind, **ohne eine einzige anzugeben** – während* LIOUVILLE *dreißig Jahre zuvor mühsam eine konstruiert hatte* (Mathematik-Band 1).

Operationen auf Sprachen

Operation	Definition	Beispiel
Vereinigung	$L_1 \cup L_2$	
Schnitt	$L_1 \cap L_2$	
Komplement	$\overline{L} = \Sigma^* \setminus L$	
Verkettung	$L_1 L_2 = \{uv\}$	$\{a\}\{b,c\} = \{ab, ac\}$
Kleene-Stern	$L^* = \bigcup_{k\geq 0} L^k$	$\{a\}^* = \{\varepsilon, a, aa, \dots\}$

*Der **Kleene-Stern** verdient Aufmerksamkeit:* $L^0 = \{\varepsilon\}$ *per Definition* – **also enthält** L^* ***immer* das leere Wort**, *auch wenn* L *selbst leer ist.* **Es gilt** $\emptyset^* = \{\varepsilon\}$**.**

STEPHEN KLEENE *führte den Operator 1951 ein; er war Churchs Doktorand und hat der Berechenbarkeitstheorie mehr Begriffe gegeben als irgendjemand sonst.*

Aufgabe 1.1.

(a) Zählen Sie Wörter und Sprachen ab und bestimmen Sie die Mächtigkeiten genau.

(b) Untersuchen Sie den Unterschied zwischen abzählbar und überabzählbar an konkreten Mengen.

(c) Zeigen Sie, dass es Sprachen gibt, die kein Programm auch nur *beschreiben* kann.

Lösung Schritt für Schritt.

(a) Abzählungen genau.
Teil 1 – Wie viele Wörter der Länge n?

$$|\Sigma^n| = |\Sigma|^n .$$

Bei $|\Sigma| = 2$*:*

n	2^n	**kumuliert**
0	1	1
1	2	3
2	4	7
3	8	15
n	2^n	$2^{n+1} - 1$

Die kumulierte Spalte ist die Nummerierung. Das k-te Wort in längen-lexikographischer Ordnung erhält man, indem man $k + 1$ binär schreibt und das führende Bit streicht.

Probe für $k = 4$*:* $k + 1 = 5 = (101)_2$*; führendes Bit streichen ergibt* 01*. Und tatsächlich ist die Liste* $\varepsilon, 0, 1, 00, 01, \ldots$ *– das Wort mit Index 4 ist* 01*.* ✓

Diese Bijektion zwischen $\mathbb{N}$ *und* Σ^* *ist bequem und wird im ganzen Band benutzt.*

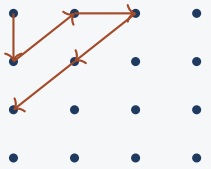
Teil 2 – Die Mächtigkeiten im Vergleich.

Menge	Mächtigkeit	abzählbar?
Σ^n (feste Länge)	endlich	trivial
Σ^*	$\aleph_0$	ja
alle Programme	$\aleph_0$	ja
alle **endlichen** Sprachen	$\aleph_0$	ja
alle Sprachen	$2^{\aleph_0}$	**nein**
$\mathbb{R}$	$2^{\aleph_0}$	nein

Warum die endlichen Sprachen abzählbar sind.
Eine endliche Sprache ist eine endliche Menge endlicher Wörter – also selbst durch einen endlichen Text beschreibbar, etwa $\{01, 110, \varepsilon\}$. **Endliche Texte sind abzählbar.** ✓

> *Der Sprung ins Überabzählbare geschieht erst bei* ***unendlichen*** *Teilmengen –* **und dort auch nur, weil man beliebige zulässt.**

(b) Abzählbar gegen überabzählbar.
Fall 1 – $\mathbb{N} \times \mathbb{N}$ **ist abzählbar.**
Man zählt diagonal:



Cantorsche Paarungsfunktion

$$\pi(x, y) = \frac{(x+y)(x+y+1)}{2} + y.$$

Probe: $\pi(0,0) = 0$, $\pi(0,1) = 2$, $\pi(1,0) = 1$, $\pi(1,1) = 4$. ✓

> ***Diese Funktion ist eine Bijektion*** $\mathbb{N}^2 \to \mathbb{N}$ – und sie wird in Kapitel 6 gebraucht, um Programm und Eingabe zu **einer** Zahl zusammenzufassen.

Fall 2 – $\mathbb{Q}$ **ist abzählbar.**
Jeder Bruch $\frac{p}{q}$ *ist ein Paar; nach Fall 1 abzählbar.* ✓

Bemerkenswert: Zwischen je zwei rationalen Zahlen liegen unendlich viele weitere – und trotzdem lassen sie sich in einer Reihe aufzählen. **Dichte und Abzählbarkeit haben nichts miteinander zu tun.**

Fall 3 – Die berechenbaren reellen Zahlen.

Definition 1.3. Eine reelle Zahl heißt **berechenbar**, wenn es ein Programm gibt, das auf Eingabe n die n-te Nachkommastelle ausgibt.

berechenbar	π, e, $\sqrt{2}$, jede rationale Zahl, $\sum 2^{-n!}$
nicht berechenbar	**fast alle** reellen Zahlen

Denn: Es gibt nur abzählbar viele Programme, also nur abzählbar viele berechenbare Zahlen **– aber überabzählbar viele reelle.** ✓

> ***Das war Turings eigentliches Thema.*** Seine Arbeit heißt „On Computable Numbers"; das Halteproblem war ein Hilfsmittel, um zu zeigen, dass die berechenbaren Zahlen nicht alle sind.

Ein merkwürdiger Nachsatz.

> *Man kann keine einzige unberechenbare reelle Zahl* ***hinschreiben*** **– denn jede Beschreibung wäre ein endlicher Text, und aus einem endlichen Text, der die Zahl eindeutig festlegt, ließe sich meist ein Verfahren gewinnen.**

Kapitel 14 stellt trotzdem eine vor: Chaitins Zahl Ω **– sie ist wohldefiniert, unberechenbar, und man kennt einige ihrer ersten Bits.**

(c) Sprachen, die niemand beschreiben kann.

> *Die Aussage aus Satz 1.2 lässt sich verschärfen.* **Es gibt Sprachen, für die es nicht einmal eine *endliche Beschreibung* gibt** *– gleichgültig in welcher Sprache, mit welchem Formalismus.*

Schritt 1 – Was eine Beschreibung ist.
Eine Beschreibung ist ein endlicher Text über einem festen, endlichen Zeichensatz – deutsche Prosa, mathematische Formeln, Programmcode, alles zusammen.
Schritt 2 – Der Schluss.

endliche Texte	abzählbar	$\aleph_0$
Sprachen	überabzählbar	$2^{\aleph_0}$

Fast alle Sprachen haben **keine** endliche Beschreibung.

Ein Paradoxon, das keines ist

Man ist versucht zu sagen: *„Sei L die erste Sprache, die keine endliche Beschreibung hat.“*

*Aber dieser Satz **ist** eine endliche Beschreibung* – **Widerspruch.**

*Das ist eine Variante des **Berry-Paradoxons**: „Die kleinste natürliche Zahl, die sich nicht mit weniger als dreizehn Wörtern beschreiben lässt“* – **ein Satz mit zwölf Wörtern.**

Die Auflösung: *Der Begriff „beschreibbar“ ist nicht selbst in der Sprache formalisierbar, in der beschrieben wird.* **Wer ihn formalisiert, muss eine *festgelegte* Beschreibungssprache wählen – und der Satz redet dann über eine andere.**

TARSKI *zeigte 1933 die allgemeine Fassung dieses Befunds:* ***Wahrheit ist in der Sprache selbst nicht definierbar*** (Kapitel 13).

Die Ordnung „die erste“ ist übrigens auch problematisch: *Die Sprachen sind überabzählbar und nicht auf natürliche Weise angeordnet, also gibt es keine „erste“.*

Berry umgeht das, indem er über natürliche Zahlen redet, die wohlgeordnet sind. **Genau deshalb ist sein Paradoxon schärfer – und Chaitin hat 1974 daraus einen vollwertigen Beweis des Unvollständigkeitssatzes gemacht** (Kapitel 14).

Sipser[24] führt die Grundbegriffe besonders sauber ein; Hopcroft, Motwani und Ullman[10] sind die Referenz für formale Sprachen; Schöning[23] und Wegener[28] sind die knappen deutschsprachigen Standardwerke.

2 Endliche Automaten

Die einfachste denkbare Maschine

Ein endlicher Automat liest ein Wort von links nach rechts und hat dabei ***nichts als endlich viele Zustände*** **– kein Band, keinen Stapel, keinen Zähler.** *Was er sich merken will, muss in seinen Zustand passen.*

Definition 2.1. Ein **deterministischer endlicher Automat** (DEA) ist ein Tupel

$$A = (Q,\ \Sigma,\ \delta,\ q_0,\ F)$$

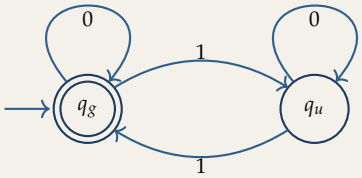
mit einer endlichen Zustandsmenge Q, einem Alphabet Σ, einer Übergangsfunktion $\delta : Q \times \Sigma \to Q$, einem Startzustand $q_0 \in Q$ und einer Menge $F \subseteq Q$ akzeptierender Zustände.

Die Fortsetzung auf Wörter:

$$\hat{\delta}(q, \varepsilon) = q, \qquad \hat{\delta}(q, wa) = \delta\big(\hat{\delta}(q, w),\ a\big).$$

$$L(A) := \{w \in \Sigma^* : \hat{\delta}(q_0, w) \in F\}.$$

Ein Beispiel. *Der Automat akzeptiert alle Wörter über* $\{0, 1\}$ *mit gerader Anzahl von Einsen.*



Der Automat hat zwei Zustände, und das genügt – denn um die Parität zu kennen, braucht man nicht die Anzahl der Einsen, sondern nur ihren Rest modulo 2.

Das Entscheidende ist, was er *nicht* kann

Kein endlicher Automat kann zählen – jedenfalls nicht unbeschränkt.

> *Die Sprache* $\{0^n 1^n : n \geq 0\}$ *– gleich viele Nullen wie Einsen – ist* ***nicht*** *regulär.* **Um sie zu erkennen, müsste die Maschine sich die Zahl** n **merken, und** n **ist unbeschränkt.**

Kapitel 3 beweist das mit dem Pumping-Lemma. **Und genau diese Grenze ist der Grund, warum Klammerstrukturen nicht mit regulären Ausdrücken geprüft werden können** *– ein Punkt, der in der Praxis regelmäßig übersehen wird.*

2.1 Nichtdeterminismus

Eine Maschine, die raten darf

Definition 2.2. Ein **nichtdeterministischer endlicher Automat** (NEA) hat statt $\delta : Q \times \Sigma \to Q$ eine Relation

$$\delta : Q \times \Sigma \to \mathcal{P}(Q),$$

liefert also eine **Menge** möglicher Nachfolgezustände. Er akzeptiert w, wenn **mindestens ein** Berechnungsweg in einem akzeptierenden Zustand endet.

> *Man kann sich das auf zwei Weisen vorstellen:* **als eine Maschine, die bei jedem Schritt rät und immer richtig rät, wenn es einen richtigen Weg gibt** *– oder als eine Maschine, die* ***alle*** *Wege gleichzeitig verfolgt.*
>
> *Die zweite Vorstellung ist die nützlichere, denn sie führt direkt zum Beweis.*

Satz 2.3 (Rabin und Scott, 1959)**.** *Zu jedem NEA mit n Zuständen gibt es einen DEA mit höchstens* 2^n *Zuständen, der dieselbe Sprache akzeptiert.*

Der Beweis – die Potenzmengenkonstruktion.

> ***Die Idee in einem Satz:*** *Der deterministische Automat merkt sich die* ***Menge*** *aller Zustände, in denen der nichtdeterministische gerade sein könnte.*

Aus $A = (Q, \Sigma, \delta, q_0, F)$ *baue man*

$Q' := \mathcal{P}(Q)$	Zustände sind **Teilmengen**
$q_0' := \{q_0\}$	
$\delta'(S,a) := \bigcup_{q\in S} \delta(q,a)$	alle erreichbaren Zustände
$F' := \{S : S \cap F \neq \emptyset\}$	akzeptiert, wenn **ein** Zustand akzeptiert

Per Induktion über $|w|$ zeigt man

$$\hat{\delta}'(\{q_0\},\ w) = \{q : q \text{ ist im NEA über } w \text{ erreichbar}\}.$$

Der Induktionsschritt ist genau die Definition von δ'. ✓ **Damit stimmen die akzeptierten Sprachen überein.**

Nichtdeterminismus bringt bei endlichen Automaten also keine zusätzliche Mächtigkeit – nur zusätzliche **Kürze**. *Beide erkennen genau die regulären Sprachen.*

Die Zustandsexplosion ist nicht vermeidbar

Die Schranke 2^n sieht großzügig aus. Sie ist scharf.
Betrachte über $\{0,1\}$ die Sprache

$$L_n := \{w : \text{das } n\text{-te Zeichen von rechts ist eine } 1\}.$$

	Zustände	**warum**
NEA	$n+1$	rät, wo die Stelle ist
DEA	$\mathbf{2^n}$	muss die letzten n Zeichen **alle** kennen

Der Nachweis der unteren Schranke: Zwei verschiedene Wörter der Länge n müssen in verschiedene Zustände führen – **sonst könnte man sie durch eine passende Fortsetzung unterscheiden.** *Es gibt 2^n solche Wörter.* ✓

Das ist der praktische Grund, warum Werkzeuge für reguläre Ausdrücke oft nichtdeterministisch simulieren, statt zu determinisieren.
Der Aufbau des DEA kann exponentiell viel Speicher kosten.

Zwei Nobelpreisträger der Informatik und ein Aufsatz von 1959

Michael Rabin und Dana Scott waren 1957 als junge Forscher bei IBM. *Ihr gemeinsamer Aufsatz „Finite Automata and Their Decision Problems“ erschien 1959 und führte den Nichtdeterminismus in die Automatentheorie ein.*

> *Der Begriff war zunächst rein technisch gemeint – eine Beweiserleichterung.* **Dass er das Zentralthema der Komplexitätstheorie werden würde (**P **gegen** NP**), ahnte damals niemand.**

Beide erhielten 1976 gemeinsam den Turing-Award – *ausdrücklich für diese Arbeit.* **In der Begründung hieß es, der Begriff des Nichtdeterminismus habe sich als „außerordentlich fruchtbar“ erwiesen.**

Rabin wandte sich später der Kryptographie zu (Rabin-Verschlüsselung, Miller-Rabin-Primzahltest), Scott der Semantik von Programmiersprachen – **die Bereichstheorie, die er mit Christopher Strachey entwickelte, ist die Grundlage der denotationellen Semantik** (Band 6).

Bemerkenswert: Der Nichtdeterminismus bei endlichen Automaten kostet nichts an Mächtigkeit und exponentiell an Zuständen. **Bei Turingmaschinen kostet er nichts an Mächtigkeit und – vermutlich – exponentiell an *Zeit*.** *Das ist* P *gegen* NP.

Aufgabe 2.1.

(a) Konstruieren Sie Automaten für konkrete Sprachen und begründen Sie die Zustandszahl.

(b) Führen Sie die Potenzmengenkonstruktion an einem Beispiel vollständig durch.

(c) Zeigen Sie, dass die regulären Sprachen unter allen Booleschen Operationen abgeschlossen sind.

Lösung Schritt für Schritt.

(a) Automaten konstruieren.
Sprache 1 – Vielfache von 3 in Binärdarstellung.
Der Automat liest die Bits von links nach rechts. Beim Anhängen eines Bits b gilt

$$n_{\text{neu}} = 2n + b \Longrightarrow n_{\text{neu}} \bmod 3 = (2n + b) \bmod 3.$$

Zustand	**bei** 0	**bei** 1
r_0	$2 \cdot 0 + 0 = 0$	$2 \cdot 0 + 1 = 1$
r_1	$2 \cdot 1 + 0 = 2$	$2 \cdot 1 + 1 = 0$
r_2	$2 \cdot 2 + 0 = 1$	$2 \cdot 2 + 1 = 2$

Start und einziger akzeptierender Zustand: r_0. **Drei Zustände genügen.**

Probe mit $110 = 6$*:*

$$r_0 \xrightarrow{1} r_1 \xrightarrow{1} r_0 \xrightarrow{0} r_0. \qquad \checkmark$$

Und drei sind auch nötig – die Wörter ε, 1, 10 (also $0, 1, 2$) müssen paarweise verschiedene Zustände erreichen, sonst könnte man sie nicht unterscheiden. ✓

Für Teilbarkeit durch k genügen stets k Zustände – **unabhängig davon, wie groß die Zahlen werden.** *Ein hübsches Beispiel dafür, dass „endlich viele Zustände“ nicht „nur kleine Zahlen“ bedeutet.*

Sprache 2 – Enthält `abc` als Teilwort.

s_0	noch nichts passendes gesehen
s_1	gerade `a` gesehen
s_2	gerade `ab` gesehen
s_3	**gefunden** (akzeptierend, Falle)

Der Rücksprung ist die Fehlerquelle

Was geschieht in s_2 bei einem `a`?
Man ist versucht, nach s_0 zu gehen. **Falsch** – *das gelesene `a` kann der Anfang eines neuen Vorkommens sein.* **Richtig ist s_1.**

> ***Genau diese Rücksprünge systematisch zu berechnen, ist der Kern des Knuth-Morris-Pratt-Algorithmus zur Textsuche*** – er baut nichts anderes als diesen Automaten, in $\mathcal{O}(m)$ statt $\mathcal{O}(m\,|\Sigma|)$.

Sprache 3 – Gleich viele `a` wie `b`.

Nicht mit endlich vielen Zuständen möglich – **Kapitel 3 beweist es.**

(b) Potenzmengenkonstruktion.
NEA für „das zweitletzte Zeichen ist eine 1“:

Zustand	**bei** 0	**bei** 1
p_0 (Start)	$\{p_0\}$	$\{p_0, p_1\}$
p_1	$\{p_2\}$	$\{p_2\}$
p_2 (Endzust.)	$\emptyset$	$\emptyset$

In p_0 „rät" der Automat bei einer 1, ob dies die entscheidende Stelle ist.

Die Konstruktion, Zustand für Zustand.

Menge	**bei** 0	**bei** 1
$\{p_0\}$	$\{p_0\}$	$\{p_0, p_1\}$
$\{p_0, p_1\}$	$\{p_0, p_2\}$	$\{p_0, p_1, p_2\}$
$\{p_0, p_2\}$	$\{p_0\}$	$\{p_0, p_1\}$
$\{p_0, p_1, p_2\}$	$\{p_0, p_2\}$	$\{p_0, p_1, p_2\}$

Von den $2^3 = 8$ ***möglichen Teilmengen sind nur 4 erreichbar.*** Die anderen – etwa $\{p_1\}$ oder $\emptyset$ – kommen nie vor, weil p_0 nie verlassen wird.

Akzeptierend sind die Mengen, die p_2 enthalten: $\{p_0, p_2\}$ und $\{p_0, p_1, p_2\}$.
Probe mit $w = 010$: das zweitletzte Zeichen ist eine 1, also soll akzeptiert werden.

$$\{p_0\} \xrightarrow{0} \{p_0\} \xrightarrow{1} \{p_0, p_1\} \xrightarrow{0} \{p_0, p_2\}. \qquad \checkmark$$

Probe mit $w = 011$: das zweitletzte Zeichen ist eine 1.

$$\{p_0\} \xrightarrow{0} \{p_0\} \xrightarrow{1} \{p_0, p_1\} \xrightarrow{1} \{p_0, p_1, p_2\}. \qquad \checkmark$$

Probe mit $w = 001$: das zweitletzte Zeichen ist eine 0.

$$\{p_0\} \xrightarrow{0} \{p_0\} \xrightarrow{0} \{p_0\} \xrightarrow{1} \{p_0, p_1\}. \qquad \text{nicht akzeptierend } \checkmark$$

Der entstandene DEA hat 4 Zustände – genau 2^2*, wie die Schranke aus dem Kasten oben vorhersagt.* **Er merkt sich effektiv die letzten zwei gelesenen Zeichen.**

(c) Abschlusseigenschaften.
Komplement – der einfachste Fall.
Man vertausche akzeptierende und nicht akzeptierende Zustände:

$$\overline{A} := (Q, \Sigma, \delta, q_0,\ Q \setminus F).$$

Da der Automat ***deterministisch*** *und* ***total*** *ist, endet jede Eingabe in genau einem Zustand –* **also gilt** $L(\overline{A}) = \overline{L(A)}$**.** ✓

Beim NEA geht das nicht

Vertauscht man beim nichtdeterministischen Automaten die Endzustände, so erhält man *nicht* das Komplement.

> *Der NEA akzeptiert, wenn **ein** Weg zum Erfolg führt.* **Nach dem Vertauschen akzeptiert er, wenn ein Weg in einem *vorher* nicht akzeptierenden Zustand endet** – *und das ist etwas anderes als „kein Weg führt zum Erfolg".*

Man muss also erst deterministieren – **was exponentiell kosten kann.** *Genau diese Asymmetrie ist der Grund, warum die Frage* NP = co-NP *offen ist* (Band 3).

Schnitt – die Produktkonstruktion.

> *Man lässt **beide** Automaten gleichzeitig laufen und merkt sich das Paar der Zustände.*

$$A_1 \times A_2 := (Q_1 \times Q_2,\ \Sigma,\ \delta,\ (q_{01}, q_{02}),\ F_1 \times F_2)$$

mit

$$\delta\big((p,q),\ a\big) := \big(\delta_1(p,a),\ \delta_2(q,a)\big).$$

*Akzeptiert wird, wenn **beide** Komponenten akzeptieren.* **Zustandszahl:** $|Q_1| \cdot |Q_2|$**.** ✓

Vereinigung – dieselbe Konstruktion, andere Endzustandsmenge.

$$F := (F_1 \times Q_2) \cup (Q_1 \times F_2).$$

*Akzeptiert, wenn **mindestens eine** Komponente akzeptiert.* ✓

Ein durchgerechnetes Beispiel.
L_1*: gerade viele Nullen (2 Zustände* g, u*).* L_2*: Vielfache von 3 (3 Zustände* r_0, r_1, r_2*).*

Der Produktautomat hat $2 \cdot 3 = 6$ *Zustände.*

Zustand	**bei** 0	**bei** 1
(g, r_0)	(u, r_0)	(g, r_1)
(g, r_1)	(u, r_2)	(g, r_0)
(u, r_0)	(g, r_0)	(u, r_1)
(u, r_1)	(g, r_2)	(u, r_0)

Für den Schnitt ist (g, r_0) *der einzige akzeptierende Zustand; für die Vereinigung sind es alle mit erster Komponente* g oder *zweiter Komponente* r_0 *– also vier.*
Die Bilanz.

Operation	abgeschlossen?	Zustandszahl
Komplement	ja	n (unverändert)
Schnitt	ja	$n_1 \cdot n_2$
Vereinigung	ja	$n_1 \cdot n_2$
Differenz	ja	$n_1 \cdot n_2$
Verkettung	ja	bis $2^{n_1+n_2}$
Kleene-Stern	ja	bis 2^n
Spiegelung	ja	bis 2^n

> ***Die regulären Sprachen sind unter allem abgeschlossen, was man sich wünschen kann*** – eine Boolesche Algebra mit zusätzlicher Struktur. **Das macht sie so handlich und ist der Grund, warum Entscheidbarkeitsfragen für sie sämtlich lösbar sind.**

Man vergleiche mit den kontextfreien Sprachen (Kapitel 4): **Sie sind unter Vereinigung abgeschlossen, aber *nicht* unter Schnitt und *nicht* unter Komplement** – *und prompt sind dort fast alle Entscheidungsprobleme unlösbar.*

> *Das ist ein Muster, das sich durch den ganzen Band zieht:* **Je mächtiger eine Sprachklasse, desto weniger lässt sich über sie entscheiden.** *Der Höhepunkt ist der Satz von Rice* (Kapitel 9) – **bei der mächtigsten Klasse ist *gar nichts* mehr entscheidbar.**

Rabin und Scotts Arbeit von 1959 ist bis heute lesenswert; Hopcroft, Motwani und Ullman[10] sowie Kozen[13] entwickeln die Automatentheorie mit vielen Beispielen; Minsky[16] verbindet sie mit der Berechenbarkeit.

3 Reguläre Ausdrücke und ihre Grenze

Dieselbe Klasse, andere Beschreibung

Definition 3.1. Reguläre Ausdrücke über Σ sind induktiv definiert:

Ausdruck	Sprache	Bemerkung
$\emptyset$	$\emptyset$	Grundfall
ε	$\{\varepsilon\}$	Grundfall
$a \in \Sigma$	$\{a\}$	Grundfall
$r_1 \mid r_2$	$L(r_1) \cup L(r_2)$	Alternative
$r_1 r_2$	$L(r_1)L(r_2)$	Verkettung
r^*	$L(r)^*$	Wiederholung

Satz 3.2 (Kleene, 1951)**.** *Eine Sprache wird genau dann von einem endlichen Automaten akzeptiert, wenn sie durch einen regulären Ausdruck beschrieben wird.*

> ***Das ist ein bemerkenswerter Satz.*** Zwei völlig verschiedene Zugänge – eine **Maschine** und eine **Grammatik** – liefern genau dieselbe Sprachklasse.

Beweisrichtung 1: Ausdruck → Automat.
Induktion über den Aufbau des Ausdrucks. Für jeden Fall gibt man einen NEA an:

a	Zwei Zustände, ein Übergang
$r_1 \mid r_2$	Neuer Startzustand mit ε-Übergängen in beide Automaten
$r_1 r_2$	Endzustände von A_1 per ε-Übergang an den Start von A_2
r^*	Neuer Start-/Endzustand; Rückkanten von den Enden zum Anfang

Man nennt das die ***Thompson-Konstruktion*** **– nach KEN THOMPSON, der sie 1968 für die Textsuche in Unix beschrieb.** ✓

Beweisrichtung 2: Automat → Ausdruck.

> *Man eliminiert die Zustände nacheinander und beschriftet die Kanten dabei mit immer komplizierteren regulären Ausdrücken.*

Wird Zustand q entfernt, so ersetzt man für jedes Paar (p, r) von verbleibenden Zuständen die Beschriftung durch

$$R_{pr} \mid R_{pq} (R_{qq})^* R_{qr}.$$

In Worten: „direkt von p nach r, oder über q – dabei darf man beliebig oft in q kreisen“. ✓

Der resultierende Ausdruck kann riesig werden

Jede Elimination kann die Ausdruckslänge quadrieren.

> *Bei n Zuständen kann der Ausdruck die Länge $2^{\Theta(n)}$ erreichen* **– und EHRENFEUCHT und ZEIGER zeigten 1976, dass das unvermeidlich ist.**
>
> *Automat und regulärer Ausdruck sind also* ***gleichmächtig, aber nicht gleich kompakt*** – ein Muster, das uns beim Nichtdeterminismus schon begegnet ist.

Was in der Praxis „regulärer Ausdruck“ heißt, ist keiner

Die Werkzeuge in Perl, Python, Java und Javascript bieten Erweiterungen, die die reguläre Klasse verlassen.

Rückverweise `\1`	Erkennt $\{ww\}$ – **nicht regulär, nicht einmal kontextfrei**
Lookahead	bleibt regulär, aber verkompliziert
Rekursive Muster	erkennt Klammerstrukturen – kontextfrei

Der Preis ist die Laufzeit. *Echte reguläre Ausdrücke lassen sich in $\mathcal{O}(n)$ prüfen – man simuliert einfach den NEA* (THOMPSONS Verfahren). **Mit Rückverweisen ist das Problem NP-vollständig**, *und die üblichen Implementierungen benutzen Rückverfolgung.*

Daraus entsteht eine reale Angriffsform: ***ReDoS*** (regular expression denial of service). **Ein Ausdruck wie `(a+)+b` braucht auf der Eingabe `aaaa...a` exponentielle Zeit.**

Am 8. Juli 2019 fiel CLOUDFLARE *weltweit für 27 Minuten aus.* **Ursache war eine einzelne Regel in der Web Application Firewall mit einem Ausdruck, der katastrophale Rückverfolgung auslöste** – *die Prozessorlast stieg auf 100 Prozent in allen Rechenzentren gleichzeitig.*

Die Werkzeuge RE2 *(Google) und die Regex-Bibliothek von Go verzichten deshalb bewusst auf Rückverweise* – **und garantieren dafür lineare Laufzeit.**

3.1 Das Pumping-Lemma

Wie man beweist, dass etwas *nicht* geht

Um zu zeigen, dass eine Sprache regulär ist, gibt man einen Automaten an. **Um zu zeigen, dass sie es *nicht* ist, braucht man ein Argument über *alle* Automaten** – *und das leistet das Pumping-Lemma.*

Satz 3.3 (Pumping-Lemma für reguläre Sprachen)**.** *Ist L regulär, so gibt es eine Zahl $p \geq 1$ (die* ***Pumpinglänge****), sodass sich* ***jedes*** *$w \in L$ mit $|w| \geq p$ zerlegen lässt als*

$$w = xyz \qquad \text{mit} \qquad |xy| \leq p, \quad |y| \geq 1,$$

und für ***alle*** *$i \geq 0$ gilt $xy^iz \in L$.*

Der Beweis – er ist das Schubfachprinzip.
Sei A ein DEA für L mit $p := |Q|$ Zuständen. Sei $w \in L$ mit $|w| \geq p$.
Beim Lesen von w durchläuft A die Zustände

$$q_0,\ q_1,\ q_2,\ \ldots,\ q_{|w|}.$$

Betrachtet man die ersten $p+1$ davon, so sind das $p+1$ Zustände aus einer Menge von nur p.

Nach dem Schubfachprinzip muss sich einer wiederholen (Mathematik-Band 15, Kapitel 1). **Es gibt also $i < j \leq p$ mit $q_i = q_j$.**

Setze

$$x := w_1 \ldots w_i, \qquad y := w_{i+1} \ldots w_j, \qquad z := w_{j+1} \ldots w_{|w|}.$$

$\lvert xy\rvert = j \leq p$	✓
$\lvert y\rvert = j - i \geq 1$	✓
y führt von q_i nach q_i	**eine Schleife**

Die Schleife lässt sich beliebig oft durchlaufen oder auslassen – der Automat merkt keinen Unterschied:

$$\hat{\delta}(q_0,\ xy^iz) = \hat{\delta}(q_0,\ xyz) \in F \Longrightarrow xy^iz \in L.$$

Der ganze Beweis besteht darin, dass eine endliche Maschine sich auf einem langen Wort wiederholen muss – und dass diese Wiederholung eine Schleife ist, die man aufblasen kann.

Die Quantoren sind die Fehlerquelle

Das Lemma hat die Form

$$\exists p\ \forall w\ \exists\, xyz\ \forall i:\ \ xy^iz \in L.$$

*Um zu **widerlegen**, dass eine Sprache regulär ist, muss man die Negation zeigen:*

$$\forall p\ \exists w\ \forall\, xyz\ \exists i:\ \ xy^iz \notin L.$$

p	ist **gegeben** – man darf es nicht wählen
w	wählt **man** – geschickt, in Abhängigkeit von p
Zerlegung	ist **gegeben** – man muss *alle* behandeln
i	wählt **man** – meist $i = 0$ oder $i = 2$

Der häufigste Fehler ist, sich eine bequeme Zerlegung auszusuchen. Man muss zeigen, dass *keine* Zerlegung funktioniert – die Bedingung $|xy| \le p$ hilft dabei entscheidend, weil sie y in den vorderen Teil zwingt.

Und die Umkehrung gilt nicht: *Es gibt nichtreguläre Sprachen, die die Pumpingbedingung erfüllen.* **Das Lemma ist notwendig, nicht hinreichend** – *wer Regularität* beweisen *will, braucht Myhill-Nerode.*

Myhill-Nerode – die exakte Charakterisierung

Definition 3.4. Für eine Sprache L heißen zwei Wörter u, v **äquivalent**, geschrieben $u \sim_L v$, wenn

$$\forall z \in \Sigma^*:\quad uz \in L \iff vz \in L.$$

*In Worten: u **und** v **sind ununterscheidbar**, denn keine Fortsetzung trennt sie.*

Satz 3.5 (Myhill und Nerode, 1958)**.** *L ist genau dann regulär, wenn $\sim_L$* ***endlich viele*** *Äquivalenzklassen hat. Ihre Anzahl ist die Zustandszahl des* ***minimalen*** *DEA.*

> ***Das ist stärker als das Pumping-Lemma:*** *Es ist eine* Charakterisierung, *keine bloß notwendige Bedingung* – **und es liefert zugleich den minimalen Automaten.**

Warum die Klassen die Zustände sind.
Ein Zustand des minimalen Automaten ***ist*** *genau die Information, die man über das bisher Gelesene noch braucht.* **Und ununterscheidbare Präfixe brauchen denselben Zustand.** ✓

Der minimale DEA ist damit bis auf Umbenennung ***eindeutig*** **– eine Eigenschaft, die es bei kontextfreien Grammatiken nicht mehr gibt.**

Aufgabe 3.1.

(a) Beweisen Sie mit dem Pumping-Lemma, dass $\{0^n1^n\}$ nicht regulär ist – und zwei weitere Fälle.
(b) Wenden Sie Myhill-Nerode an und minimieren Sie einen Automaten.
(c) Untersuchen Sie eine Sprache, die das Pumping-Lemma erfüllt und trotzdem nicht regulär ist.

Lösung Schritt für Schritt.

(a) Drei Nichtregularitätsbeweise.
Fall 1 – $L = \{0^n1^n : n \geq 0\}$.
Sei p die Pumpinglänge. ***Wir wählen***

$$w := 0^p1^p \in L, \qquad |w| = 2p \geq p. \quad \checkmark$$

Sei $w = xyz$ eine beliebige Zerlegung mit $|xy| \leq p$ und $|y| \geq 1$.

> ***Da $|xy| \leq p$ ist, liegen x und y vollständig im Nullenblock*** – die ersten p Zeichen von w sind alle Nullen. **Also besteht y aus lauter Nullen:** $y = 0^k$ mit $1 \leq k \leq p$.

Wähle $i = 2$:

$$xy^2z = 0^{p+k}1^p.$$

Da $k \geq 1$, ist $p + k \neq p$, also

$$xy^2z \notin L. \qquad \text{Widerspruch.}$$

Mit $i = 0$ ginge es genauso: $xy^0z = 0^{p-k}1^p \notin L$.

Die Bedingung $|xy| \leq p$ hat den ganzen Beweis getragen. Ohne sie hätte der Gegner y so wählen können, dass es gleich viele Nullen und Einsen enthält – und dann wäre nichts zu beweisen gewesen.

Fall 2 – $L = \{ww : w \in \{0,1\}^*\}$ (Verdopplungen).
Wähle $w := 0^p 1 0^p 1 \in L$.
Wegen $|xy| \leq p$ *ist wieder* $y = 0^k$, $k \geq 1$, *im* ***ersten*** *Nullenblock.*
Mit $i = 2$*:*

$$xy^2z = 0^{p+k}10^p1.$$

Wäre das von der Form uu*, so müsste die Zeichenkette in zwei gleiche Hälften zerfallen. Die Gesamtlänge ist* $2p + k + 2$*; die Hälfte liegt bei* $p + \frac{k}{2} + 1$. **Die erste Hälfte enthielte dann nur Nullen (denn die erste 1 steht an Stelle** $p + k + 1 > p + \frac{k}{2} + 1$**), die zweite eine 1** *– also sind sie verschieden.*

$$\Longrightarrow xy^2z \notin L.$$

Fall 3 – $L = \{0^{n^2} : n \geq 0\}$ (Quadratzahlen).
Wähle $w := 0^{p^2}$.
Jede Zerlegung hat $y = 0^k$ *mit* $1 \leq k \leq p$*. Mit* $i = 2$*:*

$$\left|xy^2z\right| = p^2 + k.$$

Nun ist

$$p^2 < p^2 + k \leq p^2 + p < p^2 + 2p + 1 = (p+1)^2.$$

Die Länge liegt echt zwischen zwei aufeinanderfolgenden Quadratzahlen – also ist sie keine. ✓

Dieselbe Rechnung zeigt, dass die Sprache der Zweierpotenzen und die der Primzahlen (in unärer Schreibweise) nicht regulär sind – **überall dort, wo die Abstände zwischen zulässigen Längen unbeschränkt wachsen.**

(b) Myhill-Nerode und Minimierung.
Teil 1 – Die Klassen für $\{0^n1^n\}$**.**
Behauptung: $0^i \not\sim_L 0^j$ *für* $i \neq j$.
Nimm $z := 1^i$*. Dann ist* $0^iz = 0^i1^i \in L$*, aber* $0^jz = 0^j1^i \notin L$. ✓

Also gibt es unendlich viele Klassen – eine für jedes i**.** Nach Myhill-Nerode ist L nicht regulär.

Dieser Beweis ist kürzer als der mit dem Pumping-Lemma und braucht keine Fallunterscheidung. **Man muss nur eine unendliche Familie paarweise unterscheidbarer Präfixe finden.**

Teil 2 – Die Klassen für eine reguläre Sprache.
L *= Wörter über* $\{0,1\}$ *mit gerader Anzahl von Einsen.*

$[\varepsilon]$	alle Wörter mit gerade vielen Einsen
$[1]$	alle Wörter mit ungerade vielen Einsen

Zwei Klassen $\Longrightarrow$ **minimaler Automat mit zwei Zuständen** ✓ – *genau der aus Kapitel 2.*

Teil 3 – Minimierung durch Partitionsverfeinerung.

*Das praktische Verfahren (*MOORE *1956,* HOPCROFT *1971) berechnet die Klassen, ohne alle Wörter zu betrachten.*

1. Beginne mit zwei Blöcken: F und $Q \setminus F$.
2. Solange ein Block existiert, dessen Zustände bei einem Zeichen in **verschiedene** Blöcke führen: spalte ihn auf.
3. Die entstandenen Blöcke sind die Zustände des minimalen Automaten.

Ein Beispiel mit fünf Zuständen, $F = \{q_2, q_4\}$*:*

Zustand	**bei** 0	**bei** 1	**akzept.?**
q_0	q_1	q_2	nein
q_1	q_0	q_3	nein
q_2	q_4	q_2	**ja**
q_3	q_4	q_2	nein
q_4	q_2	q_4	**ja**

Runde 0	$B_1 = \{q_0, q_1, q_3\}, \quad B_2 = \{q_2, q_4\}$
Runde 1	Prüfe B_1 bei 0: $q_0 \to B_1$, $q_1 \to B_1$, $q_3 \to B_2$ $\Longrightarrow$ **aufspalten**: $\{q_0, q_1\}$ und $\{q_3\}$
Runde 2	Prüfe $\{q_0, q_1\}$ bei 1: $q_0 \to B_2$, $q_1 \to \{q_3\}$ $\Longrightarrow$ **aufspalten**: $\{q_0\}$ und $\{q_1\}$
Runde 3	Prüfe $B_2 = \{q_2, q_4\}$: bei 0 beide nach B_2, bei 1 beide nach B_2 $\Longrightarrow$ **nicht trennbar**

Minimaler Automat: $\{q_0\}$, $\{q_1\}$, $\{q_3\}$, $\{q_2, q_4\}$ $--$ vier Zustände

Aus fünf wurden vier: q_2 und q_4 sind ununterscheidbar und werden verschmolzen. ✓

Hopcrofts Verfahren von 1971 leistet das in $\mathcal{O}(n\,|\Sigma|\log n)$ – **indem es stets den *kleineren* der beiden entstandenen Blöcke weiterverfolgt.** *Dieselbe Idee wie bei der Vereinigung nach Rang in Union-Find* (Band 1, Kapitel 12).

(c) Wenn das Pumping-Lemma versagt.
Betrachte über $\Sigma = \{a, b, c\}$:

$$L := \{a^i b^j c^k : i, j, k \geq 0 \text{ und } (i = 1 \Longrightarrow j = k)\}.$$

Schritt 1 – L erfüllt das Pumping-Lemma mit $p = 2$.
Sei $w \in L$ mit $|w| \geq 2$. Wir geben für jeden Fall eine Zerlegung an:

$i = 0$	Es gibt keine Bedingung. Man pumpe irgendein Zeichen – die Bedingung bleibt erfüllt, weil $i = 0$ bleibt.
$i = 1$	Setze $x = \varepsilon$, $y = a$, z der Rest. Dann ist $xy^0z = b^j c^k$ (also $i = 0$: erlaubt), und für $i \geq 2$ ist $a^i b^j c^k$ mit $i \geq 2$ ebenfalls erlaubt.
$i \geq 2$	Man pumpe das erste a; i bleibt ≥ 2 oder wird ≥ 1…

Der Fall $i = 1$ ist der lehrreiche: **Pumpt man das einzige a weg oder verdoppelt es, so verlässt man den kritischen Fall $i = 1$ –** *und die Bedingung $j = k$ ist dann gar nicht mehr zu erfüllen.*

(Für $i \geq 2$ pumpe man ein a; für $i = 0$ und $|w| \geq 2$ hat man immer ein b oder c zum Pumpen.)

Schritt 2 – L ist trotzdem nicht regulär.
Wäre L regulär, so wäre auch

$$L \cap a\,b^*c^* = \{ab^nc^n : n \geq 0\}$$

regulär – **denn die regulären Sprachen sind unter Schnitt abgeschlossen** (Kapitel 2), *und $a\,b^*c^*$ ist offenbar regulär.*
Aber $\{ab^nc^n\}$ ist im Wesentlichen $\{0^n1^n\}$ und damit nach Teil (a) nicht regulär. **Widerspruch.**

Die Lehre

Pumping-Lemma	**notwendig**, nicht hinreichend
Myhill-Nerode	**Charakterisierung** – gilt in beide Richtungen
Abschlusseigenschaften	oft der schnellste Weg

In der Praxis ist die dritte Zeile die nützlichste. Wer zeigen will, dass eine komplizierte Sprache nicht regulär ist, schneidet sie mit einer einfachen regulären und führt sie auf einen bekannten Fall zurück.

Das ist das erste Auftreten der Reduktion – *der Beweistechnik, die den ganzen Rest dieses Bandes trägt* (Kapitel 9).

Kleenes Originalarbeit von 1951 ist schwer lesbar; Hopcroft, Motwani und Ullman[10] und Kozen[13] führen beides sauber aus; Hromkovič[11] legt besonderen Wert auf die Beweistechniken der Nichtexistenz.

4 Kontextfreie Sprachen

Ein Stapel genügt für die Schachtelung

Der endliche Automat scheiterte an $\{0^n1^n\}$, weil er nicht zählen kann. **Gibt man ihm einen Stapel** (Band 1, Kapitel 2), **so kann er es** – *und damit erreicht man genau die Klasse, die Programmiersprachen beschreibt.*

Definition 4.1. Eine **kontextfreie Grammatik** ist ein Tupel $G = (V, \Sigma, P, S)$ mit Variablen V, Terminalen Σ, einem Startsymbol $S \in V$ und Regeln der Form

$$A \to \alpha, \qquad A \in V,\ \alpha \in (V \cup \Sigma)^*.$$

„Kontextfrei" heißt: Links steht* genau eine *Variable. Man darf A ersetzen, ganz gleich, was drumherum steht – der Kontext spielt keine Rolle.

Ein Beispiel. *Die Grammatik $S \to 0S1 \mid \varepsilon$ erzeugt genau $\{0^n1^n\}$.*

$$S \Rightarrow 0S1 \Rightarrow 00S11 \Rightarrow 000S111 \Rightarrow 000111.$$

Ein zweites. *Korrekt geklammerte Ausdrücke:*

$$S \to (S) \mid SS \mid \varepsilon.$$

Klasse	Maschine	Grammatikform
regulär	endlicher Automat	$A \to aB$, $A \to a$
kontextfrei	**Kellerautomat**	$A \to \alpha$
kontextsensitiv	linear beschränkt	$\alpha A \beta \to \alpha \gamma \beta$
aufzählbar	Turingmaschine	$\alpha \to \beta$

*Das ist die **Chomsky-Hierarchie** (1956), Typ 3 bis Typ 0.* **Sie ist echt aufsteigend – jede Stufe kann strikt mehr als die vorige.**

Ein Linguist erfindet die Grammatiktheorie der Informatik

Noam Chomsky war 1956 ein 27-jähriger Sprachwissenschaftler am MIT. *Sein Aufsatz „Three Models for the Description of Language“ untersuchte, welche Art von Regelsystem die menschliche Sprache beschreiben kann.*

> *Seine These:* ***Endliche Automaten reichen nicht*** *– englische Sätze enthalten geschachtelte Einbettungen („Der Mann, den die Frau, die…, kannte, kam“), und Schachtelung ist nicht regulär.*

Die Hierarchie war ein Nebenprodukt seiner linguistischen Argumentation. *Für die Sprachwissenschaft ist sie bis heute umstritten; für die Informatik wurde sie grundlegend.*

John Backus *und* Peter Naur *benutzten kontextfreie Regeln 1959/60 zur Definition von ALGOL 60 –* **die Backus-Naur-Form.** *Sie wussten zunächst nicht, dass Chomsky dieselbe Struktur beschrieben hatte.*

> *Seither wird* ***jede*** *Programmiersprache mit einer kontextfreien Grammatik definiert –* **und der Parser jedes Übersetzers ist ein Kellerautomat** (Band 6).

Eine Ironie: *Chomskys eigentliches Ziel – ein Modell natürlicher Sprache – gilt als nicht erreicht. Kontextfreie Grammatiken sind für das Deutsche und Englische zu schwach; das Schweizerdeutsche liefert sogar ein sauberes Gegenbeispiel mit gekreuzten Abhängigkeiten.*

Sein Nebenergebnis dagegen trägt seit siebzig Jahren die Übersetzertechnik.

4.1 Kellerautomaten und Mehrdeutigkeit

Der Automat mit Stapel

Definition 4.2. Ein **Kellerautomat** liest die Eingabe von links nach rechts und verwaltet dabei einen Stapel. Ein Schritt hängt ab vom Zustand, vom gelesenen Zeichen und vom **obersten Stapelsymbol**; er darf das oberste Symbol ersetzen.

> *Für $\{0^n 1^n\}$: Bei jeder 0 lege ein Symbol ab; bei jeder 1 nimm eines weg.* **Akzeptiere, wenn der Stapel am Ende leer ist.** ✓

Hier trennt sich Nichtdeterminismus doch

Anders als bei endlichen Automaten ist der nichtdeterministische Kellerautomat *echt mächtiger* als der deterministische.

$\{ww^R : w \in \Sigma^*\}$	Palindrome gerader Länge – **nur nichtdeterministisch**
$\{wcw^R\}$	mit Mittemarkierung – auch deterministisch

*Der Grund: Ohne Markierung muss der Automat **raten**, wann die Mitte erreicht ist.* **Ein deterministischer kann das nicht.**

Praktisch bedeutsam ist die Umkehrung: *Nur die **deterministisch** kontextfreien Sprachen lassen sich effizient und eindeutig zerteilen.* **Deshalb sind Programmiersprachen so entworfen, dass ihre Grammatik in eine der handhabbaren Unterklassen fällt** (LL, LR) – *Band 6 führt das aus.*

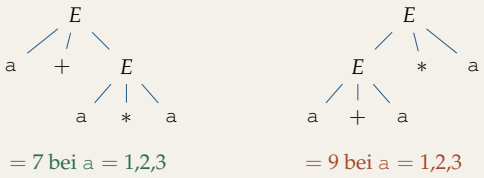
Mehrdeutigkeit

Definition 4.3. Eine Grammatik heißt **mehrdeutig**, wenn es ein Wort mit zwei verschiedenen Ableitungsbäumen gibt.

Das Standardbeispiel:

$$E \to E + E \mid E * E \mid \mathtt{a}.$$

Das Wort `a+a*a` *hat zwei Bäume:*



$= 7$ bei $\mathtt{a} = 1,2,3$ $\qquad$ $= 9$ bei $\mathtt{a} = 1,2,3$

Der Ableitungsbaum bestimmt die Bedeutung. Zwei Bäume heißen: zwei Werte. **Deshalb ist Mehrdeutigkeit in einer Programmiersprache nicht bloß unschön, sondern fatal.**

Man behebt sie durch Umformung der Grammatik – Vorrangebenen einziehen:

$$E \to E + T \mid T, \qquad T \to T * F \mid F, \qquad F \to \mathtt{a} \mid (E).$$

Jetzt ist die Grammatik eindeutig, und die Punktrechnung bindet stärker – **weil sie tiefer im Baum liegt.** ✓

Manche Sprachen sind *inhärent* mehrdeutig

Es gibt kontextfreie Sprachen, für die *jede* Grammatik mehrdeutig ist.
Das Standardbeispiel:

$$L = \{a^i b^j c^k : i = j \text{ oder } j = k\}.$$

> *Wörter mit $i = j = k$ lassen sich auf zwei Weisen erklären – über die erste oder über die zweite Bedingung* – **und keine Grammatik kann sich für eine entscheiden.**

Und ob eine gegebene Grammatik mehrdeutig ist, ist *unentscheidbar* – *ein Ergebnis, das wir in Kapitel 12 beweisen.*

Aufgabe 4.1.

(a) Zeigen Sie mit dem Pumping-Lemma für kontextfreie Sprachen, dass $\{a^n b^n c^n\}$ nicht kontextfrei ist.
(b) Führen Sie den CYK-Algorithmus an einem Beispiel vollständig durch.
(c) Untersuchen Sie die Abschlusseigenschaften und ihre Folgen für die Entscheidbarkeit.

Lösung Schritt für Schritt.

(a) Das Pumping-Lemma für kontextfreie Sprachen.

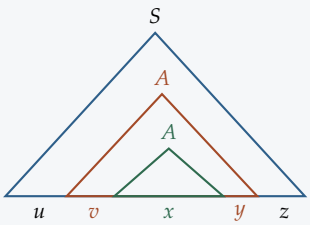
Satz 4.4. *Ist L kontextfrei, so gibt es $p \geq 1$, sodass jedes $w \in L$ mit $|w| \geq p$ zerlegbar ist als*

$$w = uvxyz \quad \textit{mit} \quad |vxy| \leq p, \quad |vy| \geq 1,$$

und $uv^i xy^i z \in L$ für alle $i \geq 0$.

Warum *zwei* Stücke gepumpt werden.

> *Der Beweis betrachtet den Ableitungs**baum**. Ist das Wort lang genug, so muss auf einem Pfad von der Wurzel zu einem Blatt **dieselbe Variable zweimal** vorkommen* (Schubfachprinzip über die endlich vielen Variablen).



> *Das Teilstück zwischen den beiden Vorkommen von A lässt sich beliebig oft einsetzen oder weglassen* – **und es erzeugt Material *links* und *rechts* vom inneren Teilbaum.** *Daher v und y.*

Der Beweis für $L = \{a^n b^n c^n : n \geq 1\}$.
Wähle $w := a^p b^p c^p$.
Sei $w = uvxyz$ eine beliebige Zerlegung mit $|vxy| \leq p$ und $|vy| \geq 1$.
Die Schlüsselbeobachtung.

> ***Da $|vxy| \leq p$ ist, erstreckt sich vxy über höchstens zwei der drei Blöcke*** – die Blöcke haben je die Länge p, ein Fenster der Breite p kann nicht alle drei berühren.

Fall 1: vxy in a's und b's	Beim Pumpen ändert sich die Zahl der a oder b, **nicht die der** c
Fall 2: vxy in b's und c's	Die Zahl der a bleibt fest
Fall 3: vxy in einem Block	Nur ein Buchstabe ändert sich

In jedem Fall gilt für $i = 2$: Mindestens einer der drei Buchstaben behält seine Anzahl, mindestens einer ändert sie (wegen $|vy| \geq 1$).

$$\Longrightarrow uv^2xy^2z \notin L.$$

> ***Die Grenze der kontextfreien Sprachen liegt also bei* zwei *gekoppelten Zählern.*** Ein Stapel kann eine Übereinstimmung erzwingen, aber nicht zwei unabhängige gleichzeitig.

*Genau deshalb ist die Typprüfung einer Programmiersprache **nicht** kontextfrei: „Jede benutzte Variable muss vorher deklariert sein" koppelt beliebig viele Stellen.* **Übersetzer erledigen das deshalb in einer eigenen Phase nach dem Parsen** (Band 6).

(b) Der CYK-Algorithmus.

> *Der Algorithmus von* COCKE, YOUNGER *und* KASAMI *entscheidet in $\mathcal{O}(n^3)$, ob ein Wort von einer kontextfreien Grammatik erzeugt wird* – **für *jede* kontextfreie Grammatik, auch mehrdeutige.**

Voraussetzung: Chomsky-Normalform.
Alle Regeln haben die Form $A \to BC$ oder $A \to a$. **Jede kontextfreie Grammatik lässt sich so umformen.**
Unsere Grammatik:

$S \to AB \mid BC$
$A \to BA \mid a$
$B \to CC \mid b$
$C \to AB \mid a$

Das Wort: $w = \texttt{baaba}$, $n = 5$.
Die Idee.

$$\boxed{T[i][j] := \{A \in V : A \Rightarrow^* w_i \dots w_{i+j-1}\}}$$

– die Menge der Variablen, aus denen sich das Teilwort der Länge j ab Position i ableiten lässt.
Zeile $j = 1$ – die einzelnen Zeichen.

	b	a	a	b	a
$j = 1$	$\{B\}$	$\{A, C\}$	$\{A, C\}$	$\{B\}$	$\{A, C\}$

Denn $B \to b$ und $A \to a$, $C \to a$. ✓
Zeile $j = 2$ – die Rekursion.

$$T[i][2] = \{A : A \to BC \text{ mit } B \in T[i][1],\ C \in T[i+1][1]\}.$$

Position 1 (ba): Paare $B \times \{A, C\}$, also BA und BC.

Paar	**Regel?**	**Ergebnis**
BA	$A \to BA$ ✓	A
BC	$S \to BC$ ✓	S

$$T[1][2] = \{A, S\}.$$

Position 2 (aa): Paare aus $\{A, C\} \times \{A, C\}$: AA, AC, CA, CC.

Nur CC kommt vor – Regel $B \to CC$. **Also** $T[2][2] = \{B\}$.

Position 3 (ab): $\{A, C\} \times \{B\}$: AB und CB.

$S \to AB$ *und* $C \to AB$. **Also** $T[3][2] = \{S, C\}$.

Position 4 (ba): wie Position 1. $T[4][2] = \{A, S\}$**.**
Die vollständige Tabelle.

j	Pos. 1	Pos. 2	Pos. 3	Pos. 4	Pos. 5
5	$\{S, A, C\}$				
4	$\{S, A, C\}$	$\{S, A, C\}$			
3	$\{\}$	$\{B\}$	$\{B\}$		
2	$\{A, S\}$	$\{B\}$	$\{S, C\}$	$\{A, S\}$	
1	$\{B\}$	$\{A, C\}$	$\{A, C\}$	$\{B\}$	$\{A, C\}$

Eine Zelle im Detail: $T[1][5]$**.**
Man prüft alle Teilungen des Wortes `baaba` *in zwei Teile:*

Teilung	**links**	**rechts**	**Regeln**
$1+4$	$\{B\}$	$\{S, A, C\}$	$BA \to A$, $BC \to S$
$2+3$	$\{A, S\}$	$\{B\}$	$AB \to S, C$
$3+2$	$\{\}$	$\{A, S\}$	—
$4+1$	$\{S, A, C\}$	$\{A, C\}$	—

$$T[1][5] = \{A, S, C\}.$$

$$\boxed{S \in T[1][5] \Longrightarrow \texttt{baaba} \in L(G) \qquad \checkmark}$$

Der Aufwand.

Was	**Anzahl**	**Kosten**
Zellen	$\mathcal{O}(n^2)$	
Teilungen je Zelle	$\mathcal{O}(n)$	
Regelprüfungen	$\mathcal{O}(\lvert P\rvert)$	
gesamt		$\mathcal{O}(\mathbf{n^3 \cdot \lvert P\rvert})$

Das ist dynamische Programmierung *(Band 1, Kapitel 14)* – dieselbe Bauform wie beim Editierabstand. **Die Teilprobleme sind die Teilwörter, und sie überlappen sich massiv.**

Für die Praxis ist $\mathcal{O}(n^3)$ zu langsam; Übersetzer benutzen LL- oder LR-Verfahren, die in $\mathcal{O}(n)$ laufen – **aber nur für eingeschränkte Grammatiken.** *CYK ist der allgemeine Fall.*

(c) Abschlusseigenschaften und ihre Folgen.

Operation	regulär	kontextfrei
Vereinigung	ja	ja
Verkettung	ja	ja
Kleene-Stern	ja	ja
Schnitt	ja	**nein**
Komplement	ja	**nein**
Schnitt mit regulärer Spr.	ja	**ja**

Das Gegenbeispiel zum Schnitt.

$$L_1 = \{a^n b^n c^m\}, \qquad L_2 = \{a^m b^n c^n\}.$$

Beide sind kontextfrei – je ein Stapel für eine Kopplung. **Aber**

$$L_1 \cap L_2 = \{a^n b^n c^n\},$$

und das ist nach Teil (a) nicht kontextfrei. ✓
Und daraus folgt das Komplement.
Wären die kontextfreien Sprachen unter Komplement abgeschlossen, so auch unter Schnitt, denn

$$L_1 \cap L_2 = \overline{\overline{L_1} \cup \overline{L_2}}$$

(de Morgan), und Vereinigung ist erlaubt. **Widerspruch.**

> ***Ein hübscher Beweis: Man zeigt eine Nichtabgeschlossenheit, ohne ein einziges Gegenbeispiel für das Komplement anzugeben.***

Die Folgen für die Entscheidbarkeit.

Frage	regulär	kontextfrei
$w \in L$?	$\mathcal{O}(n)$	$\mathcal{O}(n^3)$
$L = \emptyset$?	entscheidbar	entscheidbar
L endlich?	entscheidbar	entscheidbar
$L_1 = L_2$**?**	entscheidbar	**unentscheidbar**
$L_1 \cap L_2 = \emptyset$**?**	entscheidbar	**unentscheidbar**
$L = \Sigma^*$**?**	entscheidbar	**unentscheidbar**
G **mehrdeutig?**	—	**unentscheidbar**

Der Zusammenhang zwischen den beiden Tabellen ist kein Zufall. Wo die Abschlusseigenschaften fehlen, fehlen auch die Entscheidungsverfahren – denn viele Verfahren beruhen darauf, dass man Komplemente und Schnitte bilden darf.

Etwa: „$L_1 = L_2$?“ führt man bei regulären Sprachen auf „$(L_1 \setminus L_2) \cup (L_2 \setminus L_1) = \emptyset$?“ zurück – **und das braucht Komplemente.**

Kapitel 12 beweist die Unentscheidbarkeit der unteren vier Zeilen – **alle mit demselben Werkzeug, dem Postschen Korrespondenzproblem.**

Hopcroft, Motwani und Ullman[10] behandeln die kontextfreien Sprachen am gründlichsten; Sipser[24] ist klarer bei den Beweisen; Chomskys Aufsatz von 1956 ist erstaunlich kurz und gut lesbar.

Part II

Die Maschine

5 Die Turingmaschine

Ein Mensch mit Papier, formalisiert

Turings Analyse (Vorwort) ergab vier Fähigkeiten. Die Maschine setzt sie eins zu eins um:

Der Rechnende	Die Maschine
sieht endlich viel auf einmal	liest **ein** Feld
merkt sich endlich viel	endlich viele **Zustände**
folgt festen Regeln	**Übergangsfunktion**
schreibt und bewegt sich	**Band** und Kopf

Definition 5.1. Eine **Turingmaschine** ist ein Tupel

$$M = (Q,\ \Sigma,\ \Gamma,\ \delta,\ q_0,\ \square,\ F)$$

mit endlicher Zustandsmenge Q, Eingabealphabet Σ, Bandalphabet $\Gamma \supseteq \Sigma$, dem **Blanksymbol** $\square \in \Gamma \setminus \Sigma$, Startzustand q_0, Endzuständen $F \subseteq Q$ und einer Übergangsfunktion

$$\delta : (Q \setminus F) \times \Gamma \longrightarrow Q \times \Gamma \times \{L, R, N\}.$$

Ein Schritt liest also das Feld unter dem Kopf, schreibt ein Zeichen hinein, wechselt den Zustand und bewegt den Kopf um höchstens ein Feld. **Mehr geschieht nicht.**

Die Konfiguration.
Der vollständige Zustand der Rechnung ist ein Tripel: Bandinhalt links vom Kopf, Zustand, Bandinhalt ab dem Kopf. Man schreibt

$$uqv \qquad \text{mit } u, v \in \Gamma^*,\ q \in Q.$$

Da nur endlich viele Felder je beschrieben wurden, ist eine Konfiguration ein ***endliches Wort*** **– ein Punkt, der in Kapitel 6 entscheidend wird.**

Warum ein *unendliches* Band kein Widerspruch ist

Das Band ist unbegrenzt, aber zu jedem Zeitpunkt nur endlich beschrieben.

> *Nach t Schritten kann der Kopf höchstens t Felder besucht haben.* **Es gibt also nie unendlich viel Information** – *nur unbeschränkt viel.*

Man kann sich das Band als Papierrolle vorstellen, an die man ein Blatt anklebt, wenn man rechts hinausläuft. **Das ist genau die Idealisierung, die auch beim Begriff „Algorithmus" steckt**: *Wir wollen nicht, dass ein Verfahren an einer willkürlichen Speichergrenze scheitert.*

*Ein Rechner mit **festem** Speicher ist streng genommen ein endlicher Automat* – **mit etwa $2^{10^{12}}$ Zuständen.** *Diese Aussage ist wahr und völlig nutzlos.*

Ein vollständiges Beispiel

Eine Maschine, die eine Binärzahl um 1 erhöht. Der Kopf steht am rechten Ende der Zahl.

Zustand	gelesen	schreibe, gehe, neuer Zustand	Bedeutung
q_0	1	0, L, q_0	Übertrag läuft weiter
q_0	0	1, N, q_f	**fertig**
q_0	□	1, N, q_f	**neue Stelle**

Ablauf für □1011 *(also* 11*), Kopf auf der letzten 1:*

Schritt	Konfiguration	Regel
0	□101 q_0 1	$1 \to 0$, L
1	□10 q_0 10	$1 \to 0$, L
2	□1 q_0 000	$0 \to 1$, halt
3	□1 q_f 100	

$(1011)_2 = 11, \qquad (1100)_2 = 12.$ ✓

Drei Regeln für eine Addition um eins. Das zeigt beides: dass es geht, und wie mühsam es ist. **Niemand würde so programmieren** – *und darum geht es auch nicht.*

5.1 Robustheit des Modells

Alle Varianten sind gleich mächtig

*Man könnte einwenden, die Definition sei willkürlich: Warum **ein** Band? Warum nur eine Richtung pro Schritt?* **Die Antwort ist, dass es keine Rolle spielt.**

Variante	Mächtigkeit	Zeitverlust
k Bänder	gleich	$\mathcal{O}(t^2)$
zweidimensionales Band	gleich	polynomiell
Bandalphabet $\{0, 1, \square\}$	gleich	konstanter Faktor
einseitig unendlich	gleich	konstanter Faktor
nichtdeterministisch	gleich	**exponentiell**
Registermaschine	gleich	polynomiell

Wie man mehrere Bänder auf eines abbildet.

*Man legt die k Bänder **ineinander**: Das einzelne Band enthält in den Feldern $i, i+k, i+2k, \ldots$ das j-te Band, und zusätzlich für jedes Band eine Markierung, wo dessen Kopf steht.*

Ein Schritt der Mehrbandmaschine wird simuliert, indem die Einbandmaschine einmal über den beschriebenen Bereich läuft, alle k Kopfpositionen abliest, dann zurückläuft und alle Änderungen einträgt.

$$\text{Kosten: } \mathcal{O}(t) \text{ pro Schritt} \implies \mathcal{O}(t^2) \text{ insgesamt.} \qquad \checkmark$$

Der quadratische Verlust ist für diesen Band unerheblich – er entscheidet nicht darüber, *ob* etwas berechenbar ist. **Für Band 3 ist er sehr wohl erheblich**, *denn dort geht es um die Zeit.*

Nichtdeterminismus.
*Eine nichtdeterministische Maschine hat δ als **Relation** und akzeptiert, wenn* ein *Berechnungsweg akzeptiert.*

Die deterministische Simulation durchsucht den Berechnungsbaum – **aber *in der Breite*, nicht in der Tiefe.**

Warum Tiefensuche hier falsch wäre

Der Berechnungsbaum kann unendlich tiefe Äste haben.

Eine Tiefensuche liefe in einen solchen Ast hinein und käme nie zurück – **auch dann nicht, wenn ein anderer, kurzer Ast zum Erfolg führt.**

*Die **Breitensuche** findet jeden akzeptierenden Weg endlicher Länge in endlicher Zeit.* **Sie kostet Zeit $\mathcal{O}(b^t)$, aber sie terminiert, wann immer ein Erfolg existiert.** ✓

Dasselbe Muster kehrt in Kapitel 10 wieder, wo man mehrere Rechnungen „verzahnt" laufen lässt – **die Technik des Dovetailing.**

Entscheiden und Akzeptieren – der wichtigste Unterschied dieses Bandes

Definition 5.2. Sei $L \subseteq \Sigma^*$.

L ist **entscheidbar**	Es gibt M, die auf **jeder** Eingabe hält und richtig „ja" oder „nein" sagt
L ist **aufzählbar**	Es gibt M, die auf $w \in L$ hält und akzeptiert; auf $w \notin L$ darf sie **ewig laufen**

***Der Unterschied liegt allein im Verhalten bei* Nein.** Eine entscheidende Maschine muss auch das Nein liefern; eine akzeptierende darf schweigen.

	bei $w \in L$	**bei** $w \notin L$
entscheidbar	hält, sagt ja	**hält, sagt nein**
aufzählbar	hält, sagt ja	**läuft evtl. ewig**

Andere Namen für dasselbe: **entscheidbar** = **rekursiv**, **aufzählbar** = **rekursiv aufzählbar** = **semi-entscheidbar**.

Diese Unterscheidung ist der Angelpunkt des ganzen Bandes. Das Halteproblem ist aufzählbar, aber nicht entscheidbar – und die Lücke zwischen beiden Begriffen ist genau das, was Kapitel 10 vermisst.

Aufgabe 5.1.

(a) Konstruieren Sie eine Turingmaschine für $\{0^n1^n\}$ und verfolgen Sie einen Lauf vollständig.

(b) Simulieren Sie zwei Bänder auf einem und bestimmen Sie den Zeitverlust genau.

(c) Zeigen Sie: L ist genau dann entscheidbar, wenn L und $\overline{L}$ beide aufzählbar sind.

Lösung Schritt für Schritt.

(a) Eine Maschine für $\{0^n1^n\}$.
Die Idee.

> *Streiche abwechselnd die linkeste 0 und die linkeste 1, indem du sie durch Markierungen X und Y ersetzt.* **Wiederhole, bis nichts mehr übrig ist.**

Die Übergangstabelle.

Zustand	liest	schreibt, geht, Ziel	Bedeutung
q_0	0	X, R, q_1	0 markieren
q_0	Y	Y, R, q_3	alle 0 erledigt
q_0	$\square$	$\square$, N, q_f	leeres Wort: **ja**
q_1	0	0, R, q_1	über 0 laufen
q_1	Y	Y, R, q_1	über Y laufen
q_1	1	Y, L, q_2	**1 markieren**
q_2	0	0, L, q_2	zurücklaufen
q_2	Y	Y, L, q_2	
q_2	X	X, R, q_0	**an den Anfang**
q_3	Y	Y, R, q_3	Rest prüfen
q_3	$\square$	$\square$, N, q_f	**akzeptieren**

Alle nicht aufgeführten Fälle führen in einen Verwerfungszustand.
Ein vollständiger Lauf für $w = 0011$.

Nr.	Konfiguration	Regel
0	$q_0\,0011$	$0 \to X$, R
1	$X\,q_1\,011$	über 0
2	$X0\,q_1\,11$	$1 \to Y$, L
3	$X\,q_2\,0Y1$	zurück
4	$q_2\,X0Y1$	an den Anfang
5	$X\,q_0\,0Y1$	$0 \to X$, R
6	$XX\,q_1\,Y1$	über Y
7	$XXY\,q_1\,1$	$1 \to Y$, L
8	$XX\,q_2\,YY$	zurück
9	$X\,q_2\,XYY$	an den Anfang
10	$XX\,q_0\,YY$	Y **gelesen** $\to q_3$
11	$XXY\,q_3\,Y$	über Y
12	$XXYY\,q_3\,\square$	**akzeptieren**

0011 wird akzeptiert. ✓

Warum 0011 **akzeptiert und** 001 **verworfen wird.**
Bei 001 *endet der Lauf in Schritt 10 mit dem Bandinhalt XXY – der Zustand q_0 liest ein Y, geht nach q_3, und q_3 findet $\square$: akzeptiert?*

> ***Nein*** *– denn bei* 001 *scheitert der Lauf schon früher.* **Nach dem Markieren der ersten 0 und der einen 1 steht** $X0Y$ **auf dem Band**; *im nächsten Durchgang markiert q_0 die zweite 0 als X, und q_1 läuft nach rechts über Y hinaus, findet $\square$ statt einer 1* – **und verwirft.** ✓

Der Aufwand.

Was	Anzahl	Kosten je
Durchgänge	n	$\mathcal{O}(n)$ Schritte
gesamt		$\mathcal{O}(\mathbf{n^2})$

> *Mit* ***zwei*** *Bändern ginge es in $\mathcal{O}(n)$: Man kopiert die Nullen auf das zweite Band und streicht sie beim Lesen der Einsen wieder weg.* **Das ist genau ein Kellerautomat** (Kapitel 4).

Und tatsächlich ist bewiesen, dass eine Einband-Turingmaschine für $\{0^n 1^n\}$ ***mindestens*** $\Omega(n \log n)$ *Schritte braucht* (HENNIE, 1965) – **das zusätzliche Band ist also nicht bloß bequem.**

(b) Zwei Bänder auf einem.
Schritt 1 – Die Kodierung.
Das eine Band bekommt ein größeres Alphabet: Jedes Feld enthält ein ***Paar*** *von Symbolen plus zwei Markierungsbits.*

$$\Gamma' := (\Gamma \times \{\cdot, \bullet\})^2.$$

Band 1	a	$\bullet b$	c	$\square$
Band 2	x	y	$\bullet z$	$\square$

Der Punkt $\bullet$ *markiert die jeweilige Kopfposition.*
Schritt 2 – Einen Schritt simulieren.

1. Vom linken Rand nach rechts laufen und beide markierten Symbole im **Zustand** merken.
2. Am rechten Rand angekommen, ist die Regel bestimmt.
3. Zurücklaufen und an den beiden Markierungen schreiben und die Markierungen verschieben.

Schritt 1 braucht endlich viel Gedächtnis: höchstens $|\Gamma|^2$ *Kombinationen* – **also lässt sich das im Zustandsraum unterbringen.** ✓

Schritt 3 – Der Zeitverlust, genau.
Nach t *Schritten der Zweibandmaschine ist der beschriebene Bereich höchstens* $2t + 1$ *Felder breit. Ein simulierter Schritt kostet also* $\mathcal{O}(t)$.

$$T'(t) = \sum_{i=1}^{t} \mathcal{O}(i) = \mathcal{O}\Big(\frac{t(t+1)}{2}\Big) = \mathcal{O}(t^2).$$

Zwei Bänder → ein Band: quadratischer Zeitverlust

k **Bänder**	**Zustandszahl**	**Zeitverlust**
2	$\lvert Q\rvert \cdot \lvert\Gamma\rvert^2$	$\mathcal{O}(t^2)$
k	$\lvert Q\rvert \cdot \lvert\Gamma\rvert^k$	$\mathcal{O}(t^2)$

> ***Bemerkenswert:*** *Der Zeitverlust ist $\mathcal{O}(t^2)$ unabhängig von k* – die Bandzahl geht nur in die **Zustandszahl** ein, und die zählt in der $\mathcal{O}$-Notation nicht.

HENNIE *und* STEARNS *zeigten 1966, dass man mit zwei Bändern k Bänder sogar in $\mathcal{O}(t \log t)$ simulieren kann* – **ein deutlich feineres Ergebnis, das für die Komplexitätstheorie wichtig ist** (Band 3).

(c) Der Satz von Post.

Satz 5.3 (Post, 1944)**.**

$$\boxed{L \text{ entscheidbar} \iff L \text{ und } \overline{L} \text{ beide aufzählbar}}$$

Richtung „⇒" – die leichte.
Sei M ein Entscheider für L. Dann ist M selbst ein Aufzähler für L (er hält ja immer). Und die Maschine $\overline{M}$, die M laufen lässt und die Antwort umdreht, ist ein Entscheider für $\overline{L}$ – also erst recht ein Aufzähler. ✓

> ***Der Schritt „Antwort umdrehen" ist nur erlaubt, weil M* immer *hält.*** Bei einer bloß akzeptierenden Maschine ginge es nicht – man wüsste nicht, wann man das Nein aussprechen darf.

Richtung „⇐" – die interessante.
Seien M_1 ein Aufzähler für L und M_2 einer für $\overline{L}$. Wir bauen einen Entscheider M:

1.	Lasse M_1 und M_2 auf w **gleichzeitig** laufen – abwechselnd je einen Schritt.
2.	Hält M_1 und akzeptiert: gib **ja** aus.
3.	Hält M_2 und akzeptiert: gib **nein** aus.

Warum das immer hält.

> *Für jedes w gilt entweder $w \in L$ oder $w \in \overline{L}$* – **eines von beiden ist stets der Fall.**

$w \in L$	M_1 akzeptiert nach endlich vielen Schritten
$w \notin L$	M_2 akzeptiert nach endlich vielen Schritten

*Also hält **mindestens eine** der beiden Maschinen, und zwar nach endlich vielen Schritten. Die Verzahnung findet das.*

Die Verzahnung ist wesentlich

Man darf *nicht* erst M_1 ganz laufen lassen und dann M_2.

Denn M_1 hält bei $w \notin L$ möglicherweise nie – **und man käme zu M_2 gar nicht erst.**

*Die Technik, mehrere möglicherweise nichtterminierende Rechnungen abwechselnd voranzutreiben, heißt **Dovetailing** („Verzinken", nach der Schreinerverbindung).* **Sie kehrt in diesem Band mehrfach wieder** – *immer dann, wenn man auf ein Ereignis wartet, ohne zu wissen, welches von mehreren eintritt.*

Die Folgerung, die den Rest des Bandes bestimmt.

Der Satz lässt sich umdrehen: **Ist L aufzählbar, aber *nicht* entscheidbar, so ist $\overline{L}$ *nicht einmal* aufzählbar.**

Sprache	aufzählbar?	entscheidbar?
H (Halteproblem)	ja	nein
$\overline{\text{H}}$	**nein**	nein

*Es gibt also **zwei** Stufen jenseits der Entscheidbarkeit* – **und Kapitel 11 zeigt, dass es unendlich viele sind.**

Turings Arbeit[27] definiert die Maschine im Original; Sipser[24] und Kozen[13] führen die Varianten sauber durch; Minsky[16] zeigt, wie wenig man wirklich braucht; Cutland[4] und Davis[5] sind die klassischen Darstellungen der Berechenbarkeitstheorie.

6 Die universelle Maschine

Eine Maschine, die alle anderen nachmacht

*Bisher war jede Turingmaschine für **eine** Aufgabe gebaut.* **Turings entscheidender Schritt war zu zeigen, dass *eine* Maschine genügt** – *wenn man ihr die Beschreibung der anderen als Eingabe gibt.*

vor 1936	Eine Maschine, eine Aufgabe (Difference Engine, Hollerith)
nach 1936	**Eine Maschine, alle Aufgaben** – die Aufgabe ist Eingabe

Das ist die theoretische Rechtfertigung des Computers als Universalgerät – **und die von-Neumann-Architektur ist ihre technische Umsetzung** (Band 0, Kapitel 10).

6.1 Kodierung

Wie eine Maschine zu einem Wort wird

*Damit die universelle Maschine eine andere lesen kann, muss diese als **Wort** vorliegen.* **Das ist der Schritt, den Gödel 1931 für Formeln erfunden hatte** – *die **Gödelisierung**.*

Ein konkretes Verfahren.
Man nummeriere Zustände, Bandsymbole und Richtungen durch:

Objekt	Kodierung	Beispiel
Zustand q_i	0^{i+1}	$q_2 \to 000$
Symbol a_j	0^{j+1}	$a_1 \to 00$
Richtung L, R, N	$0, 00, 000$	

Eine Regel $\delta(q_i, a_j) = (q_k, a_l, D)$ wird zu

$$0^{i+1}\,1\,0^{j+1}\,1\,0^{k+1}\,1\,0^{l+1}\,1\,0^{D},$$

und die ganze Maschine ist die Verkettung aller Regeln, getrennt durch 11*:*

$$\langle M \rangle := 111\ \text{Regel}_1\ 11\ \text{Regel}_2\ 11\ \ldots\ 111.$$

Die Einzelheiten sind gleichgültig. Man braucht nur drei Eigenschaften, und die hat jede vernünftige Kodierung:

(K1)	Aus $\langle M \rangle$ lässt sich M eindeutig zurückgewinnen (**Injektivität**).
(K2)	Es ist **entscheidbar**, ob ein Wort eine gültige Kodierung ist.
(K3)	Jede Maschine hat **unendlich viele** Kodierungen (man hänge Nullen an).

Warum (K2) und (K3) gebraucht werden

Zu (K2): *Die universelle Maschine muss ungültige Eingaben abweisen können.* **Man vereinbart üblicherweise: Ein ungültiges Wort kodiert die Maschine, die sofort verwirft** – *dann ist* jedes *Wort eine Kodierung, und viele Beweise werden kürzer.*
Zu (K3): *Das braucht man beim* ***Padding-Lemma***: **Zu jeder Maschine gibt es beliebig viele verschiedene Programme mit demselben Verhalten.**

> *Das klingt trivial und ist es auch – man hängt totes Beiwerk an.* **Es wird aber gebraucht, um zu zeigen, dass gewisse Mengen *unendlich* sind**, *und es ist der Grund, warum „Sind diese zwei Programme gleich?“ kein Programm entscheiden kann* (Kapitel 9).

Die Folgerung: eine Abzählung aller Maschinen.

> *Aus (K1) und (K2) folgt, dass man alle Turingmaschinen in eine Liste bringen kann:*
>
> $$M_0,\ M_1,\ M_2,\ \ldots$$
>
> **Man gehe alle Wörter in längenlexikographischer Ordnung durch** (Kapitel 1) **und nehme die auf, die gültige Kodierungen sind.**

Diese Nummerierung ist ***berechenbar*** *in beide Richtungen –* **aus** i **bekommt man** M_i **und umgekehrt.** *Sie heißt* ***Gödelnummerierung****, und alles Weitere beruht auf ihr.*

Die universelle Maschine

Satz 6.1 (Turing, 1936)**.** *Es gibt eine Turingmaschine U mit*

$$U(\langle M\rangle \# w) = M(w)$$

für alle Maschinen M und Eingaben w – also: U hält genau dann, wenn M auf w hält, und liefert dann dieselbe Ausgabe.

Die Konstruktion, im Umriss.
U arbeitet mit drei Bändern (nach Kapitel 5 ohne Verlust an Mächtigkeit):

Band 1	Das simulierte Band von M
Band 2	Die Kodierung $\langle M\rangle$ – wird nie verändert
Band 3	Der aktuelle Zustand von M, als Zahl

Ein simulierter Schritt:

1. Lies das Symbol unter dem Kopf auf Band 1 und den Zustand auf Band 3.
2. Durchsuche Band 2 nach der passenden Regel.
3. Führe sie aus: Band 1 beschreiben, Kopf bewegen, Band 3 überschreiben.
4. Ist der neue Zustand ein Endzustand, so halte.

> U ***ist eine feste, endliche Maschine.*** Sie hat eine feste Zustandszahl und ein festes Alphabet – **und simuliert damit Maschinen mit beliebig vielen Zuständen.**

Das ist der Punkt, an dem viele stocken. **Die Auflösung: Der *Zustand* der simulierten Maschine steht nicht im Zustandsraum von U, sondern auf dem *Band*** – *also im unbegrenzten Teil.*

Genau das ist der Gedanke des gespeicherten Programms (Band 0, Kapitel 10): **Was nicht in die Hardware passt, kommt in den Speicher.**

Wie klein U sein kann

Die Suche nach der kleinsten universellen Maschine hat eine lange Geschichte.

Zustände × Symbole	Urheber	Jahr
7×4	Minsky	1962
$4 \times 6, 7 \times 4$	Rogozhin	1996
2×3	**Wolfram/Smith**	**2007**

Die 2×3-Maschine wurde 2007 von dem 20-jährigen Studenten Alex Smith *als universell nachgewiesen* – **im Rahmen eines von Stephen Wolfram ausgeschriebenen Preises von 25 000 Dollar.**

Der Nachweis ist umstritten, *weil er eine unendliche, aber periodische Anfangsbelegung des Bandes voraussetzt.* **Ob das noch „universell" im üblichen Sinn ist, wird verschieden beurteilt.**

Sechs Regeln – **und daraus alles, was berechenbar ist.** *Der Preis ist die Geschwindigkeit: Die Simulation ist exponentiell langsam.*

Das s-m-n-Theorem: Programme, die Programme schreiben

Satz 6.2 (Kleene, 1943). *Zu je zwei Zahlen m, n gibt es eine* ***berechenbare, totale*** *Funktion s_n^m mit*

$$\varphi_{s_n^m(e,\, x_1,\ldots,x_m)}(y_1,\ldots,y_n) \;=\; \varphi_e(x_1,\ldots,x_m,\, y_1,\ldots,y_n),$$

wobei φ_e die von Programm e berechnete Funktion bezeichnet.

Die Aussage in einem Satz: *Man kann einige Argumente eines Programms* ***fest einbauen*** *und erhält dabei berechenbar ein neues Programm.*

Warum das trivial und trotzdem wichtig ist.
Der Beweis ist buchstäblich Textmanipulation: Man nehme den Quelltext von e und setze am Anfang die Zeilen `x1 := 5; x2 := 7;` *davor.* **Das kann ein Programm offensichtlich leisten.**

In der Praxis heißt das	**partielle Auswertung**, **Currying**, **Spezialisierung**
In Beweisen heißt das	Man darf sagen: „Sei P das Programm, das bei Eingabe y Folgendes tut…“ und dabei Parameter benutzen

Ohne dieses Theorem wären fast alle Beweise dieses Bandes nicht führbar. Jede Reduktion in Kapitel 9 baut aus einer gegebenen Maschine eine neue – und dass dieser Bau selbst berechenbar ist, sagt genau das *s-m-n*-Theorem.

Der eigenartige Name kommt daher, dass Kleene die Funktion S_n^m nannte – s für „substitution“. **Er hat sich gehalten, obwohl er nichts erklärt.**

Aufgabe 6.1.

(a) Kodieren Sie eine konkrete Maschine vollständig und rechnen Sie ihre Gödelnummer aus.

(b) Beweisen Sie das Rekursionstheorem und konstruieren Sie damit einen Quine.

(c) Untersuchen Sie, wie viel die universelle Simulation an Zeit kostet.

Lösung Schritt für Schritt.

(a) Eine Maschine als Zahl.
Wir kodieren die Maschine, die eine 1 schreibt und hält:

Regel	von	nach	Richtung
1	$q_0, \square$	$q_1, 1$	N

Schritt 1 – Die Nummerierung festlegen.

$q_0 \to 0$	$q_1 \to 1$	$\square \to a_0$	$1 \to a_1$
$L \to 1$	$R \to 2$	$N \to 3$	

Schritt 2 – Die Regel kodieren.

$$\delta(q_0, a_0) = (q_1, a_1, N) \;\longrightarrow\; 0^1\,1\,0^1\,1\,0^2\,1\,0^2\,1\,0^3.$$

$$= \texttt{0 1 0 1 00 1 00 1 000}.$$

Ohne Zwischenräume: `010100100 1000`, *also* `0101001001000`, *13 Bit.*

Schritt 3 – Die ganze Maschine.

$$\langle M \rangle = \texttt{111 0101001001000 111}.$$

Das sind 19 Bit.
Schritt 4 – Als Zahl.
Man liest das Wort als Binärzahl mit vorangestellter 1 (um führende Nullen zu retten):

$$\texttt{1 1110101001001000111} = ?$$

Das sind 20 Bit; der Wert liegt zwischen $2^{19} = 524\,288$ *und* $2^{20} = 1\,048\,576$*.*
Ausgerechnet:

$$\langle M \rangle \mathrel{\hat{=}} 1\,001\,033\,415.$$

> ***Eine Maschine mit* einer *Regel hat eine zehnstellige Gödelnummer.*** Das zeigt, dass diese Zahlen als Rechengröße nutzlos sind – **ihr Zweck ist ein anderer:** *Sie erlauben es, über „das i-te Programm" zu reden.*

Schritt 5 – Die Umkehrung.
Aus der Zahl gewinnt man die Maschine zurück: binär schreiben, führende 1 streichen, an `111` *und* `11` *zerlegen, Nullenblöcke zählen.* **Ein einfaches, offensichtlich berechenbares Verfahren** *– das ist Eigenschaft (K1).* ✓

(b) Das Rekursionstheorem.

Satz 6.3 (Kleene, 1938)**.** *Zu jeder totalen berechenbaren Funktion f gibt es ein e mit*

$$\varphi_e = \varphi_{f(e)}.$$

> ***In Worten:*** *Formt man Programme berechenbar um, so gibt es stets ein Programm, das dabei sein* ***Verhalten*** *nicht ändert – einen* ***Fixpunkt****.*

Der Beweis.
Definiere die berechenbare Funktion

$$d(x) := \begin{cases} \text{ein Programm für } \varphi_{\varphi_x(x)} & \text{falls } \varphi_x(x) \text{ definiert,} \\ \text{ein Programm, das nirgends hält} & \text{sonst.} \end{cases}$$

Nach dem s-m-n-Theorem ist d total und berechenbar (man baut das Programm bloß hin, ohne es auszuführen). ✓
Nun ist $f \circ d$ berechenbar, hat also selbst eine Programmnummer v:

$$\varphi_v = f \circ d.$$

Setze

$$e := d(v).$$

Dann gilt

$$\varphi_e = \varphi_{d(v)} \overset{\text{Def. } d}{=} \varphi_{\varphi_v(v)} \overset{\varphi_v = f \circ d}{=} \varphi_{f(d(v))} = \varphi_{f(e)}.$$

Der Kunstgriff ist die Selbstanwendung $\varphi_x(x)$ – dieselbe Diagonalbewegung, die in Kapitel 8 den Widerspruch erzeugt. **Hier erzeugt sie einen Fixpunkt.**

Die praktische Fassung.

Jedes Programm darf seinen eigenen Quelltext benutzen, als hätte es ihn geschenkt bekommen. Man darf schreiben: „Sei P das Programm, das $\langle P \rangle$ ausgibt" – und das Rekursionstheorem garantiert, dass es P gibt.

Der Quine, konstruiert.
Sei f die Funktion, die zu einem Programm x ein neues Programm baut, das ***die Zahl*** *x* ***ausgibt*** *und hält.* f **ist offensichtlich berechenbar und total.**
Nach dem Rekursionstheorem gibt es e mit $\varphi_e = \varphi_{f(e)}$.

Die rechte Seite gibt e aus. **Also gibt auch** φ_e **die Zahl** e **aus** – *und e ist die Nummer des laufenden Programms.* ✓

Es gibt ein Programm, das seinen eigenen Quelltext ausgibt.

Band 0, Kapitel 14 hat einen Quine konstruiert*; hier ist bewiesen, dass es ihn geben* ***muss*** **– in jeder Sprache, die universell ist.**

Und dieselbe Konstruktion liefert Thompsons Hintertür (Band 0, Kapitel 12): **Sei** f **die Funktion, die ein Programm um die Regel erweitert, sich selbst bei jeder Übersetzung wieder einzubauen.** *Der Fixpunkt* ist *der kompromittierte Übersetzer.*

(c) Was die Simulation kostet.
Schritt 1 – Ein simulierter Schritt.

Teilaufgabe	**Kosten**	**warum**
Zustand von Band 3 lesen	$\mathcal{O}(\log \lvert Q \rvert)$	Zahl in Binärform
Regel auf Band 2 suchen	$\mathcal{O}(\lvert\langle M \rangle\rvert)$	linear durchsuchen
Band 1 ändern	$\mathcal{O}(1)$	

Kosten je Schritt $= \mathcal{O}(\lvert\langle M \rangle\rvert)$.

Schritt 2 – Der Gesamtaufwand.
Läuft M in $t(n)$ Schritten, so braucht U

$$\mathcal{O}\big(\lvert\langle M \rangle\rvert \cdot t(n)\big) \text{ Schritte}$$

auf einer Mehrbandmaschine. **Mit einem Band kommt der quadratische Faktor aus Kapitel 5 hinzu:**

$$\mathcal{O}\big(|\langle M\rangle| \cdot t(n)^2\big).$$

> ***Der Faktor $|\langle M\rangle|$ ist eine Konstante – er hängt von M ab, nicht von der Eingabe.*** Für die Berechenbarkeit ist er gleichgültig; für die Komplexitätstheorie ist der **logarithmische** Zusatzfaktor bei besseren Simulationen wichtig.

HENNIE *und* STEARNS *zeigten 1966 eine Simulation mit Kosten $\mathcal{O}(t \log t)$* – **und genau diese Schranke wird gebraucht, um den Zeithierarchiesatz zu beweisen** (Band 3).

Schritt 3 – Der Vergleich mit der Praxis.

Simulationsebene	**Verlangsamung**	**Beispiel**
Interpreter	10–100	Python
Bytecode-VM	2–10	Java ohne JIT
Emulator	5–50	fremde Architektur
universelle TM	$\mathcal{O}(t \log t)$	theoretisch

> ***Die letzte Zeile ist die überraschende:*** *Die theoretische Simulation ist* asymptotisch *besser als jeder reale Interpreter* – weil sie nur einen logarithmischen Faktor kostet, während ein Interpreter einen konstanten, aber großen Faktor hat.

Der Grund ist, dass die Turingmaschine ohnehin so langsam ist, dass die Simulation kaum ins Gewicht fällt. **Ein Modell, das nichts kann, lässt sich billig nachmachen.** Turings Originalarbeit[27] enthält die universelle Maschine in Abschnitt 6; Kleene[12] entwickelt *s-m-n* und das Rekursionstheorem; Rogers[22] und Odifreddi[18] sind die Standardwerke der Rekursionstheorie; Hofstadter[9] macht den Selbstbezug zum Thema.

7 Die Church-Turing-These

Eine Behauptung, die kein Satz ist

Jede Funktion, die überhaupt „im intuitiven Sinn“ berechenbar ist, ist von einer Turingmaschine berechenbar.

Warum man das nicht beweisen kann

Die These setzt einen *unpräzisen* Begriff mit einem *präzisen* gleich.

„intuitiv berechenbar“	Vorwissenschaftlicher Begriff – was ein Mensch oder eine Maschine „ausrechnen“ kann
„Turing-berechenbar“	Mathematisch definiert (Kapitel 5)

Ein Beweis bräuchte eine mathematische Definition der linken Seite – **und die gibt es nicht, denn genau sie soll ja gefunden werden.**

Die These ist also eine *Adäquatheitsbehauptung*, *vergleichbar der Aussage, dass die ε-δ-Definition das trifft, was man vorher unter „Grenzwert“ verstand* (Mathematik-Band 3).

*Sie ist **widerlegbar**: Es genügte, ein Verfahren anzugeben, das jeder als „Berechnung“ anerkennt und das eine nicht Turing-berechenbare Funktion liefert.* **In neunzig Jahren hat es niemand geschafft.**

7.1 Warum man sie trotzdem glaubt

Vier Argumente

Erstens: Die Konfluenz der Modelle.

Zwischen 1931 und 1936 wurden mehrere völlig verschiedene Präzisierungen vorgeschlagen – **von Leuten, die teilweise nichts voneinander wussten.** *Alle erwiesen sich als gleichwertig.*

Modell	**Urheber**	**Grundgedanke**
allgemein rekursive Fkt.	Gödel, Herbrand 1933	Gleichungssysteme
λ-Kalkül	Church 1936	Funktionsanwendung
Turingmaschine	Turing 1936	**Mensch mit Papier**
Postsystem	Post 1936	Zeichenkettenersetzung
μ-Rekursion	Kleene 1936	Grundfunktionen + Suche
Registermaschine	Shepherdson, Sturgis 1963	Zähler und Sprünge
Zellularautomat	von Neumann 1948	lokale Regeln

*Alle berechnen **genau dieselben** Funktionen.* **Das ist kein Beweis, aber ein starkes Indiz** – *denn die Zugänge haben nichts miteinander gemein.*

Zweitens: Turings Analyse.

*Turing hat nicht bloß ein Modell vorgeschlagen, sondern **argumentiert**, warum es vollständig ist* (Vorwort) – **aus den Beschränkungen des rechnenden Menschen.** *Das ist der Grund, warum Gödel gerade seinen Zugang für überzeugend hielt.*

Drittens: Neunzig Jahre erfolgloser Widerlegung.
Kein einziges der seither vorgeschlagenen Rechenmodelle – Quantenrechner, DNA-Rechner, neuronale Netze, analoge Maschinen – übersteigt die Turingberechenbarkeit.
Viertens: Die Robustheit gegen Erweiterungen.
Man kann der Maschine Orakel, Zufall, Nichtdeterminismus, beliebig viele Bänder geben – **die Klasse der berechenbaren Funktionen bleibt dieselbe** (Kapitel 5).

Zwei verschiedene Thesen – die Verwechslung ist verbreitet

Die klassische These	Was *überhaupt* berechenbar ist, ist Turing-berechenbar
Die „erweiterte" These	Was *effizient* berechenbar ist, ist es auch auf einer Turingmaschine (mit polynomiellem Verlust)

Die erste gilt als sicher. Die zweite ist durch Quantenrechner in Frage gestellt.

SHORS *Algorithmus (1994) faktorisiert Zahlen in polynomieller Zeit auf einem Quantenrechner; klassisch kennt man nur subexponentielle Verfahren.* **Wäre das ein Gegenbeispiel zur *erweiterten* These – aber nicht zur klassischen.**

Ein Quantenrechner kann nichts berechnen, was eine Turingmaschine nicht auch berechnet – **er ist möglicherweise nur schneller.** *Band 15 führt das aus.*

Was die These praktisch bedeutet

Sie erlaubt es, Beweise umgangssprachlich zu führen. Statt eine Turingmaschine zu konstruieren, schreibt man: „Man gehe alle Wörter durch und prüfe jeweils…" – und beruft sich darauf, dass sich das umsetzen ließe.

Ohne die These müsste jeder Beweis dieses Bandes eine explizite Maschine angeben. **Mit ihr genügt eine Beschreibung, die ein Programmierer umsetzen könnte.**

Man nennt das die Church-Turing-These als Beweismittel.

Erlaubt	„Die Maschine simuliert M auf w für t Schritte"
Erlaubt	„Sie zählt alle Paare (i, j) auf"
Nicht erlaubt	„Sie prüft, ob M jemals hält"

Die dritte Zeile ist die Grenze: **Erlaubt ist alles, was ein Programmierer wirklich schreiben könnte** – *nicht alles, was man sich wünschen kann.*

Aufgabe 7.1.

(a) Führen Sie die μ-rekursiven Funktionen ein und zeigen Sie, dass die Ackermannfunktion nicht primitiv rekursiv ist.

(b) Zeigen Sie, dass eine Registermaschine mit zwei Zählern universell ist.

(c) Untersuchen Sie den Lambda-Kalkül an einem Beispiel.

Lösung Schritt für Schritt.

(a) μ-Rekursion und die Ackermannfunktion.
Schritt 1 – Die primitiv rekursiven Funktionen.

Grundfunktionen	Nullfunktion **0**, Nachfolger $s(x) = x + 1$, Projektionen π_i^n
Komposition	$h(\vec{x}) = f(g_1(\vec{x}), \ldots, g_k(\vec{x}))$
Primitive Rekursion	$h(\vec{x}, 0) = f(\vec{x})$, $h(\vec{x}, n+1) = g(\vec{x}, n, h(\vec{x}, n))$

Jede primitiv rekursive Funktion ist total – die Rekursion zählt streng abwärts und endet garantiert. **Das ist zugleich ihr Vorzug und ihre Schwäche.**

Beispiele: Addition, Multiplikation, Potenz, Fakultät, Primzahltest, ggT – **alles primitiv rekursiv.** *Man kommt sehr weit.*
Schritt 2 – Die Ackermannfunktion.

$$A(0,n) = n+1, \qquad A(m+1,0) = A(m,1), \qquad A(m+1,n+1) = A\big(m,\, A(m+1,n)\big).$$

m	$A(m,n)$	**Wachstum**
0	$n+1$	Nachfolger
1	$n+2$	
2	$2n+3$	linear
3	$2^{n+3}-3$	exponentiell
4	$2^{2^{\cdot^{\cdot^{2}}}} - 3$ ($n+3$ Zweien)	**Turm**

Konkret:

$$A(4,0) = 13, \qquad A(4,1) = 65\,533, \qquad A(4,2) = 2^{65\,536} - 3.$$

$A(4,2)$ *hat etwa 19 729 Dezimalstellen.* $A(4,3)$ **lässt sich nicht mehr aufschreiben.**

Schritt 3 – Warum sie nicht primitiv rekursiv ist.

Satz 7.1. *Zu jeder primitiv rekursiven Funktion f gibt es ein k mit*

$$f(\vec{x}) < A\big(k,\ \max \vec{x}\big) \qquad \textit{für alle } \vec{x}.$$

Beweisskizze durch Induktion über den Aufbau von f:

Grundfunktionen	werden von $A(1,\cdot)$ übertroffen
Komposition	Die Schranken addieren sich; k wächst um eine Konstante
Primitive Rekursion	k wächst um 1 – genau das leistet die dritte Ackermann-Gleichung

Der Schluss.
Wäre A selbst primitiv rekursiv, so auch $d(n) := A(n,n)$. Nach dem Satz gäbe es ein k mit

$$d(n) < A(k,n) \qquad \text{für alle } n.$$

Setze $n := k$:

$$A(k,k) = d(k) < A(k,k). \qquad \text{Widerspruch.}$$

> ***Wieder ein Diagonalargument*** – man setzt die Funktion auf der Diagonale mit sich selbst in Beziehung. **Dieselbe Bewegung wie bei Cantor, bei Turing und beim Rekursionstheorem.**

Schritt 4 – Die Folgerung: der μ-Operator ist nötig.

Definition 7.2. Der **μ-Operator** (unbeschränkte Suche):

$$\mu y\,[f(\vec{x},y) = 0] := \text{das kleinste } y \text{ mit } f(\vec{x},y) = 0,$$

undefiniert, falls es kein solches y gibt.

> ***Der μ-Operator ist genau die `while`-Schleife*** – die einzige Konstruktion, die nicht garantiert terminiert. **Und sie ist unverzichtbar:**

Klasse	**Sprachmittel**	**Terminierung**
primitiv rekursiv	nur `for`	immer
μ-rekursiv	auch `while`	**nicht immer**

*Und die μ-rekursiven Funktionen sind **genau** die Turing-berechenbaren* (Kleene 1936).

Der Preis der Universalität

Eine Sprache, die nur `for`*-Schleifen hat, ist* ***total****: Jedes Programm hält, und das Halteproblem ist trivial.* **Sie kann aber nicht alles berechnen – und insbesondere keinen Interpreter für sich selbst.**

	total	**universell**
Halteproblem	trivial	unlösbar
Selbstinterpreter	unmöglich	existiert
Beispiele	Coq, Agda	C, Python, Go

Warum ein Selbstinterpreter unmöglich ist: Gäbe es ihn, so könnte man das Diagonalargument nachbauen und erhielte eine Funktion außerhalb der Klasse. **Die Totalität schließt sich selbst aus.** ✓

Man kann nicht beides haben **– Band 0, Kapitel 15 hat das als Schlussgedanken formuliert, und hier ist der Beweis dafür.**

(b) Zwei Zähler genügen.

Satz 7.3 (Minsky, 1961)**.** *Eine Maschine mit* ***zwei*** *unbeschränkten Zählregistern, die nur erhöhen, erniedrigen und auf null testen kann, ist Turing-vollständig.*

Das ist ein bemerkenswert starkes Ergebnis. Zwei Zahlen, drei Operationen – und daraus alles.

Schritt 1 – Drei Zähler simulieren eine Turingmaschine.
Das Band zerfällt in den Teil links und rechts vom Kopf. Beide Teile liest man als Zahlen zur Basis $|\Gamma|$*:*

Zähler L	der linke Bandteil als Zahl
Zähler R	der rechte Bandteil als Zahl
Zähler H	Hilfsregister

Ein Schritt nach rechts heißt dann: $L := L \cdot b + (R \bmod b)$ **und** $R := \lfloor R/b \rfloor$ *– beides mit Zählern und dem Hilfsregister ausführbar.* ✓
Schritt 2 – Drei Zähler auf zwei.
Man kodiert die drei Zähler (a, b, c) *in* ***einer*** *Zahl:*

$$n := 2^a \cdot 3^b \cdot 5^c.$$

Nach dem Satz von der eindeutigen Primfaktorzerlegung (Mathematik-Band 1) *lässt sich* (a, b, c) *aus* n *eindeutig zurückgewinnen.* ✓

a erhöhen	$n := n \cdot 2$
a erniedrigen	$n := n/2$
$a = 0$?	Ist n durch 2 teilbar?

Multiplikation und Division mit Rest lassen sich mit ***zwei*** *Zählern durchführen – man zählt den einen herunter und den anderen entsprechend hoch.* **Der zweite Zähler ist das Hilfsregister.** ✓

Der Preis ist absurd

Die Simulation ist korrekt und praktisch völlig unbrauchbar.

Um den Zähler a um 1 zu erhöhen, muss man n verdoppeln – und das heißt, n-mal herunter- und $2n$*-mal hochzuzählen.* **Ein einziger Turingschritt kostet exponentiell viele Zählschritte.**

Die Verlangsamung ist etwa $2^{\mathcal{O}(t)}$. **Aber die *Berechenbarkeit* bleibt unberührt** – *und genau darum geht es in diesem Band.*

Mit ***einem*** *Zähler geht es übrigens nicht:* **Eine Ein-Zähler-Maschine ist nicht mächtiger als ein Kellerautomat.** *Der Sprung liegt genau zwischen eins und zwei.*

(c) Der Lambda-Kalkül.

Churchs Modell kennt ***keine*** *Zahlen, keine Zustände und kein Band – nur Funktionen.*

Term	**Form**	**Bedeutung**
Variable	x	
Abstraktion	$\lambda x.\, M$	Funktion mit Parameter x
Anwendung	$M\, N$	M auf N angewandt

Die einzige Rechenregel ist die ***β-Reduktion****:*

$$(\lambda x.\, M)\, N \;\to\; M[x := N].$$

Zahlen im Lambda-Kalkül.
***Church-Zahlen**: n ist die Funktion, die eine Funktion n-mal anwendet.*

0	$\lambda f.\,\lambda x.\,x$
1	$\lambda f.\,\lambda x.\,f\,x$
2	$\lambda f.\,\lambda x.\,f\,(f\,x)$
3	$\lambda f.\,\lambda x.\,f\,(f\,(f\,x))$

Die Addition.

$$\mathrm{ADD} := \lambda m.\,\lambda n.\,\lambda f.\,\lambda x.\,m\,f\,(n\,f\,x).$$

Nachrechnen von ADD 1 2*:*

$$\to \lambda f.\lambda x.\,1\,f\,(2\,f\,x)$$
$$\to \lambda f.\lambda x.\,f\,\big(2\,f\,x\big) \to \lambda f.\lambda x.\,f\,\big(f\,(f\,x)\big) = 3. \qquad \checkmark$$

„m-mal anwenden, dann n-mal anwenden“ ist „$(m+n)$-mal anwenden“ – **die Addition ist buchstäblich die Hintereinanderausführung.**

Rekursion ohne Namen: der Y-Kombinator.

Im Lambda-Kalkül kann sich eine Funktion nicht bei ihrem Namen aufrufen – sie hat keinen. **Churchs Ausweg:**

$$Y := \lambda f.\,(\lambda x.\,f\,(x\,x))\,(\lambda x.\,f\,(x\,x)).$$

Rechnen wir $Y\,F$ aus:

$$Y\,F \to (\lambda x.\,F\,(x\,x))\,(\lambda x.\,F\,(x\,x))$$
$$\to F\,\Big((\lambda x.\,F\,(x\,x))\,(\lambda x.\,F\,(x\,x))\Big) = F\,(Y\,F).$$

$$\boxed{Y\,F = F\,(Y\,F) \qquad -- \text{ ein } \textbf{Fixpunkt} \text{ von } F}$$

Der Term $x\,x$ ist der Kern: eine Funktion, auf sich selbst angewandt. Dieselbe Selbstanwendung wie in $\varphi_x(x)$ beim Rekursionstheorem und beim Halteproblem.

Damit lässt sich die Fakultät ohne Namen schreiben:

$$\mathrm{FAK} := Y\,\big(\lambda f.\,\lambda n.\,\text{if } n = 0 \text{ then } 1 \text{ else } n \cdot f(n-1)\big).$$

Die innere Funktion bekommt sich selbst als Parameter f übergeben – **der Y-Kombinator besorgt das.**

Der Lambda-Kalkül ist nicht bloß historisch bedeutsam. Er ist die theoretische Grundlage jeder funktionalen Sprache – Lisp, ML, Haskell, und die Lambda-Ausdrücke in Python, Java und C++ tragen seinen Namen zu Recht. **Band 6 führt das aus.**

Churchs Arbeit[3] ist der Ursprung; Turing[27] bewies 1937 die Äquivalenz; Boolos, Burgess und Jeffrey[1] und Cutland[4] entwickeln die μ-Rekursion sauber; Minsky[16] ist die Quelle für die Zweizählermaschine.

Part III

Die Grenze

8 Das Halteproblem

Die Sprache, um die es geht

Definition 8.1.

$$\mathrm{H} := \{\langle M\rangle \# w \ : \ M \text{ hält bei Eingabe } w\}.$$

Die **Diagonalfassung** ist

$$\mathrm{H}_0 := \{\langle M\rangle \ : \ M \text{ hält bei Eingabe } \langle M\rangle\}.$$

*Die zweite Fassung wirkt künstlich – warum sollte man eine Maschine auf ihren **eigenen Code** anwenden?* **Weil genau das den Widerspruch erzeugt** – *und weil die beiden Fassungen aufeinander reduzierbar sind* (Kapitel 9).

Satz 8.2 (Turing, 1936)**.** H *ist **aufzählbar**, aber **nicht entscheidbar**.*

8.1 Der Beweis

Schritt für Schritt

Teil 1 – H ist aufzählbar.
Die universelle Maschine U aus Kapitel 6 leistet es:

1. Prüfe, ob die Eingabe die Form $\langle M\rangle \# w$ hat; wenn nicht, verwirf.
2. Simuliere M auf w.
3. Hält die Simulation, so akzeptiere.

Hält M auf w, so hält die Simulation nach endlich vielen Schritten und akzeptiert. **Hält M nicht, so läuft die Simulation ewig** – *was für Aufzählbarkeit erlaubt ist.* ✓

Teil 2 – H ist nicht entscheidbar.
Annahme. *Es gäbe einen Entscheider H mit*

$$H(\langle M\rangle \# w) = \begin{cases} \text{ja} & \text{falls } M \text{ auf } w \text{ hält,} \\ \text{nein} & \text{sonst,} \end{cases}$$

*und H hält **immer**.*
Konstruktion. *Wir bauen daraus eine Maschine D, die **eine** Eingabe nimmt:*

`D(x):`

`  if H(x # x) = ja then` *gehe in eine Endlosschleife*

`  else` *halte an*

---

*D fragt: „Hält die Maschine x, wenn man ihr ihren eigenen Code gibt?“ – und tut dann das **Gegenteil**.*

D ist ein völlig gewöhnliches Programm. **Wenn H existiert, existiert auch D,** *und D hat eine Gödelnummer $\langle D \rangle$.*

Der Widerspruch. *Wir fragen: Hält D auf der Eingabe $\langle D \rangle$?*

Fall 1: $D(\langle D \rangle)$ hält	Dann liefert $H(\langle D \rangle \# \langle D \rangle)$ die Antwort „ja“. Also geht D nach seiner Vorschrift in die Endlosschleife. **Also hält $D(\langle D \rangle)$ nicht. Widerspruch.**
Fall 2: $D(\langle D \rangle)$ hält nicht	Dann liefert H die Antwort „nein“. Also hält D nach seiner Vorschrift an. **Also hält $D(\langle D \rangle)$. Widerspruch.**

D hält oder hält nicht – ein Drittes gibt es nicht. Beide Fälle führen auf einen Widerspruch. **Also war die einzige Annahme falsch:** H existiert nicht.

Wo genau die Annahme eingeht

Man sollte sich klarmachen, welche Eigenschaft von H den Widerspruch trägt.

H ist **total**	Deshalb ist auch D auf jeder Eingabe definiert – die Fallunterscheidung ist erschöpfend
H ist **korrekt**	Deshalb stimmt die Antwort mit der Wirklichkeit überein
H ist ein **Programm**	Deshalb hat D eine Gödelnummer und darf sich selbst gefüttert werden

Die dritte Zeile ist die tiefste. Ohne die Kodierbarkeit von Programmen als Daten (Kapitel 6) gäbe es keinen Selbstbezug – und ohne Selbstbezug keinen Widerspruch.

Lässt man eine der drei Eigenschaften fallen, so bricht der Beweis zusammen:

H darf „weiß nicht“ sagen	Dann ist D nicht mehr definiert – **solche H gibt es**
H darf irren	Dann gibt es Näherungen – **gibt es auch**
H ist kein Programm	**Ein Orakel** (Kapitel 11)

Die Diagonale, sichtbar gemacht

Man stelle sich eine unendliche Tabelle vor: Zeilen sind Maschinen, Spalten sind deren Kodierungen.

	$\langle M_0 \rangle$	$\langle M_1 \rangle$	$\langle M_2 \rangle$	…
M_0	**hält**	hält nicht	hält	…
M_1	hält	**hält nicht**	hält	…
M_2	hält nicht	hält	**hält**	…
…				
D	**hält nicht**	**hält**	**hält nicht**	…

D ist so gebaut, dass es sich in Spalte i von M_i ***unterscheidet.*** **Also ist D keine der Maschinen M_i.**

Der Unterschied zu Cantor – er ist entscheidend

	Cantor	**Turing**
Liste enthält	alle reellen Zahlen	**alle Maschinen**
Diagonalobjekt	steht *nicht* in der Liste	**muss darin stehen**
Folgerung	Liste war unvollständig	H **existiert nicht**

Bei Cantor darf d fehlen – die Liste war eben nicht vollständig, und daraus folgt die Überabzählbarkeit.

Bei Turing* muss D *in der Liste stehen, denn die Liste enthält **alle** Maschinen, und D ist eine. **Also muss stattdessen die andere Zutat falsch sein – die Existenz von H.**

Die Beweistechnik ist dieselbe, der Schluss ein anderer. **Das lohnt sich zu merken, denn beide Formen kommen im Rest des Bandes vor.**

Aufgabe 8.1.

(a) Führen Sie den Beweis in einer Programmiersprache aus und untersuchen Sie, warum die üblichen Einwände nicht greifen.

(b) Zeigen Sie, dass $\overline{\mathrm{H}}$ nicht einmal aufzählbar ist, und leiten Sie daraus weitere Unentscheidbarkeiten her.

(c) Untersuchen Sie, was ein *Zeitschranken*-Halteproblem ändert.

Lösung Schritt für Schritt.

(a) Der Beweis in Pseudocode.
Angenommen, es gäbe eine Funktion `haelt(programm, eingabe)`*, die stets terminiert und korrekt antwortet.*

```
def D(x):
  if haelt(x, x):
    while True:  pass     niemals fertig
  else:
    return     sofort fertig
```

```
quelltext = lies_eigene_datei()     oder per Quine
D(quelltext)
```

Nach dem Rekursionstheorem (Kapitel 6) *braucht man das Lesen der eigenen Datei gar nicht* – **jedes Programm kann seinen eigenen Quelltext erzeugen.** *Der Beweis funktioniert also auch ohne Dateisystem.*

Die vier üblichen Einwände.
Einwand 1: *„Man müsste* `haelt` *eben verbieten, sich selbst zu analysieren."*

Dann ist `haelt` *kein Entscheider mehr für* alle *Programme* – **und der Satz behauptet nichts anderes.** *Es gibt sehr wohl Verfahren für eingeschränkte Klassen.*

Einwand 2: *„Man könnte* `haelt` *drei Antworten geben lassen: ja, nein, weiß nicht."*

Das ist erlaubt und wird gemacht. **Ein solches Verfahren ist aber kein *Entscheider*** – *und es muss auf unendlich vielen Eingaben „weiß nicht" sagen.*

Denn: Sagte es nur endlich oft „weiß nicht", so könnte man diese endlich vielen Fälle in einer Tabelle nachschlagen und hätte einen Entscheider. **Widerspruch.** ✓

Einwand 3: *„Reale Rechner haben endlichen Speicher, also ist alles entscheidbar."*

Formal richtig: Ein Rechner mit b Bit Speicher hat 2^b *Zustände; wiederholt er einen, so läuft er ewig.* **Das Verfahren wäre:** 2^b **Schritte simulieren und schauen.**

Für $b = 10^{10}$ (etwa ein Gigabyte) wären das

$$2^{10^{10}} \text{ Schritte,}$$

eine Zahl mit etwa drei Milliarden Dezimalstellen. **Das Universum hat etwa 10^{80} Teilchen.**

Die Aussage ist wahr und ohne jeden Gehalt. **Sie ist der Grund, warum man das unbegrenzte Band idealisiert** (Kapitel 5).

Einwand 4: *„Ein Mensch kann doch oft sehen, ob ein Programm hält."*

Oft ja, immer nein. **Man betrachte das Programm:** *„Suche das kleinste Gegenbeispiel zur Collatz-Vermutung und halte dann."*

Ob es hält, weiß seit 1937 niemand (Band 1, Kapitel 13). **Und man kann für jede offene mathematische Vermutung ein solches Programm schreiben** – *Kapitel 14 zeigt, wie weit das führt.*

*Übrigens: Wäre der Mensch tatsächlich besser, so wäre **er** ein Gegenbeispiel zur Church-Turing-These* (Kapitel 7) – **eine These, die PENROSE tatsächlich vertreten hat und die weithin abgelehnt wird.**

(b) Das Komplement.

Satz 8.3. $\overline{\mathrm{H}}$ *ist nicht aufzählbar.*

Beweis: Wären H ***und*** $\overline{\mathrm{H}}$ *aufzählbar, so wäre* H *nach dem Satz von Post* (Satz 5.3) *entscheidbar* – **im Widerspruch zu Satz 8.2.**

Damit gibt es drei Stufen:

Sprache	**aufzählbar**	**entscheidbar**
$\{0^n 1^n\}$	ja	ja
H	ja	nein
$\overline{\mathrm{H}}$	**nein**	nein

Was das anschaulich heißt.

M **hält**	lässt sich **nachweisen**: man simuliere und warte
M **hält nicht**	lässt sich **nicht** nachweisen – kein endlicher Beleg

*Für „hält“ gibt es immer ein **Zertifikat**: die Zahl der Schritte.* **Für „hält nicht“ gibt es keines** – *man kann noch so lange warten, ohne je sicher zu sein.*

*Diese Asymmetrie ist derselbe Sachverhalt, der in der Wissenschaftstheorie als **Falsifizierbarkeit** auftritt: Eine Allaussage lässt sich widerlegen, aber nicht verifizieren.*
Folgerungen – weitere unentscheidbare Probleme.

Problem	**aufzählbar?**	**entscheidbar?**
Hält M auf w?	ja	nein
Hält M auf *irgendeiner* Eingabe?	ja	nein
Hält M auf **allen** Eingaben?	nein	nein
Ist $L(M) = \emptyset$?	nein	nein
Ist $L(M)$ endlich?	nein	nein
Ist $L(M_1) = L(M_2)$?	nein	nein

Die Zeilen 2 und 3 sind lehrreich: ***„es gibt ein w“ bleibt aufzählbar, „für alle w“ nicht.*** Man kann alle Paare (w, t) durchgehen und auf einen Erfolg warten – aber man kann nicht auf unendlich viele Erfolge warten.

Kapitel 11 macht daraus eine Hierarchie: Jeder zusätzliche Wechsel von „es gibt“ zu „für alle“ steigt eine Stufe.

(c) Das beschränkte Halteproblem.

Definition 8.4.

$$\mathrm{H}_t := \{\langle M\rangle \# w \# 1^t \ : \ M \text{ hält auf } w \text{ innerhalb von } t \text{ Schritten}\}.$$

Satz 8.5. H_t *ist **entscheidbar**.*

Beweis: Simuliere M auf w für höchstens t Schritte und schaue nach. **Das terminiert immer.** ✓

Die Unentscheidbarkeit entsteht also allein durch die unbeschränkte Wartezeit. Sobald eine Schranke da ist, ist alles trivial.

Warum t in *unärer* Schreibweise angegeben wird

Die Kodierung 1^t statt der Binärdarstellung ist kein Schönheitsfehler.

Kodierung von t	Eingabelänge	Aufwand
unär (1^t)	$\Theta(t)$	polynomiell
binär	$\Theta(\log t)$	**exponentiell**

*Bei binärer Kodierung ist die Eingabe kurz und die Simulation lang – das Problem wird **EXPTIME-vollständig**.* **Bei unärer ist es NP-vollständig für nichtdeterministische Maschinen** – *und tatsächlich **das** generische NP-vollständige Problem* (Band 3).

Dieselbe Unterscheidung wie beim Rucksackproblem (Band 1, Kapitel 13): **pseudopolynomiell gegen polynomiell.**

Was das für die Praxis heißt.

Testen	Ein Testlauf mit Zeitlimit ist ein Entscheider für H_t
Zeitschranken	Jeder Webserver bricht nach n Sekunden ab
Gasgrenzen	Ethereum-Verträge zahlen für jeden Schritt – Terminierung ist **erzwungen**

Die dritte Zeile ist die eleganteste Umgehung des Halteproblems, die die Praxis hervorgebracht hat. Ethereums „Gas“ macht aus einer universellen Sprache eine total terminierende, ohne die Sprache selbst einzuschränken **– man begrenzt die *Ressource*, nicht die Ausdruckskraft.**

Der Preis: Ein Programm kann mitten in der Ausführung abbrechen, und man muss festlegen, was dann gilt. **Genau daraus sind mehrere teure Fehler in Verträgen entstanden.** Turings Arbeit[27] enthält den Beweis in Abschnitt 8; Sipser[24] führt ihn besonders klar; Davis[5] und Rogers[22] entwickeln die Theorie dahinter; Davis' Sammelband[6] enthält die Originalarbeiten von Gödel, Church, Turing, Kleene, Rosser und Post im Faksimile.

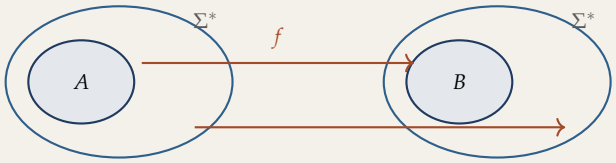
9 Reduktionen und der Satz von Rice

Die wichtigste Beweistechnik des Bandes

*Für **jedes** weitere unentscheidbare Problem einen eigenen Diagonalbeweis zu führen, wäre mühsam.* **Stattdessen führt man es auf das Halteproblem zurück.**

Definition 9.1. A ist auf B **many-one-reduzierbar**, geschrieben $A \leq_m B$, wenn es eine **totale berechenbare** Funktion f gibt mit

$$w \in A \iff f(w) \in B \qquad \text{für alle } w.$$



*f bildet A nach B ab **und** das Komplement nach dem Komplement.* **Das ist die entscheidende Bedingung.**

Satz 9.2. *Gilt $A \leq_m B$, so folgt:*

B entscheidbar	$\Longrightarrow$ *A entscheidbar*
B aufzählbar	$\Longrightarrow$ *A aufzählbar*
*A **unentscheidbar***	$\Longrightarrow$ *B **unentscheidbar***
A nicht aufzählbar	$\Longrightarrow$ *B nicht aufzählbar*

Beweis der ersten Zeile: Ist E ein Entscheider für B, so entscheidet „berechne $f(w)$, dann rufe E auf" die Sprache A. **Beides terminiert, also terminiert die Hintereinanderschaltung.** ✓
Die unteren beiden Zeilen sind die Kontraposition der oberen. ✓

Die Richtung ist die Fehlerquelle

Um zu zeigen, dass B unentscheidbar ist, reduziert man ein bekannt unentscheidbares A auf B – *also* $A \leq_m B$, *nicht* *umgekehrt.*

> ***Merkregel:*** *Man zeigt, dass B* mindestens so schwer *ist wie A.* **Der Pfeil zeigt vom Bekannten zum Neuen.**

Die verkehrte Richtung $B \leq_m A$ zeigte, dass B ***höchstens*** *so schwer ist* – **und daraus folgt gar nichts.**

Dieselbe Falle gibt es bei NP-Vollständigkeitsbeweisen (Band 3) *und sie ist dort die häufigste Fehlerquelle überhaupt.*

Und f muss *total* sein.

> *Eine Reduktion darf nicht selbst hängenbleiben* – **sonst wäre die Hintereinanderschaltung kein Entscheider.** *Deshalb ist es wesentlich, dass f ein Programm* konstruiert, *ohne es auszuführen* (das *s-m-n*-Theorem, Kapitel 6).

9.1 Ein durchgeführtes Beispiel

Das leere Sprachproblem

Definition 9.3.

$$L_\emptyset := \{\langle M\rangle \;:\; L(M) = \emptyset\}.$$

Satz 9.4. *$L_\emptyset$ ist unentscheidbar.*

Der Beweis: $H \leq_m \overline{L_\emptyset}$.
Gegeben sei eine Eingabe $\langle M\rangle\#w$. Wir konstruieren daraus eine neue Maschine N:

```
N(y):      y wird ignoriert
  simuliere M auf w;
  wenn die Simulation hält:  akzeptiere
```

> *N* ***ignoriert seine eigene Eingabe vollständig.*** Das ist der Kunstgriff: *N* akzeptiert entweder *alles* oder *nichts*, je nachdem, ob *M* auf *w* hält.

Die Abbildung $f(\langle M\rangle\#w) := \langle N\rangle$ ist berechenbar – **man schreibt bloß den Quelltext von N hin, mit $\langle M\rangle$ und w fest eingebaut.** *Das leistet das s-m-n-Theorem.* ✓
Die Äquivalenz nachprüfen.

M **hält auf** w	$L(N) = \Sigma^*$	$\langle N \rangle \notin L_\varnothing$
M **hält nicht**	$L(N) = \varnothing$	$\langle N \rangle \in L_\varnothing$

$$\langle M \rangle \# w \in \mathrm{H} \iff f(\langle M \rangle \# w) \in \overline{L_\varnothing}. \qquad \checkmark$$

Da H *unentscheidbar ist, ist* $\overline{L_\varnothing}$ *unentscheidbar* – **und damit auch** $L_\varnothing$, *denn Entscheidbarkeit ist unter Komplement abgeschlossen.*

> ***Man beachte, was hier* nicht *geschieht:*** *f führt M* ***nicht*** *aus.* **Es baut nur ein Programm, das M ausführen *würde*.** *Deshalb ist f total.*

9.2 Der Satz von Rice

Alles auf einmal

> *Statt für jede Eigenschaft einzeln zu reduzieren, beweist man den allgemeinen Satz* – **und er ist von erstaunlicher Reichweite.**

Definition 9.5. Eine Menge S von **Funktionen** (nicht von Programmen) heißt eine **semantische Eigenschaft**. Die zugehörige Sprache ist

$$L_S := \{\langle M \rangle \ : \ \varphi_{\langle M \rangle} \in S\}.$$

S heißt **trivial**, wenn $S = \varnothing$ oder S alle berechenbaren Funktionen enthält.

Satz 9.6 (Rice, 1953). *Für jede* ***nichttriviale*** *semantische Eigenschaft S ist L_S unentscheidbar.*

> ***In Worten:*** *Über das* Verhalten *eines Programms lässt sich* ***gar nichts*** *entscheiden – außer den beiden trivialen Antworten „immer ja“ und „immer nein“.*

Der Beweis.
Sei $\perp$ die überall undefinierte Funktion (das Programm, das nirgends hält).
Fall A: *$\perp \notin S$. (Der andere Fall geht durch Übergang zum Komplement.)*
Da S nichttrivial ist, gibt es eine berechenbare Funktion $g \in S$, etwa berechnet von der Maschine G.
Wir reduzieren $\mathrm{H} \leq_\mathrm{m} L_S$. *Zu $\langle M \rangle \# w$ konstruiere:*

```
N(y):
  simuliere M auf w;      Ergebnis wird verworfen
  wenn das hält:  gib G(y) zurück
```

Die Fallunterscheidung.

M **hält auf** w	$\varphi_N = g$	$g \in S$, also $\langle N \rangle \in L_S$
M **hält nicht**	$\varphi_N = \bot$	$\bot \notin S$, also $\langle N \rangle \notin L_S$

Die erste Zeile: Hält die Simulation, so verhält sich N auf jeder Eingabe genau wie G. ✓

Die zweite: Hält sie nicht, so kommt N nie zur zweiten Zeile und ist überall undefiniert. ✓

$$\langle M \rangle \# w \in \mathrm{H} \iff \langle N \rangle \in L_S.$$

Was der Satz *nicht* sagt

Er redet über *semantische* Eigenschaften – über das, was ein Programm *tut*.

Frage	Art	entscheidbar?
Hält M auf allen Eingaben?	semantisch	nein
Berechnet M die Nullfunktion?	semantisch	nein
Ist $L(M)$ regulär?	semantisch	nein
Hat M weniger als 100 Zustände?	**syntaktisch**	**ja**
Kommt in $\langle M \rangle$ das Symbol q_7 vor?	**syntaktisch**	**ja**
Hält M auf w in 100 Schritten?	**syntaktisch***	**ja**

* *genauer: keine Eigenschaft der berechneten* Funktion, *sondern des Ablaufs.*

> ***Die Trennlinie verläuft zwischen Text und Bedeutung.*** Alles, was man dem Programmtext ansieht, ist entscheidbar; alles, was man erst durch Ausführen erführe, nicht.

Und der Satz sagt nichts über *Näherungen*. *Ein Verfahren, das oft richtig antwortet und manchmal „weiß nicht“ sagt, bleibt möglich* – **und die gesamte Programmanalyse besteht aus solchen Verfahren.**

Aufgabe 9.1.

(a) Führen Sie drei Reduktionen vollständig durch.
(b) Wenden Sie den Satz von Rice auf praktische Werkzeuge an.
(c) Untersuchen Sie eine Eigenschaft, auf die der Satz *nicht* anwendbar ist.

Lösung Schritt für Schritt.

(a) Drei Reduktionen.
Reduktion 1 – $\mathrm{H}_0 \leq_{\mathrm{m}} \mathrm{H}$.

H_0 *fragt: Hält M auf* $\langle M \rangle$*?*

$$f(\langle M \rangle) := \langle M \rangle \# \langle M \rangle.$$

f ist offensichtlich total und berechenbar – man verdoppelt bloß die Eingabe. ✓

$$\langle M \rangle \in H_0 \iff \langle M \rangle \# \langle M \rangle \in H. \qquad \checkmark$$

Reduktion 2 – $H \leq_m H_0$ *(die interessantere Richtung).*
Zu $\langle M \rangle \# w$ *konstruiere die Maschine*

```
N(y):      y wird ignoriert
  simuliere M auf w
```

Dann hält N auf ***jeder*** *Eingabe genau dann, wenn M auf w hält – insbesondere auf* $\langle N \rangle$*.*

$$\langle M \rangle \# w \in H \iff \langle N \rangle \in H_0. \qquad \checkmark$$

Beide Richtungen gelten – die Probleme sind gleich schwer. *Man schreibt* $H \equiv_m H_0$.

Reduktion 3 – *Das Totalitätsproblem.*

$$\text{TOT} := \{\langle M \rangle : M \text{ hält auf } \textbf{jeder} \text{ Eingabe}\}.$$

Behauptung: $\overline{H} \leq_m$ TOT *– also ist* TOT ***nicht aufzählbar****.*
Zu $\langle M \rangle \# w$ *konstruiere:*

```
N(y):
  simuliere M auf w für genau |y|
Schritte;
  wenn M in dieser Zeit gehalten hat:
gehe in Endlosschleife;
  sonst:  halte
```

Die Fallunterscheidung.

M **hält nie auf** w	Für jedes y hat M nach $\lvert y \rvert$ Schritten **nicht** gehalten $\Longrightarrow$ N hält immer $\Longrightarrow$ $\langle N \rangle \in$ TOT
M **hält nach** k **Schritten**	Für alle y mit $\lvert y \rvert \geq k$ geht N in die Endlosschleife $\Longrightarrow$ $\langle N \rangle \notin$ TOT

$$\langle M \rangle \# w \in \overline{\mathrm{H}} \iff \langle N \rangle \in \mathrm{TOT}. \qquad \checkmark$$

Da $\overline{\mathrm{H}}$ *nicht aufzählbar ist* (Kapitel 8) *und die Reduktion die Aufzählbarkeit überträgt* (Satz 9.2), **ist** TOT **nicht aufzählbar.**

Das ist bemerkenswert: **„Hält** M **auf** w**?“ ist wenigstens halb lösbar. „Hält** M **immer?“ ist nicht einmal das.**

(b) Der Satz von Rice in der Praxis.

Werkzeug	**Was der Satz verbietet**
Virenscanner	„Verhält sich P bösartig?“ ist semantisch
Optimierer	„Ist dieser Zweig erreichbar?“ ist semantisch
Typprüfer	„Kann hier ein Typfehler auftreten?“ ist semantisch
Testabdeckung	„Wird diese Zeile je ausgeführt?“ ist semantisch
Refactoring	„Tun diese zwei Fassungen dasselbe?“ ist semantisch

Wie die Werkzeuge trotzdem arbeiten.

Virenscanner	Sucht *syntaktische* Muster (Signaturen) – die sind entscheidbar. Deshalb wirkt jede kleine Änderung des Schadcodes
Optimierer	Ist **konservativ**: Im Zweifel wird nicht optimiert
Typprüfer	Weist im Zweifel **zurück** – deshalb gibt es korrekte Programme, die nicht übersetzen

Die dritte Zeile ärgert jeden Programmierer irgendwann. Sie ist keine Nachlässigkeit der Sprachentwerfer – **sie ist der Satz von Rice.** *Ein Typsystem, das* genau *die korrekten Programme akzeptiert, kann es nicht geben.*

Die Richtung des Fehlers ist eine Entwurfsentscheidung.

	lässt Fehler durch	**lehnt Korrektes ab**
Typprüfer	nein	ja
Virenscanner	ja	selten

Ein Typprüfer, der Fehler durchließe, wäre nutzlos. **Ein Virenscanner, der jede zweite harmlose Datei blockierte, ebenfalls.** *Beide wählen die für ihren Zweck erträgliche Fehlerrichtung.*

(c) Wo der Satz nicht greift.
Beispiel 1 – Eine syntaktische Eigenschaft.

$$S_1 := \{\langle M\rangle : \langle M\rangle \text{ hat gerade Länge}\}.$$

Trivial entscheidbar – **und kein Widerspruch, denn S_1 ist keine Eigenschaft der *Funktion*.**

> ***Die Probe:*** *Nach dem Padding-Lemma* (Kapitel 6) *gibt es zu jeder Maschine gleichwertige mit gerader **und** ungerader Kodierungslänge.* **S_1 ist also nicht wohldefiniert als Funktionseigenschaft** – *genau das schließt Rice aus.*

Beispiel 2 – Eine triviale Eigenschaft.

$$S_2 := \{\text{alle berechenbaren Funktionen}\}.$$

$L_{S_2} = \Sigma^*$*, also entscheidbar* – **man sagt immer ja.** *Der Satz nimmt diesen Fall ausdrücklich aus.*

Beispiel 3 – Eine Eigenschaft des *Ablaufs*.

$$S_3 := \{\langle M\rangle : M \text{ hält auf } \varepsilon \text{ in höchstens 100 Schritten}\}.$$

Entscheidbar: 100 Schritte simulieren. ✓

> *Zwei Maschinen können dieselbe Funktion berechnen und trotzdem verschieden schnell sein* – **also ist S_3 keine Funktionseigenschaft.**

Der Satz von Rice hat eine schärfere Fassung

Rice sagt nur, dass L_S nicht *entscheidbar* ist – über die Aufzählbarkeit schweigt er.

Eigenschaft	aufzählbar?	
$L(M) \neq \emptyset$	ja	es genügt *ein* Zeuge
$L(M) = \emptyset$	nein	
$L(M)$ unendlich	nein	
M hält auf w	ja	
M hält immer	nein	

Der Satz von Rice-Shapiro (1955) klärt, welche Eigenschaften aufzählbar sind: genau die, die sich an **endlichen** Teilstücken des Funktionsverhaltens ablesen lassen.

*Anschaulich: Aufzählbar ist eine Eigenschaft genau dann, wenn ihr Zutreffen sich durch endlich viele Beobachtungen **bestätigen** lässt –* **„M akzeptiert das Wort w“ kann man beobachten, „M akzeptiert nichts“ nicht.**

Kapitel 11 ordnet diese Beobachtungen in eine Hierarchie ein – **und zeigt, dass sie nie endet.**

Rices Originalarbeit[21] ist vier Seiten lang; Sipser[24] und Kozen[13] führen viele Reduktionen vor; Rogers[22] entwickelt die Theorie der Reduzierbarkeiten systematisch; Soare[26] ist die moderne Standarddarstellung.

10 Aufzählbarkeit

Warum die Klasse so heißt

*Bisher wurde „aufzählbar“ über eine **akzeptierende** Maschine definiert* (Kapitel 5). **Der Name kommt aber von einer anderen, gleichwertigen Beschreibung.**

Satz 10.1. *Für $L \subseteq \Sigma^*$ sind gleichwertig:*

(1)	*Es gibt M, die genau die Wörter aus L akzeptiert.*
(2)	*Es gibt E, die nacheinander **alle** Wörter aus L ausgibt – in beliebiger Reihenfolge, mit Wiederholungen.*
(3)	*$L = \emptyset$ oder L ist Bild einer totalen berechenbaren Funktion $f : \mathbb{N} \to \Sigma^*$.*
(4)	*L ist Definitionsbereich einer berechenbaren Funktion.*

Beweis (1) $\Rightarrow$ (2) – hier steckt die Technik.

Der naive Versuch scheitert: „Gehe alle Wörter der Reihe nach durch und lasse M darauf laufen; gib aus, was akzeptiert wird.“

*Das geht schief, sobald M auf einem Wort **nicht** hält* – **dann kommt man nie zum nächsten.**

Die Lösung: Verzahnung (Dovetailing).

```
1.  Für t = 1, 2, 3, ...:
2.      Für jedes der ersten t Wörter w_1, ..., w_t:
3.          Lasse M auf w_i genau t Schritte laufen.
4.          Akzeptiert M dabei, so gib w_i aus.
```

***Jeder einzelne Durchlauf ist endlich* – also kommt man immer zum nächsten.** *Und akzeptiert M das Wort w_i nach k Schritten, so wird es spätestens in Runde $\max(i, k)$ ausgegeben.* ✓

Die Wörter werden mehrfach ausgegeben – in jeder späteren Runde wieder. **Das ist erlaubt und lässt sich mit etwas Buchführung vermeiden.**

Die Reihenfolge ist nicht beliebig wählbar

Ein Aufzähler gibt die Wörter in *irgendeiner* Reihenfolge aus.

Satz 10.2. *L ist genau dann **entscheidbar**, wenn es einen Aufzähler gibt, der die Wörter in **aufsteigender** Ordnung ausgibt.*

Beweis „⇐“: Um zu prüfen, ob $w \in L$, lasse man den Aufzähler laufen, bis er w ausgibt (dann ja) oder ein Wort $> w$ ausgibt (dann nein). **Eines von beiden tritt nach endlich vielen Schritten ein.** ✓

> *Ist L endlich, so hört der Aufzähler irgendwann auf – auch das ist ein endliches Signal.* ✓

> ***Die Sortierung ist also genau der Unterschied zwischen aufzählbar und entscheidbar.*** Bei H kann man alle haltenden Programme ausgeben – aber nicht der Reihe nach.

Die Abschlusseigenschaften

Operation	entscheidbar	aufzählbar
Vereinigung	ja	ja
Schnitt	ja	ja
Komplement	**ja**	**nein**
Verkettung	ja	ja
Kleene-Stern	ja	ja
Homomorphes Bild	**nein**	ja

Warum die Vereinigung geht. *Man verzahnt beide Maschinen; akzeptiert eine, akzeptiert man.* ✓
Warum der Schnitt geht. *Man lässt beide laufen und akzeptiert, wenn **beide** akzeptieren.* **Hier braucht man nicht einmal Verzahnung – man kann sie nacheinander laufen lassen, denn wenn die erste nicht hält, hält der Schnitt auch nicht.** ✓

> ***Die dritte Zeile ist der Kern des Kapitels.*** Wäre die Klasse unter Komplement abgeschlossen, so wären nach dem Satz von Post alle aufzählbaren Sprachen entscheidbar – **und H ist ein Gegenbeispiel.**

Die letzte Zeile ist überraschend: ***Entscheidbarkeit ist* nicht *unter homomorphen Bildern abgeschlossen.*** **Man kann durch Löschen von Zeichen aus einer entscheidbaren eine unentscheidbare Sprache machen** – *denn das Löschen verbirgt gerade die Information über die Rechenzeit.*

Aufgabe 10.1.

(a) Zeigen Sie die Gleichwertigkeit (2) $\Leftrightarrow$ (3) und untersuchen Sie den Sonderfall der endlichen Sprachen.

(b) Konstruieren Sie eine entscheidbare Sprache, deren homomorphes Bild unentscheidbar ist.

(c) Untersuchen Sie den Satz von Rice-Shapiro an Beispielen.

Lösung Schritt für Schritt.

(a) Aufzähler und Bildmenge.
Richtung (2) $\Rightarrow$ (3).
Sei E ein Aufzähler für $L \neq \emptyset$. Definiere

$$f(n) := \text{das } n\text{-te von } E \text{ ausgegebene Wort.}$$

Ist das total? Nur, wenn E unendlich viele Wörter ausgibt. **Bei endlichem L bleibt f stehen.**

Abhilfe: Man wiederhole das letzte Wort. Genauer – sei w_0 irgendein festes Element von L:

$$f(n) := \begin{cases} n\text{-te Ausgabe} & \text{falls } E \text{ mindestens } n \text{ Wörter liefert,} \\ w_0 & \text{sonst.} \end{cases}$$

Und ist dieses f berechenbar?

Hier steckt eine Feinheit. *Um den zweiten Fall zu erkennen, müsste man wissen, dass E nie mehr ausgibt* – **und das ist unentscheidbar.**

> ***Die Auflösung:*** *Man braucht die Fallunterscheidung gar nicht* zu treffen. **Man definiert:**
>
> $f(n) :=$ das zuletzt ausgegebene Wort nach n Rechenschritten von E,
>
> *und w_0, falls noch keines ausgegeben wurde.*

Das ist offensichtlich total und berechenbar – man simuliert n Schritte und schaut nach. ✓

Und das Bild von f ist genau L: **Jedes ausgegebene Wort wird getroffen, und nur solche.** ✓

> *Der Kunstgriff –* ***eine Schranke einführen, über die man laufen lässt*** *–* ist derselbe wie beim Dovetailing. **Er ersetzt eine unentscheidbare Frage durch eine entscheidbare Näherung.**

Richtung (3) $\Rightarrow$ (2).
Sei $L = f(\mathbb{N})$ mit totalem berechenbarem f. Der Aufzähler gibt einfach $f(0), f(1), f(2), \ldots$ aus. ✓

Jedes $f(n)$ ist nach endlich vielen Schritten berechnet, also kommt man immer weiter. **Hier braucht man keine Verzahnung, weil f total ist.**

Der Sonderfall $L = \emptyset$.

> *Die leere Menge ist Bild* ***keiner*** *totalen Funktion $\mathbb{N} \to \Sigma^*$* – denn eine solche liefert immer mindestens einen Wert. **Deshalb steht in (3) die Ausnahme.**

Und $\emptyset$ ist trivialerweise entscheidbar – **man sagt immer nein.** *Die Ausnahme ist also ein technischer Schönheitsfehler, kein sachlicher.*

(b) Ein homomorphes Bild.

Definition 10.3. Ein **Homomorphismus** $h : \Sigma^* \to \Delta^*$ ist durch die Bilder der einzelnen Zeichen festgelegt und auf Wörter zeichenweise fortgesetzt: $h(a_1 \ldots a_n) = h(a_1) \cdots h(a_n)$.

Er darf Zeichen ***löschen*** *($h(a) = \varepsilon$) – und darin liegt die Kraft.*

Die Konstruktion.

Betrachte über dem Alphabet $\Sigma = \{0, 1, \#, \square\}$ die Sprache

$$K := \{\langle M\rangle \# w \# \square^t \ : \ M \text{ hält auf } w \text{ in genau } t \text{ Schritten}\}.$$

Schritt 1 – K ist entscheidbar.
Man liest t aus der Zahl der Kästchen ab, simuliert M auf w für t Schritte und prüft, ob genau dann gehalten wurde. **Das terminiert immer.** ✓

> ***Der Trick ist, dass die Rechenzeit in der Eingabe* mitgeliefert *wird.*** Damit ist die Simulation von vornherein begrenzt.

Schritt 2 – Der Homomorphismus.

$$h(0) := 0, \qquad h(1) := 1, \qquad h(\#) := \#, \qquad h(\square) := \varepsilon.$$

Er löscht also genau die Kästchen – **und damit die Information über t.**

Schritt 3 – Das Bild.

$$h(K) = \{\langle M\rangle \# w \# \ : \ \exists t : M \text{ hält auf } w \text{ in } t \text{ Schritten}\} = \mathrm{H} \text{ (mit Zusatzzeichen)}.$$

K entscheidbar,	$h(K)$ unentscheidbar	✓

> ***Der Homomorphismus hat aus einem „für dieses t“ ein „es gibt ein t“ gemacht*** – und genau dieser Quantorensprung erzeugt die Unentscheidbarkeit.

Man sieht hier den Mechanismus, der in Kapitel 11 zur ganzen Hierarchie ausgebaut wird:
Jedes vorangestellte „es gibt“ kostet eine Stufe.

Und warum die aufzählbaren Sprachen doch abgeschlossen sind.
Ist L aufzählbar mit Aufzähler E, so zählt „gib $h(x)$ aus für jede Ausgabe x von E“ die Sprache $h(L)$ auf. ✓

> ***Aufzählbarkeit überträgt sich, Entscheidbarkeit nicht*** – weil man beim Aufzählen nie „nein“ sagen muss.

(c) Der Satz von Rice-Shapiro.

Satz 10.4 (Rice und Shapiro, 1955/56)**.** *Sei S eine Menge berechenbarer Funktionen und L_S aufzählbar. Dann gilt für jede berechenbare Funktion g:*

$$g \in S \iff \text{es gibt eine } \textbf{endliche} \text{ Teilfunktion } \theta \subseteq g \text{ mit } \theta \in S.$$

> ***In Worten:*** *Eine aufzählbare Verhaltenseigenschaft muss sich an* ***endlich vielen*** *Ein-Ausgabe-Paaren bereits entscheiden lassen.*

*Anschaulich: Wenn man beobachtend feststellen kann, dass die Eigenschaft **zutrifft**, dann nach endlich vielen Beobachtungen.* **Und was endlich viele Beobachtungen bestätigen, muss auch für jede Fortsetzung gelten.**

Anwendung 1 – TOT **ist nicht aufzählbar.**
Sei $S = \{\text{totale Funktionen}\}$.
*Die Nullfunktion $g \equiv 0$ ist total, also $g \in S$. Nach dem Satz müsste eine **endliche** Teilfunktion $\theta \subseteq g$ in S liegen.*

> *Aber eine endliche Teilfunktion ist **nicht total** –* sie ist nur an endlich vielen Stellen definiert. **Also liegt keine in** *S***.** *Widerspruch.*

Anwendung 2 – $L_\emptyset$ **ist nicht aufzählbar.**
Sei $S = \{\perp\}$, die überall undefinierte Funktion.
Es gilt $\perp \in S$. Die einzige endliche Teilfunktion von $\perp$ ist $\perp$ selbst – die ist in S. **Bis hierher kein Widerspruch.**
*Aber nun die **andere** Richtung: Sei g irgendeine Funktion, die $\perp$ als Teilfunktion enthält – also* jede *Funktion, denn $\perp \subseteq g$ immer.* **Nach dem Satz wäre dann jede Funktion in** *S***.** *Widerspruch, denn $S = \{\perp\}$.*
Anwendung 3 – Eine Eigenschaft, die aufzählbar *ist*.

$$S := \{g : g(5) \text{ ist definiert und } = 7\}.$$

Die endliche Teilfunktion $\{(5,7)\}$ liegt in S, und jede Funktion mit $g(5) = 7$ enthält sie.
Die Bedingung ist erfüllt. ✓

Und tatsächlich ist L_S aufzählbar: Man simuliert M auf 5 und wartet, ob 7 herauskommt. ✓

Eigenschaft	**aufzählbar**	**Grund**
$g(5) = 7$	ja	endliche Beobachtung
g ist irgendwo definiert	ja	ein Zeuge genügt
g ist total	nein	unendlich viele Bedingungen
g ist nirgends definiert	nein	Negativaussage
g hat unendlichen Definitionsbereich	nein	

> ***Das Muster ist durchgehend:*** *Aufzählbar sind die* existenziellen *Eigenschaften – solche, für die ein endlicher Beleg existiert.* **Nicht aufzählbar sind die *universellen*** – solche, die unendlich viele Bedingungen zugleich fordern.

*Kapitel 11 macht daraus eine präzise Klassifikation: die **arithmetische Hierarchie**.* **Die aufzählbaren Sprachen sind genau die Stufe Σ_1, ihre Komplemente Π_1** – *und darüber geht es unendlich weiter.* Post[20] begründete die Theorie der aufzählbaren Mengen; Rogers[22] und Odifreddi[18] sind die Standardwerke; Soare[26] bietet die modernste Darstellung; Cutland[4] ist die zugänglichste Einführung.

11 Grade der Unlösbarkeit

Jenseits der Grenze geht es weiter

Bisher gab es zwei Sorten von Problemen: entscheidbare und unentscheidbare. **Das ist eine grobe Einteilung** – *denn unter den unentscheidbaren gibt es solche, die* viel *schwerer sind als andere.*

Definition 11.1. A heißt **Turing-reduzierbar** auf B, geschrieben $A \leq_T B$, wenn es ein Entscheidungsverfahren für A gibt, das ein **Orakel** für B benutzen darf – also jederzeit fragen kann „ist $x \in B$?“ und eine korrekte Antwort erhält.

$A \leq_m B$	**Many-One**: eine einzige Übersetzung der Eingabe, eine einzige Frage, Antwort wird übernommen
$A \leq_T B$	**Turing**: beliebig viele Fragen, beliebige Verrechnung der Antworten

Die Turing-Reduktion ist echt allgemeiner. **Es gilt stets $A \leq_m B \Longrightarrow A \leq_T B$, aber nicht umgekehrt.**

Das Orakel ist ein Gedankending

Es gibt keine Orakel. *Die Konstruktion dient dazu,* ***relative*** *Schwierigkeit zu messen:* „Wenn ich B lösen könnte, könnte ich dann auch A lösen?“

> *Das ist dieselbe Denkfigur wie in der Mathematik, wenn man sagt: „Unter Annahme der Riemannschen Vermutung folgt…“* – **man rechnet mit etwas, das man nicht hat, um Beziehungen sichtbar zu machen.**

TURING *führte Orakelmaschinen 1939 in seiner Dissertation ein* – **in einer einzigen Fußnote, fast beiläufig.** *Er schrieb ausdrücklich, man werde nicht weiter darauf eingehen, was ein Orakel sei.*

Aus dieser Fußnote ist ein ganzes Teilgebiet geworden.

Turinggrade

Definition 11.2. A und B heißen **Turing-äquivalent**, wenn $A \leq_T B$ und $B \leq_T A$. Die Äquivalenzklassen heißen **Turinggrade**.

$\mathbf{0}$	Der Grad der **entscheidbaren** Mengen – der kleinste
$\mathbf{0}'$	Der Grad des **Halteproblems** („Null Strich“)
$\mathbf{0}''$	Halteproblem *relativ zum* Halteproblem
$\mathbf{0}^{(n)}$	n-fach iteriert

Definition 11.3. Der **Turingsprung** A' einer Menge A ist das Halteproblem für Maschinen mit Orakel A.

Satz 11.4. *Für jede Menge A gilt $A <_T A'$ – der Sprung ist **echt** schwerer.*

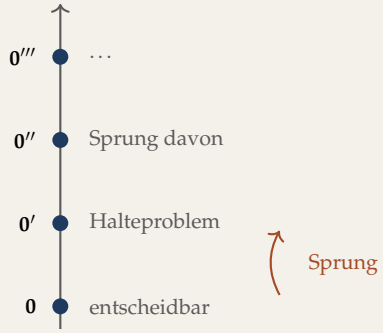
Der Beweis ist derselbe wie beim gewöhnlichen Halteproblem.

> *Man diagonalisiert über alle Maschinen **mit Orakel** A.* **Das Argument aus Kapitel 8 funktioniert unverändert** – *es benutzt nirgends, dass die Maschinen orakelfrei sind.*

Und $A \leq_T A'$ ist klar: Mit dem Orakel A' kann man A entscheiden, denn A' enthält mehr Information. ✓

$$\boxed{\mathbf{0} < \mathbf{0}' < \mathbf{0}'' < \mathbf{0}''' < \dots}$$

Es gibt also eine unendliche, echt aufsteigende Kette von Schwierigkeitsgraden. Jenseits des Halteproblems liegt ein Halteproblem, das noch schwerer ist – und so fort, ohne Ende.



11.1 Das Postsche Problem

Eine Frage, die dreizehn Jahre offen blieb

EMIL POST stellte 1944 in einem Vortrag vor der American Mathematical Society eine Frage, die einfach klingt:

> *„Gibt es eine **aufzählbare** Menge, die weder entscheidbar noch so schwer wie das Halteproblem ist?“*

Anders gesagt: Liegt zwischen 0 und $0'$ noch etwas?

*Alle damals bekannten unentscheidbaren aufzählbaren Mengen waren **vollständig** –* **also genau so schwer wie das Halteproblem.** *Post fragte, ob das notwendig so ist.*

> *Post war eine tragische Gestalt der Logik.* **Er hatte bereits um 1921 Ergebnisse gefunden, die Gödels Unvollständigkeitssatz vorwegnahmen, und sie nicht veröffentlicht** *– aus Zweifel, ob sie vollständig seien.* **Er litt an einer bipolaren Störung und starb 1954 mit 57 Jahren.**

Seine Frage blieb dreizehn Jahre offen. *1956/57 lösten sie **zwei Leute unabhängig voneinander**:* RICHARD FRIEDBERG, *ein 21-jähriger Student in Harvard, und* ALBERT MUCHNIK *in Moskau.*

> *Die Antwort lautet **ja** –* **es gibt Zwischengrade, und zwar unendlich viele.**
>
> *Wichtiger als die Antwort war die **Methode**: die **Prioritätsmethode**.* **Sie ist heute das Standardwerkzeug der Berechenbarkeitstheorie** *– und Friedberg hat sie als Student erfunden und danach das Fach verlassen.*
>
> *Er wurde Arzt und Psychiater.*

Wie die Prioritätsmethode arbeitet

> *Man will eine Menge A bauen, die **unendlich viele Bedingungen** gleichzeitig erfüllt –* **und die Bedingungen widersprechen einander teilweise.**

Die Bedingungen im Postschen Problem.

R_e	Die e-te Maschine entscheidet A **nicht** (sonst wäre A entscheidbar)
S_e	Die e-te Reduktion führt H **nicht** auf A zurück (sonst wäre A vollständig)

Das Verfahren.

1. Man ordnet die Bedingungen nach **Priorität**: $R_0, S_0, R_1, S_1, \ldots$
2. Man baut A schrittweise auf und lässt jede Bedingung eingreifen, wenn sie verletzt scheint.
3. Greift eine Bedingung ein, so kann sie Bedingungen **niedrigerer** Priorität zerstören – die werden **verletzt** und beginnen von vorn.
4. Eine Bedingung höherer Priorität greift nur **endlich oft** ein.

Der Beweis besteht darin, Punkt 4 zu zeigen *– durch Induktion über die Priorität.* **Wenn jede Bedingung nur endlich oft gestört wird, ist sie ab einem gewissen Zeitpunkt endgültig erfüllt.** ✓

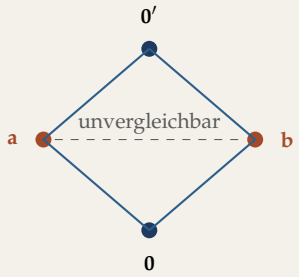
Das Bild: Ein Bauplan, bei dem jeder Handwerker die Arbeit der nachrangigen einreißen darf **– aber jeder nur endlich oft, sodass am Ende doch alles steht.**

Satz 11.5 (Friedberg und Muchnik, 1956/57)**.** *Es gibt aufzählbare Mengen A, B mit*

$$A \not\leq_T B \quad \textit{und} \quad B \not\leq_T A.$$

Insbesondere ist keine von beiden entscheidbar und keine Halteproblem-vollständig.

Die Turinggrade sind also keine Kette, sondern ein verzweigtes Gebilde *–* eine partielle Ordnung mit unvergleichbaren Elementen.



Man weiß heute weit mehr: Die Struktur der Turinggrade ist außerordentlich reichhaltig – **es gibt überabzählbar viele, jede abzählbare partielle Ordnung lässt sich in sie einbetten, und die Theorie erster Stufe der Turinggrade ist selbst unentscheidbar.**

11.2 Die arithmetische Hierarchie

Schwierigkeit nach Quantoren gemessen

Es gibt einen zweiten Weg, Unlösbarkeit zu messen: ***nach der Zahl der Quantorenwechsel*** *in der logischen Beschreibung des Problems.*

Definition 11.6. Eine Menge A gehört zu

Klasse	Form	Bedeutung
$\Sigma_0^0 = \Pi_0^0$	$R(x)$	entscheidbar
Σ_1^0	$\exists y\, R(x,y)$	**aufzählbar**
Π_1^0	$\forall y\, R(x,y)$	Komplement davon
Σ_2^0	$\exists y \forall z\, R$	
Π_2^0	$\forall y \exists z\, R$	

wobei R jeweils entscheidbar ist.

Warum Σ_1^0 genau die aufzählbaren Mengen sind.
„M hält bei x" bedeutet: ***Es gibt*** *eine Schrittzahl t, nach der M hält.*

$$x \in \mathrm{H} \iff \exists t : \text{„}M \text{ hält bei } x \text{ nach } t \text{ Schritten}\text{“}.$$

Die Aussage in Anführungszeichen ist ***entscheidbar*** **– man simuliert t Schritte und schaut nach. ✓**

Der Existenzquantor über die Zeit ist die Quelle der Unentscheidbarkeit.
Man kann nicht alle t durchprobieren, weil man nie weiß, wann man aufhören darf.

Problem	Klasse	Grad
$x \in \mathrm{H}$?	Σ_1^0	$\mathbf{0'}$
M hält *nie*?	Π_1^0	$\mathbf{0'}$
$L(M) = \emptyset$?	Π_1^0	$\mathbf{0'}$
$L(M)$ unendlich?	Π_2^0	$\mathbf{0''}$
M hält bei *allen* Eingaben?	Π_2^0	$\mathbf{0''}$
$L(M_1) = L(M_2)$?	Π_2^0	$\mathbf{0''}$

Satz 11.7 (Post, 1948).

$$A \in \Sigma^0_{n+1} \iff A \text{ ist aufzählbar relativ zum Orakel } \mathbf{0}^{(n)}.$$

Das verbindet die beiden Maße. Die arithmetische Hierarchie zählt Quantoren, die Turinggrade messen relative Berechenbarkeit – und Posts Satz sagt, dass beides dasselbe misst.

Die Hierarchie ist echt

Für jedes n gilt $\Sigma^0_n \neq \Pi^0_n$ und $\Sigma^0_n \subsetneq \Sigma^0_{n+1}$.
Der Beweis ist wieder Diagonalisierung – **man baut auf jeder Stufe ein Problem, das die vorige nicht erfasst.**

> *Bemerkenswert ist der Unterschied zur* ***Komplexitätstheorie****: Dort weiß man von der analogen Hierarchie* nicht*, ob sie echt ist* (die polynomielle Hierarchie; Band 3). **Schon** P **gegen** NP **ist offen.**

Die Berechenbarkeitstheorie ist in diesem Sinn die ***einfachere*** *Theorie: Ihre Hierarchien sind bewiesen echt.* **Der Grund ist, dass Diagonalisierung dort funktioniert und hier an der Relativierungsschranke scheitert** (Kapitel 15).

Aufgabe 11.1.

(a) Ordnen Sie konkrete Probleme in die arithmetische Hierarchie ein und begründen Sie jede Einordnung.
(b) Zeigen Sie, dass die Turing-Reduktion echt allgemeiner ist als die Many-One-Reduktion.
(c) Untersuchen Sie das Problem „hält bei allen Eingaben“ genau.

Lösung Schritt für Schritt.

(a) Einordnungen.
Problem 1 – $\mathrm{H} = \{\langle M\rangle\, x : M \text{ hält bei } x\}$.

$$\langle M\rangle x \in \mathrm{H} \iff \exists t : \ \mathtt{sim}(M, x, t) \text{ hält.}$$

$$\textit{Ein Existenzquantor über einer entscheidbaren Aussage} \Longrightarrow \Sigma^0_1. \ \checkmark$$

Problem 2 – $\overline{\mathrm{H}}$.

$$\langle M\rangle x \in \overline{\mathrm{H}} \iff \forall t : \ \mathtt{sim}(M, x, t) \text{ hält nicht.}$$

$$\textit{Ein Allquantor} \Longrightarrow \Pi^0_1. \ \checkmark$$

Problem 3 – $\texttt{LEER} = \{\langle M\rangle : L(M) = \emptyset\}$.

$$\langle M\rangle \in \texttt{LEER} \iff \forall x\, \forall t:\ M \text{ hält bei } x \text{ nicht nach } t \text{ Schritten.}$$

Zwei Allquantoren nacheinander lassen sich zu einem zusammenfassen – **man kodiert das Paar** (x, t) **mit der Cantorschen Paarungsfunktion** (Kapitel 1) **zu einer Zahl.** ✓

$$\Longrightarrow \texttt{LEER} \in \Pi_1^0.$$

***Gleichartige Quantoren verschmelzen; nur der* Wechsel *zählt.* Deshalb heißt die Stufe „Zahl der Quantorenwechsel", nicht „Zahl der Quantoren".**

Problem 4 – $\texttt{UNENDLICH} = \{\langle M\rangle : L(M) \text{ ist unendlich}\}$.
„Unendlich" heißt: Zu jeder Schranke gibt es ein längeres akzeptiertes Wort.

$$\langle M\rangle \in \texttt{UNENDLICH} \iff \forall n\, \exists x\, \exists t:\ |x| > n \ \wedge\ M \text{ akzeptiert } x \text{ in } t \text{ Schritten.}$$

$$\forall\exists \textit{ über entscheidbarem Kern} \Longrightarrow \Pi_2^0. \checkmark$$

Problem 5 – $\texttt{TOTAL} = \{\langle M\rangle : M \text{ hält bei } \textit{jeder} \text{ Eingabe}\}$.

$$\langle M\rangle \in \texttt{TOTAL} \iff \forall x\, \exists t:\ M \text{ hält bei } x \text{ nach } t \text{ Schritten.}$$

$$\Longrightarrow \texttt{TOTAL} \in \Pi_2^0.$$

Und das ist echt schwerer als das Halteproblem. TOTAL ist Π_2^0-vollständig, liegt also im Grad $\mathbf{0''}$ – **selbst mit einem Orakel für das Halteproblem bliebe es unlösbar.**

Die Zusammenfassung.

Problem	Klasse	aufzählbar?
H	Σ_1^0	ja
$\overline{\text{H}}$	Π_1^0	nein
LEER	Π_1^0	nein
UNENDLICH	Π_2^0	nein
TOTAL	Π_2^0	nein

(b) Turing gegen Many-One.
Schritt 1 – Die eine Richtung ist klar.
Gilt $A \leq_\text{m} B$ *über* f*, so entscheidet man* A*, indem man* $f(x)$ *berechnet und das Orakel einmal fragt.* ✓

$$\Longrightarrow A \leq_\text{m} B \Longrightarrow A \leq_\text{T} B.$$

Schritt 2 – Das trennende Beispiel.
Behauptung:

$$\overline{\mathrm{H}} \leq_{\mathrm{T}} \mathrm{H}, \qquad \text{aber} \qquad \overline{\mathrm{H}} \not\leq_{\mathrm{m}} \mathrm{H}.$$

Die Turing-Reduktion.

Mit einem Orakel für H *fragt man „$x \in \mathrm{H}$?“ und* ***dreht die Antwort um.*** ✓

Warum die Many-One-Reduktion unmöglich ist.
Angenommen $\overline{\mathrm{H}} \leq_{\mathrm{m}} \mathrm{H}$ *über* f*. Dann gilt für alle* x

$$x \in \overline{\mathrm{H}} \iff f(x) \in \mathrm{H}.$$

Da H *aufzählbar ist* (Kapitel 10) *und Aufzählbarkeit unter Many-One-Reduktion* ***abwärts*** *vererbt wird, wäre* $\overline{\mathrm{H}}$ *aufzählbar.*

Aber nach dem ***Satz von Post****: Sind* A *und* $\overline{A}$ *beide aufzählbar, so ist* A *entscheidbar.* **Dann wäre** H **entscheidbar** – *Widerspruch.*

Schritt 3 – Was das bedeutet.

Many-One	**unterscheidet** zwischen H und $\overline{\mathrm{H}}$ – feiner
Turing	**identifiziert** sie – gröber

Beide Begriffe haben ihren Platz. **Für Unentscheidbarkeitsbeweise nimmt man Many-One, weil sie mehr Information erhält** (etwa Aufzählbarkeit). *Für Schwierigkeitsgrade nimmt man Turing, weil sie robuster ist.*

(c) `TOTAL` genau betrachtet.
Schritt 1 – Nicht aufzählbar.
Wäre `TOTAL` *aufzählbar, so gäbe es eine Aufzählung* $M_1, M_2, \ldots$ *aller totalen Maschinen. Definiere*

$$g(n) := M_n(n) + 1.$$

g ist berechenbar (man sucht M_n in der Aufzählung und lässt sie laufen – sie hält, weil sie total ist) und total. **Also müsste** g **selbst als ein** M_k **vorkommen.**

$$g(k) = M_k(k) + 1 = g(k) + 1. \qquad \text{Widerspruch.}$$

Das ist ***Cantors Diagonalargument*** *in seiner ursprünglichen Form* – **man baut eine Funktion, die sich von jeder in der Liste an einer Stelle unterscheidet.**

Schritt 2 – Auch das Komplement ist nicht aufzählbar.

Wäre $\overline{\texttt{TOTAL}}$ *aufzählbar, so ließe sich das Halteproblem darauf zurückführen –* **und mit Schritt 1 wäre `TOTAL` dann entscheidbar.** *Widerspruch.*

`TOTAL` ist weder aufzählbar noch koaufzählbar.

Das ist eine schärfere Aussage als beim Halteproblem. H ist immerhin aufzählbar – man kann alle Ja-Fälle nach und nach finden. **Bei `TOTAL` geht nicht einmal das.**

Schritt 3 – Die praktische Bedeutung.

H	„Hält dieses Programm bei *dieser* Eingabe?“
`TOTAL`	„Terminiert dieses Programm *immer*?“

Die zweite Frage ist die, die man in der Praxis stellt. Niemand will wissen, ob ein Programm bei einer bestimmten Eingabe hält – man will wissen, ob es *überhaupt* terminiert.

Und diese Frage liegt eine ganze Stufe höher. **Ein Terminierungsprüfer arbeitet also an einem Π_2^0-vollständigen Problem** – *und dass moderne Werkzeuge trotzdem erstaunlich viele reale Programme entscheiden, liegt daran, dass reale Programme eine winzige, gutartige Teilmenge aller Programme sind* (Kapitel 15).

Der Ausweg ist stets derselbe: ***Man beweise Terminierung durch eine fallende Ordnung*** (Band 1, Kapitel 13). **Findet man eine, ist man fertig; findet man keine, weiß man nichts.**

Post[20] stellte die Frage; Soare[26] und Odifreddi[18] sind die Standardwerke der modernen Berechenbarkeitstheorie; Rogers[22] ist der Klassiker, an dem sich alle späteren orientieren.

Part IV

Jenseits der Informatik

12 Unentscheidbares in der Mathematik

Die Grenze verlässt die Informatik

Bisher waren alle unentscheidbaren Probleme Fragen ***über Programme.*** **Das könnte den Verdacht nähren, die Unentscheidbarkeit sei eine Eigentümlichkeit des Selbstbezugs** – *eine Kuriosität, die entsteht, wenn Maschinen über Maschinen reden.*

Dieses Kapitel widerlegt den Verdacht. **Es zeigt unentscheidbare Probleme, in deren Formulierung kein Programm vorkommt** – *über Wortersetzung, über Gleichungen in ganzen Zahlen, über das Auslegen der Ebene mit Kacheln.*

Postsches Korrespondenzproblem	Dominosteine passend legen
Wortproblem	Gleichheit in einer Gruppe entscheiden
Hilberts zehntes Problem	Ganzzahlige Lösungen von Polynomgleichungen
Kachelproblem	Die Ebene lückenlos auslegen

Der Beweis ist in allen vier Fällen derselbe: *Man zeigt, dass sich eine Turingmaschine in dem Problem* simulieren *lässt* – **und dann überträgt sich die Unentscheidbarkeit durch Reduktion.**

12.1 Das Postsche Korrespondenzproblem

Ein Dominospiel

Definition 12.1. Gegeben sind endlich viele **Dominosteine**

$$\left[\frac{u_1}{v_1}\right], \left[\frac{u_2}{v_2}\right], \ldots, \left[\frac{u_k}{v_k}\right]$$

mit Wörtern u_i, v_i über einem Alphabet. Gefragt: Gibt es eine nichtleere Folge von Steinen (Wiederholung erlaubt), sodass oben und unten dasselbe Wort steht?

Ein lösbares Beispiel.

$$\left[\frac{b}{ca}\right], \quad \left[\frac{a}{ab}\right], \quad \left[\frac{ca}{a}\right], \quad \left[\frac{abc}{c}\right].$$

Die Folge 2, 1, 3, 1, 4 *liefert:*

Stein	2	1	3	1	4
oben	a	b	ca	b	abc
unten	ab	ca	a	ca	c

Oben: $a\,b\,ca\,b\,abc = \texttt{abcababc}$. *Unten:* $ab\,ca\,a\,ca\,c = \texttt{abcaacac}$.

Das stimmt nicht – und das ist der Punkt

Die beiden Wörter sind verschieden. *Ich habe die Folge absichtlich falsch gewählt, um zu zeigen, wie schwer das Spiel ist.*

> *Man kann bei einem PKP-Problem nicht ansehen, ob es lösbar ist.* **Es gibt Instanzen mit vier Steinen, deren kürzeste Lösung über 200 Steine lang ist** – *und andere, die unlösbar sind, ohne dass man es je bemerkt.*

Ein berühmtes Beispiel von Zhao *(2002) mit drei Steinen hat als kürzeste Lösung eine Folge der Länge 252.*

Für ***zwei*** *Steine ist das Problem entscheidbar* (Ehrenfeucht, Karhumäki, Rozenberg 1982); **für fünf Steine ist es unentscheidbar** (Neary 2015). *Für drei und vier ist der Status offen.*

Satz 12.2 (Post, 1946)**.** *Das Postsche Korrespondenzproblem ist unentscheidbar.*

Die Beweisidee.

> *Man baut zu einer Turingmaschine M und einer Eingabe w eine PKP-Instanz, deren Lösungen genau die* ***haltenden Rechnungen*** *von M auf w sind.*

Der Kunstgriff: Die untere Zeile läuft der oberen stets ***eine Konfiguration voraus****.*

oben	$\#K_0\#K_1\#\ldots\#K_{n-1}$
unten	$\#K_0\#K_1\#\ldots\#K_{n-1}\#K_n$

Damit die beiden Zeilen ***gleich*** *werden, muss die Folge irgendwann enden – und das geht nur, wenn die Maschine hält.* **Die Steine sind so gebaut, dass jeder Anbau genau einem Rechenschritt entspricht.**

Für jede Übergangsregel $\delta(q, a) = (q', b, R)$ *gibt es einen Stein*

$$\left[\frac{qa}{bq'}\right],$$

und für unveränderte Bandzellen Steine $\left[\frac{c}{c}\right]$. **Ein Sonderstein am Anfang erzeugt den Vorsprung, Schlusssteine holen ihn wieder ein, wenn ein Haltezustand erreicht ist.** ✓

Die Reduktion ist $\mathrm{H} \leq_{\mathrm{m}}$ ***PKP*** – also überträgt sich die Unentscheidbarkeit.

Post veröffentlichte das Problem 1946 auf zweieinhalb Seiten. **Es ist seither das Standardwerkzeug für Unentscheidbarkeitsbeweise über formale Sprachen** – *weil es viel bequemer ist als eine Turingmaschine direkt.*

Was daraus über Grammatiken folgt

In Kapitel 4 standen vier unentscheidbare Fragen über kontextfreie Grammatiken. **Alle vier folgen aus dem PKP.**

Der Kunstgriff.
Zu einer PKP-Instanz mit Steinen $\left[\frac{u_i}{v_i}\right]$ *definiere man zwei kontextfreie Sprachen:*

$$L_o := \{u_{i_1} \dots u_{i_n} \# i_n \dots i_1\}, \qquad L_u := \{v_{i_1} \dots v_{i_n} \# i_n \dots i_1\}.$$

Beide sind kontextfrei – die umgekehrte Indexfolge hinter dem # lässt sich mit einem Stapel erzeugen. **Und** $L_o \cap L_u \neq \emptyset$ **gilt genau dann, wenn die PKP-Instanz lösbar ist.** ✓

$L_1 \cap L_2 = \emptyset$?	direkt aus obiger Konstruktion
$L = \Sigma^*$?	über die Komplemente
$L_1 = L_2$?	folgt daraus
G mehrdeutig?	Grammatik für $L_o \cup L_u$ ist genau dann mehrdeutig, wenn PKP lösbar

Die letzte Zeile ist besonders hübsch: Ein Wort im Durchschnitt hat zwei Ableitungen – eine über L_o, eine über L_u. **Mehrdeutigkeit und Lösbarkeit sind dasselbe.**

12.2 Hilberts zehntes Problem

Siebzig Jahre für eine Frage

Am 8. August 1900 stellte David Hilbert auf dem Internationalen Mathematikerkongress in Paris 23 Probleme vor. *Das zehnte lautete:*

> *„Eine diophantische Gleichung mit irgendwelchen Unbekannten und mit ganzen rationalen Zahlkoeffizienten sei vorgelegt: man soll ein Verfahren angeben, nach welchem sich mittelst einer endlichen Anzahl von Operationen entscheiden lässt, ob die Gleichung in ganzen rationalen Zahlen lösbar ist.“*

Man beachte die Formulierung: „man soll ein Verfahren angeben“. *Hilbert zweifelte nicht daran, dass es eines gibt* – **er hielt die Frage für eine Aufgabe, nicht für eine offene Frage.**

Das entsprach seiner Überzeugung, die er 1930 in Königsberg in den Satz fasste: **„Wir müssen wissen, wir werden wissen.“**

> *Die Ironie der Geschichte: Am Tag zuvor, auf derselben Tagung, hatte ein 24-jähriger Wiener namens* Kurt Gödel *in einer Diskussionsrunde erstmals seinen Unvollständigkeitssatz erwähnt.* **Hilbert war nicht anwesend.**

Die Arbeit am zehnten Problem begann in den 1950er Jahren. Martin Davis, Hilary Putnam *und* Julia Robinson *zeigten über zwanzig Jahre hinweg immer mehr:*

1949	Davis	„Davis-Normalform“
1952	Robinson	Reduktion auf eine Hypothese
1961	Davis, Putnam, Robinson	Exponentielle Gleichungen genügen
1970	**Matijassewitsch**	**Fibonaccizahlen sind diophantisch**

> *Es fehlte nur noch* ***ein*** *Baustein: ein diophantisches System, dessen Lösungen exponentiell wachsen.* **Julia Robinson hatte 1950 gezeigt, dass *ein einziges* solches Beispiel genügen würde** – *und zwanzig Jahre lang fand niemand eines.*

Im Januar 1970 fand es Juri Matijassewitsch, 22 Jahre alt, in Leningrad. *Sein Beispiel: die Fibonaccizahlen.* **Die Gleichung**

$$y^2 - xy - x^2 = \pm 1$$

hat als Lösungen genau die Paare aufeinanderfolgender Fibonaccizahlen – **und die wachsen exponentiell.** ✓

> *Julia Robinson erfuhr davon aus zweiter Hand und schrieb Matijassewitsch:*
> **„Ich bin besonders erfreut zu denken, dass ich, als ich meine Vermutung ursprünglich aufstellte, sagte, dass sie vielleicht von einem klugen jungen Menschen bewiesen würde, der noch nicht geboren war."**

Sie war 1970 fünfzig, Matijassewitsch 22. ***Als sie ihre Vermutung 1950 formulierte, war er zwei Jahre alt.*** **Die beiden arbeiteten danach jahrelang zusammen und wurden Freunde.**

Der Satz und seine Folgen

Satz 12.3 (Matijassewitsch, 1970 – „MRDP-Satz"). *Eine Menge $A \subseteq \mathbb{N}$ ist genau dann* ***aufzählbar****, wenn sie* ***diophantisch*** *ist – also von der Form*

$$A = \{a : \exists x_1, \ldots, x_n \in \mathbb{N} \;\; P(a, x_1, \ldots, x_n) = 0\}$$

für ein Polynom P mit ganzzahligen Koeffizienten.

> ***Das ist ein erstaunlicher Satz.*** Er sagt: **Polynomgleichungen über den ganzen Zahlen sind genau so ausdrucksstark wie Turingmaschinen.** *Rechnen und Lösen von Gleichungen sind dasselbe.*

Die unmittelbare Folgerung.
Da H *aufzählbar ist, gibt es ein Polynom P_{H}, sodass*

$$\langle M \rangle x \in \mathrm{H} \iff \exists \vec{y} : \; P_{\mathrm{H}}(\langle M \rangle x, \, \vec{y}) = 0.$$

Könnte man die Lösbarkeit diophantischer Gleichungen entscheiden, so könnte man das Halteproblem entscheiden. **Also ist Hilberts zehntes Problem unlösbar.**

Größe	Wert	Bemerkung
Unbekannte	**9** genügen	Matijassewitsch 1977
Grad	**4** genügt	
Grad 2	entscheidbar	Siegel 1972
eine Unbekannte	entscheidbar	triviale Schranken

> ***Neun Unbekannte und Grad vier genügen für Unentscheidbarkeit.*** Wo genau die Grenze liegt, ist offen – **schon für drei Unbekannte weiß man es nicht.**

Ein Polynom, dessen positive Werte die Primzahlen sind

Aus dem Satz folgt eine Kuriosität, die wie ein Zaubertrick aussieht.

Die Primzahlen bilden eine entscheidbare, also erst recht aufzählbare Menge. **Nach dem MRDP-Satz gibt es folglich ein Polynom, dessen *positive* Werte über den natürlichen Zahlen genau die Primzahlen sind.**

JONES, SATO, WADA *und* WIENS *gaben 1976 eines an:* **26 Variablen – eine für jeden Buchstaben des Alphabets –, Grad 25.**

Es ist praktisch nutzlos: Fast alle Werte sind negativ, und um eine Primzahl zu erzeugen, muss man die richtigen 26 Zahlen erraten.

Aber es beantwortet eine Frage, die seit Euler offen war – **„Gibt es eine Formel für die Primzahlen?“** *Die Antwort ist ja, und sie ist enttäuschend.*

Aufgabe 12.1.

(a) Führen Sie die PKP-Reduktion an einer kleinen Maschine konkret durch.
(b) Untersuchen Sie das Wortproblem für Halbgruppen und Gruppen.
(c) Analysieren Sie das Kachelproblem und den Zusammenhang zu aperiodischen Parkettierungen.

Lösung Schritt für Schritt.

(a) Die PKP-Reduktion konkret.
Wir nehmen eine winzige Maschine über $\{0,1\}$ *mit Bandalphabet* $\{0,1,\square\}$*:*

Regel	Bedeutung
$\delta(q_0,1) = (q_0,1,R)$	über Einsen laufen
$\delta(q_0,\square) = (q_h,\square,N)$	am Ende halten

Eingabe: $w = 1$.
Schritt 1 – Die Konfigurationsfolge.

K_0	$q_0 1$	Start
K_1	$1 q_0 \square$	nach rechts
K_2	$1 q_h \square$	hält

Schritt 2 – Die Steine.

Art	**Stein**	**Zweck**
Start	$\left[\frac{\#}{\#q_0 1 \#}\right]$	erzeugt den Vorsprung
Kopieren	$\left[\frac{1}{1}\right]$, $\left[\frac{\square}{\square}\right]$, $\left[\frac{\#}{\#}\right]$	unveränderte Zellen
Regel 1	$\left[\frac{q_0 1}{1 q_0}\right]$	Übergang
Regel 2	$\left[\frac{q_0 \square}{q_h \square}\right]$	Übergang
Abschluss	$\left[\frac{1 q_h}{q_h}\right]$, $\left[\frac{q_h \square}{q_h}\right]$	frisst die Konfiguration auf
Ende	$\left[\frac{q_h \# \#}{\#}\right]$	holt den Vorsprung ein

Schritt 3 – Die Lösung.

oben	**unten**
$\#$	$\# q_0 1 \#$
$q_0 1 \square \#$	$1 q_0 \square \#$
$1\, q_0 \square \#$	$1\, q_h \square \#$
$1 q_h \square \#$	$q_h \#$
$q_h \# \#$	$\#$

Die untere Zeile ist stets eine Konfiguration voraus. Erst wenn der Haltezustand erreicht ist, greifen die Abschlusssteine und verkürzen die untere Zeile, bis beide gleich lang sind. ✓

Schritt 4 – Warum die Reduktion korrekt ist.

M hält bei w	$\Longrightarrow$ die Konfigurationsfolge ist endlich $\Longrightarrow$ die Steine lassen sich abschließen
M hält nicht	$\Longrightarrow$ die untere Zeile bleibt **ewig** voraus $\Longrightarrow$ keine Lösung

*Eine technische Feinheit: Man verlangt üblicherweise, dass der Startstein **zuerst** kommen muss* (**modifiziertes PKP**) – **sonst könnte man mit einem Kopierstein beginnen und trivial lösen.** *Beide Fassungen sind aufeinander reduzierbar.*

(b) Das Wortproblem.

Definition 12.4. Gegeben eine endliche Präsentation $\langle S \mid R \rangle$ – Erzeuger und Relationen – und zwei Wörter u, v: Sind sie in der erzeugten Struktur gleich?

Fall 1 – Halbgruppen (Post und Markow, unabhängig 1947).
*Eine Halbgruppenpräsentation ist ein **Semi-Thue-System**: Regeln $\alpha \to \beta$, die man in beiden Richtungen anwenden darf.*

> ***Ein Semi-Thue-System kann eine Turingmaschine unmittelbar simulieren.*** Eine Konfiguration $uqav$ ist ein Wort; ein Rechenschritt ist eine Ersetzungsregel.

Damit ist H *direkt reduzierbar* – **das Wortproblem für Halbgruppen ist unentscheidbar.** ✓

Post und Markow fanden das 1947 unabhängig voneinander – **Post in New York, Markow in Moskau, mitten im beginnenden Kalten Krieg.**

Fall 2 – Gruppen (Novikow 1955, Boone 1958).

> *Bei Gruppen ist es **viel** schwerer, weil jedes Element ein Inverses hat.* **Man kann Schritte rückgängig machen** – *und damit lässt sich die Einbahnstraße einer Turingrechnung nicht mehr unmittelbar nachbilden.*

Der Beweis brauchte acht Jahre länger und ist ungleich schwieriger. **Pjotr Nowikow legte 1955 einen 143 Seiten langen Beweis vor.**

Struktur	Wortproblem	Jahr
endliche Gruppen	entscheidbar	trivial
abelsche Gruppen	entscheidbar	
freie Gruppen	entscheidbar	Kürzen
allgemeine Gruppen	**unentscheidbar**	**1955**
Halbgruppen	unentscheidbar	1947

> ***Das ist ein Ergebnis der reinen Algebra, dessen Beweis vollständig aus der Berechenbarkeitstheorie stammt.*** Es hat die kombinatorische Gruppentheorie umgestaltet: Seither weiß man, dass viele natürliche Fragen über Gruppen prinzipiell unbeantwortbar sind.

Etwa: „Ist diese Gruppe trivial?“ und „Sind diese beiden Präsentationen isomorph?“ sind ebenfalls unentscheidbar – **und daraus folgt, dass die Klassifikation von 4-Mannigfaltigkeiten in der Topologie unentscheidbar ist** (Markow 1958).

(c) Das Kachelproblem.

Definition 12.5. Ein **Wang-Plättchen** ist ein Einheitsquadrat mit gefärbten Kanten. Gegeben eine endliche Menge solcher Plättchen: Lässt sich die Ebene lückenlos parkettieren, sodass anliegende Kanten gleiche Farbe haben? (Drehen und Spiegeln ist **nicht** erlaubt, beliebige Wiederholung schon.)



Wangs Vermutung von 1961.

> HAO WANG *vermutete: „Wenn eine Plättchenmenge die Ebene parkettiert, dann auch* ***periodisch.“*** **Daraus hätte die Entscheidbarkeit gefolgt** – *man könnte einfach alle Periodenlängen durchprobieren und parallel nach einer Lücke suchen.*

Genauer: Man ließe zwei Verfahren parallel laufen:

Verfahren A	sucht eine periodische Parkettierung – findet sie, wenn es eine gibt
Verfahren B	sucht ein endliches Quadrat, das sich **nicht** auslegen lässt

Unter Wangs Vermutung hält eines von beiden immer an. **Also wäre das Problem entscheidbar.**

Berger widerlegte die Vermutung 1964.

ROBERT BERGER, *Wangs Doktorand, zeigte in seiner Dissertation:* ***Es gibt Plättchenmengen, die die Ebene parkettieren – aber* nur *aperiodisch.*** **Und daraus folgte die Unentscheidbarkeit.**

Seine erste aperiodische Menge hatte ***20 426*** *Plättchen.* **Er reduzierte sie später auf 104.**

Jahr	Plättchen	Urheber
1964	20 426	Berger
1966	104	Berger
1971	56	Robinson
1974	2 (Rauten)	Penrose
1996	13 (Wang)	Culik, Kari
2015	**11**	**Jeandel, Rao** – optimal

JEANDEL *und* RAO *zeigten 2015 durch erschöpfende Rechnersuche, dass 11 Plättchen und 4 Farben das Minimum sind* – **und dass jede Menge mit weniger entweder gar nicht oder periodisch parkettiert.**

Warum das unentscheidbar macht.
Bergers Beweis simuliert eine Turingmaschine: Die Plättchen sind so gebaut, dass eine Zeile der Parkettierung eine Konfiguration darstellt und die nächste Zeile die Nachfolgekonfiguration.

Die Parkettierung setzt sich genau dann unendlich fort, wenn die Maschine* nicht *hält. **Damit ist $\overline{\mathrm{H}}$ auf das Kachelproblem reduziert.** ✓

Der überraschende Nachhall.

Aus dieser rein logischen Konstruktion wurde ein Gebiet der Geometrie. **ROGER PENROSE fand 1974 eine aperiodische Parkettierung aus nur *zwei* Formen** – *und 1982 entdeckte* DAN SHECHTMAN ***Quasikristalle****, deren Atomanordnung genau solche aperiodische Ordnung zeigt.*

Shechtman wurde jahrelang angefeindet – LINUS PAULING *sagte, es gebe keine Quasikristalle, nur Quasiwissenschaftler.* **2011 erhielt Shechtman den Nobelpreis für Chemie.**

Im März 2023 fand eine Gruppe um DAVID SMITH *schließlich das „Einstein“-Plättchen:* ***eine*** **einzige** ***Form****, die die Ebene nur aperiodisch parkettiert* – **ein Problem, das seit 1961 offen war.**

Der Weg führte von Hilberts Entscheidungsproblem über Turing und Berger bis zu einem Fund, den ein pensionierter Drucktechniker in seiner Freizeit machte.

Post[20] führte das Korrespondenzproblem ein; Matijassewitsch[15] schildert die Geschichte des zehnten Problems selbst; Davis[5] und Sipser[24] bringen die Reduktionen vollständig; Papadimitriou[19] ordnet sie in die Komplexitätstheorie ein.

13 Gödels Unvollständigkeitssätze

Dieselbe Grenze, in der Logik

> GÖDEL *bewies seine Sätze 1931 – fünf Jahre bevor es den Begriff der Turingmaschine gab.* **Sein Beweis ist deshalb ungleich mühsamer als der, den wir heute führen können.**

Wer die Berechenbarkeitstheorie zur Verfügung hat, bekommt die Unvollständigkeit fast geschenkt. **Dieses Kapitel zeigt, wie.**

Definition 13.1. Ein formales System T (etwa die Peano-Arithmetik oder ZFC) heißt

widerspruchsfrei	Es beweist nicht zugleich φ und $\neg\varphi$
vollständig	Für jede Aussage φ beweist es φ oder $\neg\varphi$
effektiv axiomatisiert	Die Menge der Axiome ist **entscheidbar**

Die dritte Bedingung ist die, die den Bezug zur Informatik herstellt. Ein System, dessen Axiome man nicht erkennen kann, wäre unbrauchbar – man wüsste nie, ob ein vorgelegter Beweis gültig ist.

Der erste Unvollständigkeitssatz aus dem Halteproblem

Satz 13.2. *Sei T ein widerspruchsfreies, effektiv axiomatisiertes System, das stark genug ist, um Turingmaschinen zu beschreiben. Dann gibt es eine wahre Aussage, die T nicht beweist.*

Der Beweis in fünf Schritten.
Schritt 1 – Die Menge der Sätze ist aufzählbar.

> *Da die Axiome entscheidbar und die Schlussregeln endlich sind, kann man* ***alle*** *Beweise systematisch erzeugen:* **nach Länge geordnet, alle Zeichenfolgen durchgehen und jede auf Beweiseigenschaft prüfen.**

Das ist ein Aufzählungsverfahren (Kapitel 10). **Die Menge der beweisbaren Sätze ist aufzählbar.** ✓

Schritt 2 – Die Nichthaltefälle sind ausdrückbar.
Für jede Maschine M und Eingabe w lässt sich in der Sprache von T eine Aussage formulieren:

$$\varphi_{M,w} := \text{„}M \text{ hält bei } w \text{ nicht.“}$$

Das ist eine arithmetische Aussage – nach dem MRDP-Satz (Kapitel 12) *sogar eine über Polynomgleichungen.* ✓

Schritt 3 – Annahme: T wäre vollständig.
Dann beweist T für jedes Paar (M, w) entweder $\varphi_{M,w}$ oder $\neg\varphi_{M,w}$.
Schritt 4 – Daraus folgt ein Entscheidungsverfahren für H**.**

---

1. Zähle **parallel** alle Beweise von T auf.
2. Simuliere **gleichzeitig** M auf w.
3. Hält die Simulation, so antworte „hält“.
4. Erscheint ein Beweis von $\varphi_{M,w}$, so antworte „hält nicht“.

Unter der Annahme der Vollständigkeit ***tritt genau einer der beiden Fälle ein****, und zwar nach endlicher Zeit.* **Also wäre das Halteproblem entscheidbar.**

Schritt 5 – Der Widerspruch.
Das Halteproblem ist unentscheidbar (Kapitel 8). **Also war die Annahme falsch:**

T ist unvollständig.

Der ganze Beweis ist eine halbe Seite – und er benutzt nirgends einen selbstbezüglichen Satz. Gödel brauchte 1931 dreißig Seiten, weil er die Berechenbarkeit und die Kodierung erst selbst entwickeln musste.

Was der Beweis liefert – und was Gödels Beweis mehr liefert

Der obige Beweis zeigt: Es *gibt* eine unbeweisbare wahre Aussage. *Er nennt keine.*

> *Gödels Beweis ist* ***konstruktiv****: Er gibt zu jedem System T eine konkrete Aussage G_T an, die weder beweisbar noch widerlegbar ist* – **den berühmten Satz „Ich bin in T nicht beweisbar“.**

Man kann das aus dem obigen Argument nachliefern: *Man wähle eine Maschine M^*, die T nach einem Beweis von „M^* hält nicht“ durchsucht und anhält, sobald sie einen findet.*

Hält M^*?	Dann gibt es einen Beweis von „M^* hält nicht“ – T beweist Falsches
Hält M^* nicht?	Dann findet sie keinen Beweis – also ist „M^* hält nicht“ **wahr und unbeweisbar**

Bei widerspruchsfreiem T tritt der zweite Fall ein. **Und diese Maschine kann man tatsächlich hinschreiben** – *Yedidia und Aaronson taten es 2016 mit 748 Zuständen für ZFC* (Band 0, Kapitel 14).

Königsberg, September 1930

Vom 5. bis 7. September 1930 fand in Königsberg eine Tagung zur Erkenntnislehre der exakten Wissenschaften statt. *Es war die Zusammenkunft, auf der die drei großen Richtungen der Grundlagenforschung ihre Programme vortrugen:* Carnap *für den Logizismus,* Heyting *für den Intuitionismus,* von Neumann *für Hilberts Formalismus.*

> Kurt Gödel, *24 Jahre alt und gerade promoviert, hielt einen zwanzigminütigen Vortrag über seine Dissertation – den* ***Vollständigkeitssatz*** *der Prädikatenlogik.* **Ein positives Ergebnis, ganz im Sinne Hilberts.**

Am dritten Tag, in einer abschließenden Diskussionsrunde, sagte er beiläufig einen Satz, der alles änderte:

> *„Man kann sogar (unter Voraussetzung der Widerspruchsfreiheit der klassischen Mathematik) Beispiele für Sätze angeben, die zwar inhaltlich richtig, aber im formalen System der klassischen Mathematik unbeweisbar sind.“*

Im Protokoll steht keine Reaktion. **Offenbar verstand niemand die Tragweite** – *mit einer Ausnahme.*

> JOHN VON NEUMANN *suchte Gödel nach der Sitzung auf und ließ sich die Sache erklären.* **Er begriff sofort** – *und leitete binnen weniger Wochen selbständig den* ***zweiten*** *Unvollständigkeitssatz her: dass ein System seine eigene Widerspruchsfreiheit nicht beweisen kann.*

Als er Gödel davon schrieb, antwortete dieser, er habe das Ergebnis bereits und die Arbeit sei im Druck. *Von Neumann veröffentlichte nichts dazu und hielt fortan in seinen Vorlesungen Gödels Sätze* – **er soll gesagt haben, Gödel sei der größte Logiker seit Aristoteles.**

> *Am selben Tag, dem 8. September 1930, hielt* HILBERT *in Königsberg seine Radioansprache mit dem Satz* **„Wir müssen wissen, wir werden wissen“**, *der auf seinem Grabstein steht.*

Er war einen Tag zu spät widerlegt worden, im Nebensaal, von einem 24-Jährigen – **und erfuhr davon erst Monate später.**

Der zweite Unvollständigkeitssatz

Satz 13.3. *Ist T widerspruchsfrei und hinreichend stark, so beweist T nicht die Aussage „T ist widerspruchsfrei“.*

> ***Die Beweisidee:*** *Der erste Satz sagt „Wenn T widerspruchsfrei ist, dann ist G_T unbeweisbar“.* **Diese Implikation lässt sich *innerhalb* von T formalisieren.**

Also beweist T:

$$\mathrm{Con}(T) \to G_T.$$

Könnte T nun $\mathrm{Con}(T)$ *beweisen, so bewiese es auch G_T* – **im Widerspruch zum ersten Satz.**

> ***Das ist der Satz, der Hilberts Programm beendete.*** Hilbert wollte die Widerspruchsfreiheit der Mathematik mit *finiten* Mitteln beweisen – also mit Mitteln, die schwächer sind als das zu sichernde System. **Gödel zeigte, dass nicht einmal das System selbst genügt.**

Drei verbreitete Fehldeutungen

Erstens: „Es gibt Wahrheiten, die man nie beweisen kann."
Falsch in dieser Allgemeinheit. G_T **ist in einem *stärkeren* System sehr wohl beweisbar** – *etwa in* $T + \mathrm{Con}(T)$. **Der Satz sagt: Kein *einzelnes* System erfasst alles.**

> *Man kann beliebig weit hinaufsteigen:* T, $T + \mathrm{Con}(T)$, $T + \mathrm{Con}(T + \mathrm{Con}(T))$, … **– eine transfinite Folge, die TURING 1939 in seiner Dissertation untersuchte** (Ordinallogiken).

Zweitens: „Also kann der Mensch mehr als jede Maschine."
Das ist LUCAS' *und* PENROSES *Argument, und es ist umstritten.* **Der Einwand: Der Mensch *sieht* G_T nur dann als wahr, wenn er die Widerspruchsfreiheit von T voraussetzt** – *und die kann er ebenso wenig beweisen.*

Genau dasselbe kann eine Maschine: „Wenn T widerspruchsfrei ist, dann gilt G_T*" ist in T beweisbar.* **Der behauptete Vorsprung verschwindet bei genauem Hinsehen.**

Drittens: Anwendungen außerhalb der Mathematik.

> *Der Satz handelt von formalen Systemen, die die Arithmetik enthalten.* **Er sagt nichts über Rechtssysteme, Gesellschaften, Sprachen oder das Bewusstsein**, *so oft er dafür in Anspruch genommen wird.*

GÖDEL *selbst hat sich an solchen Verallgemeinerungen nie beteiligt.* **Die Logikerin TORKEL FRANZÉN hat ihnen 2005 ein ganzes Buch gewidmet** – *mit dem Untertitel „An Incomplete Guide to Its Use and Abuse".*

Aufgabe 13.1.

(a) Führen Sie die Gödelisierung durch und zeigen Sie, wie Selbstbezug entsteht.
(b) Untersuchen Sie den Satz von Tarski über die Undefinierbarkeit der Wahrheit.
(c) Betrachten Sie konkrete unabhängige Aussagen der Mathematik.

Lösung Schritt für Schritt.

(a) Gödelisierung und Selbstbezug.
Schritt 1 – Formeln als Zahlen.
Man ordnet jedem Zeichen eine Zahl zu und kodiert eine Zeichenfolge über die Primfaktorzerlegung:

$$\langle s_1 s_2 \dots s_n \rangle := 2^{c(s_1)} \cdot 3^{c(s_2)} \cdot 5^{c(s_3)} \cdots p_n^{c(s_n)}.$$

Nach dem Fundamentalsatz der Arithmetik ist die Zerlegung eindeutig (Mathematik-Band 1) – **also lässt sich die Formel aus der Zahl zurückgewinnen.** ✓

Heute würde man einfach die Bitfolge des Textes als Binärzahl lesen (Kapitel 6). **Gödel brauchte die Primzahlkodierung, weil er zeigen musste, dass die Ent- und Verschlüsselung *arithmetisch ausdrückbar* ist.**

Schritt 2 – Beweisbarkeit wird arithmetisch.
Gödel zeigte: Es gibt eine arithmetische Formel $\mathrm{Bew}(x, y)$ *mit der Bedeutung*

„y ist die Nummer eines Beweises für die Formel mit Nummer x."

***Das war der mühsame Teil seiner Arbeit** – 46 Definitionen, die schrittweise zeigen, dass jeder syntaktische Begriff („ist eine Variable", „ist eine Formel", „folgt durch Modus ponens") sich durch Addition, Multiplikation und Quantoren ausdrücken lässt.*

Heute sagt man dazu: ***Die Beweisrelation ist entscheidbar, also arithmetisch*** *– und beruft sich auf die Church-Turing-These* (Kapitel 7).

Schritt 3 – Der Diagonalsatz.

Satz 13.4 (Diagonallemma). *Zu jeder Formel $\psi(x)$ mit einer freien Variablen gibt es einen Satz σ mit*

$$T \vdash \sigma \leftrightarrow \psi(\langle\sigma\rangle).$$

In Worten: ***Zu jeder Eigenschaft gibt es einen Satz, der von sich selbst behauptet, diese Eigenschaft zu haben.***

Die Konstruktion.
Sei $\mathrm{sub}(x, y)$ *die arithmetische Funktion, die in der Formel mit Nummer x die freie Variable durch die Zahl y ersetzt. Setze*

$$\theta(x) := \psi\big(\mathrm{sub}(x, x)\big), \qquad \sigma := \theta(\langle\theta\rangle).$$

Dann gilt

$$\sigma = \psi\big(\mathrm{sub}(\langle\theta\rangle, \langle\theta\rangle)\big) = \psi(\langle\theta(\langle\theta\rangle)\rangle) = \psi(\langle\sigma\rangle). \qquad \checkmark$$

***Das ist exakt die Konstruktion des Quine** (Band 0, Kapitel 14):* Der Ausdruck $\mathrm{sub}(x, x)$ ist „setze den Text seinem eigenen Zitat nach" – **und $\theta(\langle\theta\rangle)$ ist die Anwendung auf sich selbst.**

Schritt 4 – Der Gödelsatz.
Man wende das Lemma an auf

$$\psi(x) := \neg\exists y\, \mathrm{Bew}(x, y) \qquad (\text{„}x \text{ ist unbeweisbar"}).$$

Man erhält G mit

$$T \vdash G \leftrightarrow \text{„}G \text{ ist unbeweisbar"}.$$

Schritt 5 – Die Fallunterscheidung.

Wäre G beweisbar	so wäre G wahr, also unbeweisbar – **Widerspruch**
Wäre $\neg G$ beweisbar	so wäre G beweisbar – **Widerspruch**

$$\Longrightarrow G \text{ ist unabhängig von } T. \qquad \checkmark$$

*Und G ist **wahr*** – denn G behauptet die eigene Unbeweisbarkeit, und die haben wir gerade gezeigt. **Eine wahre, unbeweisbare Aussage.**

(b) Der Satz von Tarski.

Satz 13.5 (Tarski, 1933/36). *Es gibt keine arithmetische Formel* $\mathrm{Wahr}(x)$*, sodass für alle Sätze φ gilt*

$$\mathrm{Wahr}(\langle\varphi\rangle) \leftrightarrow \varphi.$$

Der Beweis ist zwei Zeilen lang.
Angenommen, Wahr *existierte. Nach dem Diagonallemma gibt es einen Satz λ mit*

$$\lambda \leftrightarrow \neg\mathrm{Wahr}(\langle\lambda\rangle).$$

Also

$$\lambda \leftrightarrow \neg\lambda. \qquad \text{Widerspruch.}$$

Das ist das Lügnerparadoxon – als Satz. λ sagt: „Ich bin falsch." Die Auflösung liegt nicht darin, den Satz zu verbieten, sondern darin, dass das Prädikat „wahr" in der Sprache selbst nicht ausdrückbar ist.

Der Vergleich mit Gödel.

	Beweisbarkeit	**Wahrheit**
ausdrückbar?	**ja**	**nein**
Satz	Gödel	Tarski
Folge	Unvollständigkeit	Undefinierbarkeit

Der Unterschied ist der Grund für Gödels Erfolg. Beweisbarkeit ist eine **aufzählbare** Eigenschaft – man kann alle Beweise durchgehen. **Wahrheit ist es nicht**, *und deshalb lässt sie sich nicht formalisieren.*

*In der Sprache von Kapitel 11: Beweisbarkeit liegt in Σ_1^0; arithmetische Wahrheit liegt in **keiner** Stufe der arithmetischen Hierarchie.*

(c) Konkrete unabhängige Aussagen.

Gödels G ist künstlich – ein Satz, der nur konstruiert wurde, um unabhängig zu sein. **Jahrzehntelang fragte man, ob es *natürliche* mathematische Aussagen gibt, die unabhängig sind.**

Beispiel 1 – Der Satz von Goodstein (1944 / 1982).
*Man schreibt eine Zahl in **vollständiger Basisdarstellung**, etwa*

$$266 = 2^{2^3} + 2^3 + 2 = 2^{2^{2+1}} + 2^{2+1} + 2.$$

Dann ersetzt man überall die Basis 2 durch 3 und zieht 1 ab. Und so fort mit 4, 5, 6, …

Schritt	Basis	Wert
0	2	266
1	3	$\approx 4{,}4 \cdot 10^{38}$
2	4	$\approx 3{,}2 \cdot 10^{616}$
3	5	astronomisch

Satz 13.6 (Goodstein, 1944)**.** *Jede Goodstein-Folge erreicht nach endlich vielen Schritten den Wert **null**.*

Das ist völlig unglaubwürdig – und wahr. Der Beweis ordnet jeder Zahl eine Ordinalzahl unterhalb ε_0 zu; diese fällt echt bei jedem Schritt, und eine fallende Folge von Ordinalzahlen ist endlich.

KIRBY *und* PARIS *zeigten 1982:* ***Dieser Satz ist in der Peano-Arithmetik unbeweisbar** – denn sein Beweis braucht transfinite Induktion bis ε_0, und die geht über PA hinaus.*

Es ist eine natürliche, elementar formulierbare Aussage über natürliche Zahlen – **und sie ist unabhängig.**

Beispiel 2 – Die Kontinuumshypothese.

1940	**Gödel**	CH ist mit ZFC *verträglich*
1963	**Cohen**	¬CH ist ebenfalls verträglich

Zusammen: CH ist von ZFC unabhängig (Mathematik-Band 10, Kapitel 10). **Cohen erhielt dafür 1966 die Fields-Medaille** – *die bislang einzige für ein Ergebnis der Logik.*

Beispiel 3 – Der Satz von Paris und Harrington (1977).
Eine Verschärfung des endlichen Ramsey-Satzes (Mathematik-Band 15, Kapitel 8), *die sich von der gewöhnlichen Fassung nur durch eine Zusatzbedingung unterscheidet* – **und die in PA unbeweisbar ist.**

> ***Es war das erste Beispiel einer* natürlichen *kombinatorischen Aussage, die von PA unabhängig ist*** – und es kam 46 Jahre nach Gödel.

Die Bilanz.

Aussage	**unabhängig von**	**Jahr**
Gödelsatz G	PA, ZFC	1931
Con(PA)	PA	1931
Kontinuumshypothese	ZFC	1940/63
Goodstein	PA	1982
Paris-Harrington	PA	1977
Hydra-Spiel	PA	1982

> ***Die praktische Lehre ist beruhigender, als der Satz klingt.*** Für die Mathematik, die tatsächlich betrieben wird, spielt die Unvollständigkeit fast keine Rolle – **man musste über vierzig Jahre suchen, um natürliche Beispiele zu finden.**

Der Satz beschreibt eine Grenze, die weit außerhalb des Gebiets liegt, in dem man sich gewöhnlich bewegt – **ganz wie das Halteproblem, das keinen Programmierer daran hindert, Terminierung für seine konkreten Schleifen zu beweisen.** Gödels Originalarbeit[8] ist überraschend lesbar; Nagel und Newman[17] erklären sie allgemeinverständlich; Smullyan[25] und Boolos, Burgess und Jeffrey[1] führen sie streng aus; Hofstadter[9] macht den Selbstbezug zum Thema eines ganzen Buches.

14 Kolmogorow-Komplexität

Wie viel Information steckt in einem Objekt?

Shannons Informationsbegriff (Band 0, Kapitel 1) *misst die Entropie einer* ***Quelle*** *– er braucht eine Wahrscheinlichkeitsverteilung.* **Was aber ist der Informationsgehalt *eines einzelnen* Objekts?**

Man betrachte zwei Bitfolgen der Länge 64:

w_1	`0101010101010101010101010101010101010101010101010101010101010101`
w_2	`1100100100001111110110101010001000011010001100001000110100`

Beide haben dieselbe Länge, und beide haben nach Shannon dieselbe Entropie, wenn man sie aus einer Gleichverteilung gezogen denkt. **Und doch ist die erste offenkundig „einfacher".**

Man kann w_1 in wenigen Worten beschreiben: „32-mal 01". **Für w_2 fällt einem nichts Kürzeres ein als die Folge selbst.**

Definition 14.1. Die **Kolmogorow-Komplexität** $K(x)$ eines Wortes x ist die Länge des **kürzesten Programms**, das x ausgibt und anhält:

$$K(x) := \min\{|p| : U(p) = x\},$$

wobei U eine feste universelle Maschine ist.

Unabhängig entdeckt von Ray Solomonoff *(1960/64),* Andrei Kolmogorow *(1965) und* Gregory Chaitin *(1966)* – **drei Personen, drei Motive.**

Solomonoff	suchte eine Theorie der **induktiven Inferenz**
Kolmogorow	suchte eine Grundlegung der **Wahrscheinlichkeit** ohne Wahrscheinlichkeit
Chaitin	war **18 Jahre alt** und suchte den Kern der Unvollständigkeit

Die Wahl der Maschine spielt fast keine Rolle

Die Definition hängt von U ab – ein Programm in C ist anders lang als eines in Python. *Der folgende Satz sagt, dass das nur eine Konstante ausmacht.*

Satz 14.2 (Invarianzsatz). *Für je zwei universelle Maschinen U, V gibt es eine Konstante c mit*

$$|K_U(x) - K_V(x)| \leq c \qquad \text{für alle } x.$$

Beweis: U kann V simulieren. **Sei s das Programm, das V auf U nachbildet – dann ist jedes V-Programm p als U-Programm $s\,p$ verwendbar, also**

$$K_U(x) \leq K_V(x) + |s|\,. \qquad \checkmark$$

Symmetrisch in der anderen Richtung, mit $c := \max(|s|, |s'|)$.

> ***Die Konstante ist die Größe eines Interpreters*** – typischerweise einige tausend Bit. **Für kurze Wörter ist das viel, für lange bedeutungslos.**

Deshalb sind Aussagen der Form „$K(x) = 17$“ sinnlos, Aussagen der Form „$K(x_n) \geq n - \mathcal{O}(1)$“ dagegen sinnvoll.

Fast alles ist zufällig

Definition 14.3. Ein Wort x heißt **zufällig** (im Sinne von Kolmogorow), wenn

$$K(x) \geq |x|\,.$$

> *In Worten:* ***Es gibt keine kürzere Beschreibung als das Wort selbst.*** Das Wort ist *inkompressibel*.

Satz 14.4. *Für jedes n gibt es mindestens ein zufälliges Wort der Länge n. Mehr noch: Höchstens 2^{n-c+1} der 2^n Wörter haben $K(x) < n - c$.*

Der Beweis ist ein Abzählargument.
Es gibt

$$2^0 + 2^1 + \cdots + 2^{n-c-1} = 2^{n-c} - 1$$

Programme der Länge $< n - c$. **Also können höchstens $2^{n-c} - 1$ Wörter eine so kurze Beschreibung haben.**

c	**Anteil komprimierbar**	**Bemerkung**
1	$< 50\,\%$	
10	$< 0{,}1\,\%$	
20	$< 10^{-6}$	
100	$< 10^{-30}$	praktisch nie

Kein Kompressionsverfahren kann alle Dateien verkleinern. Verkleinert es manche, so vergrößert es notwendig andere – **denn es gibt zu wenige kurze Wörter.**

Das ist ein hübsches Argument gegen die regelmäßig auftauchenden Behauptungen, jemand habe einen Kompressor gefunden, der jede *Datei verkleinert.* **Er müsste eine injektive Abbildung einer Menge in eine echt kleinere leisten.**

K ist nicht berechenbar

Satz 14.5. *Die Funktion K ist nicht berechenbar.*

Der Beweis – das Berry-Paradoxon als Argument.
Angenommen, K wäre berechenbar. Dann betrachte das folgende Programm P_n mit Parameter n:

```
durchlaufe alle Wörter x in
längenlex. Ordnung:
  if K(x) >= n then gib x aus und
halte
```

Das Programm findet stets etwas – nach dem vorigen Satz gibt es Wörter beliebig hoher Komplexität. **Es gibt also das erste Wort mit $K(x) \geq n$ aus.**

Die Länge von P_n ist

$$|P_n| = \mathcal{O}(1) + \log_2 n$$

– ein fester Programmtext plus die Zahl n in Binärdarstellung.
Aber P_n erzeugt ein Wort x mit $K(x) \geq n$, also

$$n \leq K(x) \leq |P_n| = \mathcal{O}(1) + \log_2 n.$$

Für hinreichend großes n ist das falsch, denn $\log_2 n$ wächst langsamer als n.
Widerspruch.

> ***Das ist das Berry-Paradoxon** (Kapitel 1) **in exakter Form:** „Die erste Zahl, die sich nicht mit weniger als n Bit beschreiben lässt“ – **eine Beschreibung mit etwa $\log_2 n$ Bit.***
>
> *Der Unterschied zum Paradoxon: Hier wird nichts Widersprüchliches behauptet.* **Es folgt bloß, dass die vorausgesetzte Berechenbarkeit von K falsch war.**

Chaitins Zahl

Gregory Chaitin war 18, als er 1965 seine erste Arbeit über algorithmische Komplexität schrieb – *als Student am City College of New York, ohne zu wissen, dass Kolmogorow in Moskau dasselbe dachte.*

> *Sein bekanntestes Ergebnis kam 1975: die **Haltewahrscheinlichkeit** Ω.*

Definition 14.6.

$$\Omega := \sum_{p \text{ hält}} 2^{-|p|},$$

summiert über alle haltenden Programme einer präfixfreien universellen Maschine.

> *Anschaulich: **Man wirft für jedes Bit eine Münze und lässt die entstehende Bitfolge als Programm laufen.** Ω ist die Wahrscheinlichkeit, dass es anhält.*

Die Präfixfreiheit (Band 1, Kapitel 13) *sorgt dafür, dass die Summe konvergiert – nach der Kraftschen Ungleichung ist* $\Omega \leq 1$.
Die Eigenschaften von Ω sind bemerkenswert.

wohldefiniert	Eine feste reelle Zahl zwischen 0 und 1
unberechenbar	Keine Maschine gibt ihre Ziffern aus
zufällig	Die ersten n Bits haben $K \geq n - \mathcal{O}(1)$
allwissend	Die ersten n Bits entscheiden das Halteproblem für alle Programme der Länge $\leq n$

Zur letzten Zeile. *Kennt man Ω bis auf n Stellen genau, so lässt man alle Programme parallel laufen und summiert die gefundenen Halteereignisse. Sobald die Summe bis auf 2^{-n} an Ω heranreicht, weiß man:* ***Alle noch fehlenden haltenden Programme sind länger als** n.*

Also hält kein Programm der Länge $\leq n$ mehr an, das bis dahin nicht gehalten hat. **Das Halteproblem für kurze Programme ist entschieden. ✓**

> ***Ω ist damit die kompakteste denkbare Kodierung mathematischen Wissens.*** Die ersten 10 000 Bit entschieden praktisch jede offene Vermutung, die sich in einem kurzen Programm ausdrücken lässt – Goldbach, Riemann, Collatz.

Und man kennt sie nicht. CALUDE, DINNEEN *und* SHU *berechneten 2002 die ersten* ***64 Bit*** *für eine bestimmte Maschine* – **unter enormem Aufwand und nur, weil sie eine sehr einfache Maschine wählten.**

Der Wert beginnt mit

```
0000001000000100000110001000011
0001111111001011101110100001000
```

– die ersten 64 Bit, so weit man sie kennt.

Weiter kommt man nicht: Ab einer gewissen Stelle wären die Bits von Ω genau die Aussagen, die das jeweilige Beweissystem nicht mehr entscheiden kann.

Chaitins Unvollständigkeitssatz

Satz 14.7 (Chaitin, 1974)**.** *Zu jedem widerspruchsfreien, effektiv axiomatisierten System T gibt es eine Konstante c_T, sodass T für* ***kein*** *Wort x die Aussage*

$$K(x) > c_T$$

beweist.

> ***Das ist eine drastische Aussage.*** Fast alle Wörter sind zufällig – ihre Komplexität ist hoch. **Aber ein festes formales System kann das nur für endlich viele Wörter *beweisen*.**

Der Beweis.
Angenommen, T bewiese für beliebig große c Aussagen der Form $K(x) > c$. Dann betrachte:

```
durchlaufe alle Beweise von T der
Reihe nach:
  if der Beweis zeigt "K(x) > n" für
ein x
    then gib x aus und halte
```

Dieses Programm hat die Länge $\mathcal{O}(1) + \log_2 n$ und gibt ein x mit $K(x) > n$ aus. **Also**

$$n < K(x) \leq \mathcal{O}(1) + \log_2 n,$$

was für große n falsch ist.

> ***Chaitins Deutung:*** *Ein formales System mit c Bit an Axiomen kann keine Aussage beweisen, die mehr als etwa c Bit Information enthält.* **„Man bekommt aus einem Beweis nicht mehr heraus, als man an Axiomen hineingesteckt hat."**

Diese Deutung ist umstritten

Chaitin hat daraus weitreichende Folgerungen gezogen – *etwa, die Mathematik solle „quasi-empirisch" werden und neue Axiome aufgrund ihrer Nützlichkeit annehmen.*

> *Der Einwand von Logikern wie* Torkel Franzén *und* Panu Raatikainen*: Die Konstante c_T misst* ***nicht*** *den Informationsgehalt der Axiome.* **Sie hängt von der gewählten universellen Maschine ab und lässt sich durch andere Kodierung beliebig verschieben.**

Ein System mit ***einem*** *zusätzlichen Axiom kann eine viel größere Konstante haben* – **das Axiom „ZFC ist widerspruchsfrei" ist kurz und verschiebt c_T erheblich.**

Der Satz selbst ist unbestritten. **Umstritten ist, was er über die Natur der Mathematik aussagt** – *eine Debatte, die an die um Gödels Sätze erinnert* (Kapitel 13).

Aufgabe 14.1.

(a) Bestimmen Sie die Komplexität konkreter Wörter und begründen Sie die Schranken.

(b) Zeigen Sie, dass es keinen universellen Kompressor geben kann.

(c) Untersuchen Sie den Zusammenhang zwischen Kolmogorow-Komplexität und Shannon-Entropie.

Lösung Schritt für Schritt.

(a) Komplexität konkreter Wörter.
Wort 1 – $x = 0^n$ (n Nullen).
Ein Programm: „gib n Nullen aus". Die Länge ist ein fester Text plus die Zahl n:

$$K(0^n) \leq \log_2 n + \mathcal{O}(1).$$

Für $n = 10^{100}$ sind das etwa 333 Bit – **gegenüber 10^{100} Bit für die Folge selbst.** ✓

Wort 2 – *die ersten n Stellen von π.*

$$K(\pi_{1\ldots n}) \leq \log_2 n + \mathcal{O}(1).$$

> ***Die Ziffern von π sehen zufällig aus und sind es im Sinne Kolmogorows* nicht.** Es gibt ein kurzes Programm, das beliebig viele davon erzeugt – etwa die BBP-Formel oder eine Reihenentwicklung.

Das ist ein wichtiger Punkt: ***Statistische Zufälligkeit und algorithmische Zufälligkeit sind verschieden.*** **π besteht jeden statistischen Test und hat trotzdem logarithmische Komplexität.**

Wort 3 – *eine echt zufällig gewürfelte Folge der Länge n.*

$$K(x) \geq n - \mathcal{O}(1) \qquad \text{mit Wahrscheinlichkeit } \geq 1 - 2^{-\mathcal{O}(1)}.$$

Wort 4 – die ersten n Bits von Ω.

$$K(\Omega_{1\ldots n}) \geq n - \mathcal{O}(1).$$

> *Ω ist* ***beweisbar*** *zufällig – eine der wenigen konkret definierten Zahlen mit dieser Eigenschaft.* **Und trotzdem ist sie vollständig festgelegt, sobald man die Maschine wählt.**

Ein merkwürdiges Objekt: definiert, eindeutig, unberechenbar und zufällig – alles zugleich.

Wort 5 – die Busy-Beaver-Werte (Band 0, Kapitel 14).

$$K(\Sigma(n)) \leq \log_2 n + \mathcal{O}(1),$$

denn $\Sigma(n)$ ist durch n eindeutig bestimmt.

> ***Eine Zahl mit über $10^{10^{15}}$ Stellen hat eine Beschreibung von wenigen Dutzend Bit*** – sie ist maximal komprimierbar. **Aber niemand kann sie ausrechnen.**

Niedrige Komplexität und Berechenbarkeit sind also ***nicht*** *dasselbe. K* **misst die Länge der Beschreibung, nicht die Kosten des Ausrechnens.**

(b) Es gibt keinen universellen Kompressor.

Satz 14.8. *Sei C eine injektive Abbildung von Bitfolgen auf Bitfolgen (ein verlustfreier Kompressor). Dann gibt es ein x mit $|C(x)| \geq |x|$.*

Der Beweis – Schubfachprinzip.
Angenommen, C verkürzte ***jede*** *Folge der Länge n. Dann bildet C die 2^n Wörter der Länge n injektiv in die Menge der Wörter der Länge $< n$ ab.*
Diese hat aber nur

$$2^0 + 2^1 + \cdots + 2^{n-1} = 2^n - 1$$

Elemente. **Eine injektive Abbildung von 2^n Elementen in $2^n - 1$ gibt es nicht.**

Der Beweis braucht nicht einmal die Kolmogorow-Komplexität – er ist reines Abzählen. **Aber die Komplexität erklärt, *warum* Kompression überhaupt funktioniert:** *Die Dateien, die man tatsächlich komprimiert, haben niedrige Komplexität.*

Die praktische Fassung.

Dateityp	**Kompression**	**Grund**
Text	$\approx 4\times$	niedrige Komplexität
Programmcode	$\approx 4\times$	viel Struktur
Bitmap-Bild	$\approx 3\times$	Nachbarn ähnlich
Bereits komprimiert	$\approx 1\times$	oft *größer*
Zufallsdaten	$1\times$	stets größer

*Die vorletzte Zeile ist die Alltagserfahrung: Ein ZIP-Archiv noch einmal zu packen macht es **größer** –* **um die Größe des zusätzlichen Kopfs.**

(c) Kolmogorow gegen Shannon.

	Shannon H	**Kolmogorow** K
Gegenstand	eine **Quelle**	ein **Objekt**
braucht	Wahrscheinlichkeiten	nichts
berechenbar	ja	**nein**
Jahr	1948	1964/65

Der Zusammenhang.

Satz 14.9. *Sei X eine Quelle mit Entropie H, die Wörter der Länge n erzeugt. Dann gilt*

$$\frac{1}{n}\,\mathbb{E}\big[K(x)\big] \longrightarrow H \qquad (n \to \infty).$$

Im Mittel stimmen die beiden Maße überein. Die algorithmische Komplexität einer zufällig gezogenen Folge ist im Erwartungswert gerade die Entropie der Quelle.

Die Beweisidee.
Obere Schranke: *Ein Huffman-Code für die Quelle* (Band 1, Kapitel 13) *liefert ein Programm der Länge $\approx nH$: „dekodiere die folgenden Bits mit diesem Code“.* ✓
Untere Schranke: *Nach dem Abzählargument können nicht viele Wörter deutlich kürzer beschrieben werden.* ✓
Wo sie auseinanderfallen.

Objekt	H	K
$\pi_{1\ldots n}$	$\approx n$ (sieht zufällig aus)	$\mathcal{O}(\log n)$
echter Zufall	$\approx n$	$\approx n$
0^n	≈ 0	$\mathcal{O}(\log n)$

> ***Die erste Zeile ist der Kern des Unterschieds.*** Shannon sieht nur die Statistik und hält π für zufällig. **Kolmogorow sieht die Beschreibbarkeit und erkennt die Struktur.**

Die praktische Folgerung.

> *Ein Kompressor, der π auf logarithmische Länge bringt, müsste π* erkennen – **also gewissermaßen Mathematik betreiben.** *Kein praktischer Kompressor tut das.*

Genau hier liegt der Zusammenhang zu den lernenden Systemen (Band 13): **Ein Modell, das den nächsten Wert gut vorhersagt, *ist* ein Kompressor** – *und je besser es die Struktur der Daten erfasst, desto näher kommt es der Kolmogorow-Komplexität.*

> *Solomonoffs ursprüngliche Motivation war genau das:* ***eine Theorie des Lernens.*** **Seine universelle Induktion sagt vorher, indem sie über alle Programme summiert, gewichtet mit $2^{-|p|}$** – *eine formale Fassung von Ockhams Rasiermesser.*

Sie ist beweisbar optimal und unberechenbar – **das übliche Muster dieses Bandes.**

Was daraus für den Zufallsbegriff folgt

Kolmogorows eigentliches Ziel war eine Definition von „Zufallsfolge“, die ohne Wahrscheinlichkeitstheorie auskommt.

> *Die klassische Wahrscheinlichkeitsrechnung kann nicht sagen, ob* eine bestimmte *Folge zufällig ist* – **jede Folge der Länge n hat dieselbe Wahrscheinlichkeit 2^{-n}.**

Kolmogorow drehte die Frage um: ***Zufällig ist, was sich nicht kürzer beschreiben lässt.*** **Per Martin-Löf zeigte 1966, dass dieser Begriff genau mit dem übereinstimmt, den man über statistische Tests erhält** – *eine bemerkenswerte Bestätigung.*

> *Damit hat der Zufall eine Definition, die nichts über Wahrscheinlichkeiten voraussetzt* – **sondern nur über Berechenbarkeit.**

Li und Vitányi[14] sind das Standardwerk und erschöpfend; Chaitin[2] schreibt über sein eigenes Gebiet mit Verve und Eigenwilligkeit; Cutland[4] und Enderton[7] ordnen es in die Berechenbarkeitstheorie ein.

Part V

Aktueller Stand

15 Wo die Theorie heute steht

Was fest steht

Abgeschlossen seit	
1931	Unvollständigkeit
1936	Halteproblem, Universalität
1953	Satz von Rice
1957	Postsches Problem (Prioritätsmethode)
1970	Hilberts zehntes Problem
Offen	
P gegen NP	seit 1971 (Band 3)
PKP mit 3 oder 4 Steinen	seit 1946
Struktur der Turinggrade	viele Teilfragen offen
Collatz-Vermutung	seit 1937

Die Berechenbarkeitstheorie ist in ihren Grundzügen ein abgeschlossenes Gebiet. Anders als die Komplexitätstheorie, deren Zentralfrage seit über fünfzig Jahren offen ist, sind hier die großen Sätze bewiesen.

Was bleibt, sind Feinstruktur und Anwendungen – **und die haben sich in den letzten zwei Jahrzehnten erheblich verschoben.**

15.1 Warum die Komplexitätstheorie stecken bleibt

Die Relativierungsschranke

Die Berechenbarkeitstheorie hat ein Werkzeug, das durchweg funktioniert: die ***Diagonalisierung.*** **Warum löst sie nicht auch** P **gegen** NP**?**

Satz 15.1 (Baker, Gill, Solovay, 1975)**.** *Es gibt Orakel A und B mit*

$$\mathrm{P}^A = \mathrm{NP}^A \qquad \textit{und} \qquad \mathrm{P}^B \neq \mathrm{NP}^B.$$

Das ist ein Unmöglichkeitsergebnis über Beweismethoden. Diagonalisierungsbeweise **relativieren** – sie gelten unverändert, wenn man allen beteiligten Maschinen dasselbe Orakel gibt.

Ein relativierender Beweis für $\mathrm{P} \neq \mathrm{NP}$ *müsste also auch* $\mathrm{P}^A \neq \mathrm{NP}^A$ *zeigen* **– und das ist für das obige** A **falsch.** ✓

Umgekehrt kann kein relativierender Beweis $\mathrm{P} = \mathrm{NP}$ *zeigen, wegen* B*.*

Damit ist die einzige Technik, die in diesem Band durchweg funktioniert hat, für die Zentralfrage der Komplexitätstheorie ausgeschlossen. Das erklärt fünfzig Jahre Stillstand.

Zwei weitere Schranken kamen hinzu:

Schranke	**Jahr**	**schließt aus**
Relativierung	1975	Diagonalisierung
Natürliche Beweise	1994	kombinatorische Argumente
Algebrisierung	2008	algebraische Erweiterungen

RAZBOROV *und* RUDICH *zeigten 1994: Ein Beweis, der „natürlich" ist – also eine Eigenschaft benutzt, die sich effizient prüfen lässt und auf fast alle Funktionen zutrifft –, würde die Existenz kryptographisch sicherer Pseudozufallsgeneratoren widerlegen.*

Da man an solche Generatoren glaubt, glaubt man nicht an solche Beweise. **Band 3 führt das aus.**

15.2 Was sich seit 2000 getan hat

Vier Entwicklungen

Erstens: Terminierungsprüfer, die erstaunlich weit kommen.

> *Das Halteproblem ist unentscheidbar – und trotzdem beweisen moderne Werkzeuge die Terminierung eines großen Teils realer Programme automatisch.*

Terminator	Microsoft Research, ab 2006; prüfte Gerätetreiber
AProVE	Aachen; für Java, C, Haskell, Prolog
Ultimate	Freiburg; Ranking-Funktionen automatisch

Der Kunstgriff: Man sucht automatisch nach ***Rangfunktionen*** (Band 1, Kapitel 13) – **Größen, die bei jedem Durchlauf echt fallen und nach unten beschränkt sind.**

> ***Findet das Werkzeug eine, ist die Terminierung bewiesen.*** *Findet es keine, weiß man nichts* – **genau der Ausweg 1 aus dem Vorwort.**

Zweitens: Formale Verifikation im Einsatz.

System	was bewiesen	Jahr
seL4	Betriebssystemkern	2009
CompCert	C-Übersetzer	2008
Vierfarbensatz	Beweis in Coq	2005
Kepler-Vermutung	Beweis in HOL/Isabelle	2014
$\Sigma(5) = 4098$	Beweis in Coq	**2024**

Die letzten drei Zeilen sind bemerkenswert: ***Mathematische Sätze, deren Beweise zu groß für menschliche Prüfung sind*** – **und die deshalb maschinell verifiziert wurden.**

Drittens: Die Busy-Beaver-Grenze rückt.

n	$\Sigma(n)$	**Status**
4	13	1983
5	**4098**	**Juli 2024**
6	$> 10^{10^{15}}$	offen
748	—	in ZFC nicht bestimmbar

Der Beweis für $\Sigma(5)$ entstand in einem offenen Gemeinschaftsprojekt („bbchallenge") aus Freiwilligen – **88 Millionen Maschinen mussten analysiert werden, und das Ergebnis ist in Coq formal verifiziert.**

Für $n = 6$ ist bekannt, dass eine Maschine mindestens $10^{10^{15}}$ Einsen schreibt – **und man vermutet, dass der genaue Wert außerhalb der Reichweite jeder Analyse liegt. Viertens: Ein Ergebnis aus der Quantentheorie.**

2020 zeigten Ji, Natarajan, Vidick, Wright *und* Yuen*:* $\mathrm{MIP}^* = \mathrm{RE}$ – **die Klasse der Probleme, die sich mit verschränkten Beweisern interaktiv verifizieren lassen, ist genau die Klasse der *aufzählbaren* Probleme.**

Insbesondere lässt sich das Halteproblem so „verifizieren". **Als Nebenprodukt widerlegte die Arbeit die Connes-Einbettungsvermutung** – *eine Frage aus der Operatoralgebra, die seit 1976 offen war.*

Ein Ergebnis der Berechenbarkeitstheorie entschied eine Frage der Funktionalanalysis – **über den Umweg der Quanteninformation.**

Stand August 2026 – eine Momentaufnahme

Frage	Status	Bemerkung
Halteproblem	1936 entschieden	unlösbar, endgültig
Satz von Rice	1953	
Hilbert 10	1970	9 Unbekannte genügen
P gegen NP	**offen**	drei Beweisschranken bekannt
PKP, 3 Steine	offen	seit 1946
$\Sigma(5)$	2024: 4098	in Coq verifiziert
$\Sigma(6)$	offen	
Collatz	offen	bis 2^{68} geprüft
$\mathrm{MIP}^* = \mathrm{RE}$	2020	Connes widerlegt

Drei Beobachtungen.
Erstens: Das Gebiet ist alt und tragfähig. *Alles Wesentliche dieses Bandes stammt aus den Jahren 1931 bis 1957.* **Was seither kam, ist Verfeinerung** – *und Anwendung auf Gebiete, die es damals nicht gab.*
Zweitens: Die Unentscheidbarkeit ist praktisch handhabbar. *Terminierungsprüfer, Typsysteme, statische Analysen und Modellprüfer arbeiten alle an unentscheidbaren Problemen* – **und sind trotzdem nützlich, weil sie die vier Auswege konsequent nutzen.**
Drittens: Die Grenze wandert nicht, aber die Werkzeuge werden besser. *Was 1970 als prinzipiell unzugänglich galt, ist heute in vielen Einzelfällen entschieden* – **durch Rechenkraft, formale Verifikation und Gemeinschaftsprojekte wie bbchallenge.**

Schlussgedanke: drei Einsichten

Erstens: Die Grenze folgt aus der Universalität.

> *Eine Maschine ist genau dann universell, wenn Programme für sie Daten sind.* **Und genau dann kann sie auf sich selbst angewandt werden** – *woraus die Diagonalisierung folgt und mit ihr das Halteproblem.*

Universalität	Eine Maschine führt jedes Programm aus
Unentscheidbarkeit	Keine Maschine entscheidet alles über Programme
beides folgt aus	**Programm = Datum**

Wer das eine will, bekommt das andere. **Die totalen Sprachen** (Coq, Agda) **zeigen die Gegenprobe:** *Sie garantieren Terminierung und können deshalb keinen Interpreter für sich selbst enthalten.*
Zweitens: Unentscheidbarkeit ist kein Hindernis, sondern eine Konstruktionsvorgabe.

Jedes Werkzeug, das über Programme urteilt, muss sich für einen der vier Auswege entscheiden. **Welchen es wählt, bestimmt seinen Charakter:**

Werkzeug	Ausweg	Preis
Typprüfer	konservativ	lehnt Korrektes ab
Virenscanner	Fehler zulassen	Fehlalarme
Testverfahren	Zeitschranke	findet nicht alles
Coq, Agda	Teilklasse	keine Universalität

Drittens: Die Grenze ist strukturiert.

„Unentscheidbar" ist keine einzelne Kategorie, sondern der Anfang einer unendlichen Hierarchie (Kapitel 11). **Über dem Halteproblem liegt `TOTAL`, darüber weitere Stufen, und dazwischen liegen die Friedberg-Muchnik-Grade.**

*Und diese Hierarchie ist **bewiesen** echt* – **im Unterschied zur polynomiellen Hierarchie der Komplexitätstheorie, von der man es nur vermutet.**

Das ist die eigentümliche Lage dieses Fachs: ***Über das, was* gar nicht *geht, weiß man sehr viel. Über das, was* schnell *geht, fast nichts.***

Band 3 handelt von dieser zweiten Frage – **und der Begleitband „Der seltsame Kreis" von dem, was beiden zugrunde liegt.**

Ich glaube nicht, dass es eine wesentliche Grenze gibt.

Es ist nur so, dass wir sie nicht überschreiten können,
solange wir mit einem festen System arbeiten.

— KURT GÖDEL (1906–1978)
in einem Gespräch über seine Sätze.
Er hielt sie nie für eine Aussage über die Grenzen
des Denkens – nur über die Grenzen
jedes einzelnen Formalismus.

Part VI

Anhang

A Übersicht und Formelsammlung

A.1 Die große Tabelle

Sprachklassen

Klasse	Maschine	Chomsky	Gedächtnis
endlich	Tabelle	—	keines
regulär	endlicher Automat	Typ 3	keines
determ. kontextfrei	determ. Keller-automat	—	Stapel
kontextfrei	Kellerautomat	Typ 2	Stapel
kontextsensitiv	linear beschränkt	Typ 1	lineares Band
entscheidbar	**TM, hält stets**	—	unbegrenzt
aufzählbar	**Turingmaschine**	Typ 0	unbegrenzt
alle Sprachen	keine	—	—

*Alle Inklusionen sind **echt**.* **Die letzte Zeile ist überabzählbar, alle anderen sind abzählbar** (Kapitel 1).

Abschluss- und Entscheidungseigenschaften

	regulär	**kontextfr.**	**entsch.**	**aufzählb.**
Vereinigung	ja	ja	ja	ja
Schnitt	ja	nein	ja	ja
Komplement	ja	nein	**ja**	**nein**
Verkettung	ja	ja	ja	ja
Stern	ja	ja	ja	ja
$w \in L$?	ja	ja	ja	nein
$L = \emptyset$?	ja	ja	nein	nein
$L_1 = L_2$?	ja	nein	nein	nein
$L = \Sigma^*$?	ja	nein	nein	nein

Die fett gesetzte Zeile ist die wichtigste des Bandes. Entscheidbare Sprachen sind unter Komplement abgeschlossen, aufzählbare nicht – **und genau darin liegt der Unterschied** (Satz von Post, Kapitel 10).

Unentscheidbare Probleme

Problem	Klasse	Grad	Jahr
H (Halteproblem)	Σ_1^0	$\mathbf{0'}$	1936
$\overline{\mathrm{H}}$	Π_1^0	$\mathbf{0'}$	1936
`LEER`: $L(M) = \emptyset$?	Π_1^0	$\mathbf{0'}$	
`TOTAL`: hält immer?	Π_2^0	$\mathbf{0''}$	
$L(M_1) = L(M_2)$?	Π_2^0	$\mathbf{0''}$	
jede nichttriviale Verhaltenseigenschaft	—	—	**1953**
Postsches Korrespondenzproblem	—	$\mathbf{0'}$	1946
Wortproblem, Halbgruppen	—	$\mathbf{0'}$	1947
Wortproblem, Gruppen	—	$\mathbf{0'}$	1955
Hilberts 10. Problem	Σ_1^0	$\mathbf{0'}$	**1970**
Kachelproblem	Π_1^0	$\mathbf{0'}$	1964
Mehrdeutigkeit einer Grammatik	—	$\mathbf{0'}$	

A.2 Die Beweistechniken

Vier Werkzeuge, mehr braucht man nicht

Diagonalisierung	Baue ein Objekt, das sich von jedem in einer Liste unterscheidet. *Cantor, Halteproblem, Hierarchiesätze*
Reduktion	Zeige $A \leq_{\mathrm{m}} B$; ist A unentscheidbar, so auch B. *Rice, PKP, Hilbert 10*
Simulation	Bilde ein Modell im anderen nach. *Potenzmengenkonstruktion, Church-Turing-These*
Schubfachprinzip	Endlich viele Zustände, unendlich viele Fälle. *Pumping-Lemmata*

Das Schema einer Unentscheidbarkeitsreduktion.

1. Wähle ein bekannt unentscheidbares A (meist H).
2. Gib eine **berechenbare** Funktion f an.
3. Zeige: $x \in A \iff f(x) \in B$ – **beide Richtungen**.
4. Schließe: Wäre B entscheidbar, so auch A. Widerspruch.

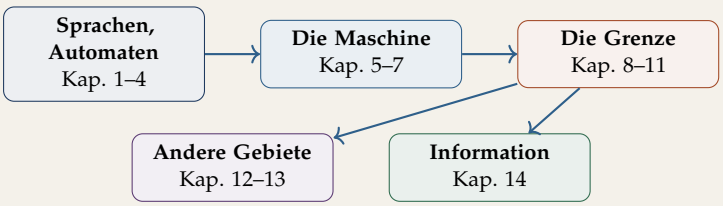
Der häufigste Fehler ist die Richtung. Man reduziert das **bekannte** Problem auf das **neue**, nicht umgekehrt – $\mathrm{H} \leq_{\mathrm{m}}$ *B zeigt, dass B mindestens so schwer ist wie* H.

Die zentralen Sätze

Kleene (Normalform)	Jede berechenbare Funktion ist $U(\mu y\, T(e,x,y))$ – **ein einziger μ-Operator genügt**
Kleene (Rekursion)	Jedes Programm darf seinen eigenen Text benutzen
Turing	H ist unentscheidbar
Rice	Jede nichttriviale Verhaltenseigenschaft ist unentscheidbar
Post	A und $\overline{A}$ aufzählbar $\Longrightarrow$ A entscheidbar
Post (Hierarchie)	Σ^0_{n+1} = aufzählbar relativ zu $\mathbf{0}^{(n)}$
Friedberg-Muchnik	Es gibt Grade echt zwischen $\mathbf{0}$ und $\mathbf{0}'$
Gödel I	Kein widerspruchsfreies effektives System ist vollständig
Gödel II	Kein solches System beweist die eigene Widerspruchsfreiheit
Tarski	Wahrheit ist nicht arithmetisch definierbar
MRDP	Aufzählbar = diophantisch
Chaitin	T beweist $K(x) > c_T$ für kein x

A.3 Die Landkarte des Bandes

Aufbau und Querverbindungen



Informatik-Band 0	Halteproblem, Quine, Bootstrapping, Busy Beaver
Informatik-Band 1	Aufwandsanalyse, dynamische Programmierung (CYK)
Informatik-Band 3	Komplexität, P gegen NP, Beweisschranken
Informatik-Band 6	Compilerbau, LL- und LR-Zerteilung
Informatik-Band 7	Shannon-Entropie, Kompression
Begleitband	Selbstbezug, Gödel, Bewusstsein
Mathematik-Band 1	Primfaktorzerlegung (Gödelisierung), diophantische Gleichungen
Mathematik-Band 10	Abzählbarkeit, Auswahlaxiom, Kontinuumshypothese
Mathematik-Band 15	Schubfachprinzip, Ramsey-Theorie

A.4 Glossar

Die Begriffe dieses Bandes

Alphabet	endliche Zeichenmenge Σ
arithmetische Hierarchie	Einteilung nach Quantorenwechseln
aufzählbar	es gibt ein Verfahren, das genau die Elemente ausgibt
Church-Turing-These	„berechenbar“ = „turingberechenbar“
Diagonalisierung	Konstruktion eines Objekts außerhalb jeder Liste
diophantisch	durch Polynomgleichung darstellbar
entscheidbar	es gibt ein stets haltendes Verfahren
Gödelisierung	Kodierung von Formeln als Zahlen
Halteproblem	„Hält M bei x?“ – unentscheidbar
Kolmogorow-Kompl.	Länge des kürzesten erzeugenden Programms
Orakel	hypothetische Antwortinstanz für ein Problem
Prioritätsmethode	Beweistechnik für Zwischengrade
Pumping-Lemma	notwendige Bedingung für Regularität bzw. Kontextfreiheit
Reduktion	Zurückführung eines Problems auf ein anderes
Rekursionstheorem	jedes Programm kennt seinen eigenen Text
Turinggrad	Klasse gleich schwerer Probleme
Turingsprung	Halteproblem relativ zu einem Orakel
universelle Maschine	führt jedes Programm aus
unvollständig	es gibt unentscheidbare Sätze im System
Wang-Plättchen	gefärbtes Quadrat; Kachelproblem

B Kommentierte Literatur

Achtundzwanzig Titel

Die Auswahl umfasst Lehrbücher, Standardwerke und *sieben Originalarbeiten* – *darunter die drei, mit denen 1931 und 1936 alles begann.* Die Reihenfolge ist alphabetisch.

[1] Boolos, G. S.; Burgess, J. P.; Jeffrey, R. C.: *Computability and Logic.* 5. Aufl., Cambridge 2007.

Verbindet Berechenbarkeit und Logik in einem Band – von Turingmaschinen über die Unvollständigkeitssätze bis zur Modelltheorie. Sorgfältig und philosophisch aufmerksam; das Standardwerk für den Übergang zwischen beiden Gebieten.

[2] Chaitin, G. J.: *Meta Math! The Quest for Omega.* Pantheon 2005.

Chaitin über sein eigenes Gebiet – enthusiastisch, eigenwillig und voller Seitenhiebe. Man sollte die weitreichenden Deutungen mit Vorsicht lesen (siehe Kapitel 14), aber die Darstellung von Ω ist unerreicht anschaulich.

[3] Church, A.: *An Unsolvable Problem of Elementary Number Theory.* American Journal of Mathematics 58, 1936.

Die Arbeit, die Turing um wenige Wochen zuvorkam. Sie löst das Entscheidungsproblem über den Lambda-Kalkül und enthält die erste Fassung der These, die heute Churchs und Turings Namen trägt.

[4] Cutland, N.: *Computability: An Introduction to Recursive Function Theory.* Cambridge 1980.

Der klassische Zugang über rekursive Funktionen statt über Maschinen. Schmal, streng und ungewöhnlich klar – besonders gut beim Normalformsatz und beim Rekursionstheorem.

[5] Davis, M.: *Computability and Unsolvability.* McGraw-Hill 1958; Dover 1982.

Das erste Lehrbuch des Gebiets überhaupt. Davis war Posts Student und schrieb es mit 30 Jahren; es prägte die Begriffsbildung einer ganzen Generation und ist als Dover-Ausgabe bis heute lieferbar.

[6] Davis, M. (Hrsg.): *The Undecidable.* Raven Press 1965; Dover 2004.

Die Originalarbeiten von Gödel, Church, Turing, Rosser, Kleene und Post in einem Band, mit Einleitungen von Davis. Wer die Quellen lesen will, braucht nur dieses Buch.

[7] Enderton, H. B.: *Computability Theory: An Introduction to Recursion Theory.* Academic Press 2011.

Endertons letztes Buch, knapp und außerordentlich sauber geschrieben. Reicht bis zu den Turinggraden und der arithmetischen Hierarchie, ohne den Leser zu überfahren.

[8] Gödel, K.: *Über formal unentscheidbare Sätze der Principia Mathematica und verwandter Systeme I.* Monatshefte für Mathematik und Physik 38, 1931.

Die berühmteste Arbeit der Logik. Überraschend lesbar, wenn man die ersten Seiten der informellen Beweisskizze folgt – Gödel erklärt dort seinen eigenen Beweis besser als die meisten Nacherzählungen.

[9] Hofstadter, D. R.: *Gödel, Escher, Bach.* Klett-Cotta 1985 (engl. 1979).

Das Buch über den Selbstbezug. Es enthält eine der besten allgemeinverständlichen Darstellungen des Unvollständigkeitssatzes – und verknüpft sie mit Fragen des Bewusstseins, die der Begleitband dieser Reihe aufnimmt.

[10] Hopcroft, J. E.; Motwani, R.; Ullman, J. D.: *Introduction to Automata Theory, Languages, and Computation.* 3. Aufl., Pearson 2006.

Die Referenz für formale Sprachen. Der „Hopcroft-Ullman" prägt das Gebiet seit 1969; die Kapitel über Automaten und kontextfreie Sprachen sind vollständiger als bei jedem Konkurrenten.

[11] Hromkovič, J.: *Theoretische Informatik.* 5. Aufl., Springer Vieweg 2014.

Deutschsprachig, mit besonderem Augenmerk auf die *Beweistechniken* – Hromkovič erklärt nicht nur, was gilt, sondern wie man auf die Beweise kommt. Für Selbststudium gut geeignet.

[12] Kleene, S. C.: *Introduction to Metamathematics.* North-Holland 1952.

Das Werk, das die Begriffe des Gebiets festlegte – Kleene-Stern, Normalformsatz, Rekursionstheorem, arithmetische Hierarchie. Nach über siebzig Jahren immer noch die Quelle, auf die sich alle beziehen.

[13] Kozen, D. C.: *Automata and Computability.* Springer 1997.

In 41 kurze Vorlesungen gegliedert, jede in einer Sitzung zu bewältigen. Kozens Zugang ist algebraischer als der übliche und dadurch oft eleganter.

[14] Li, M.; Vitányi, P.: *An Introduction to Kolmogorov Complexity and Its Applications.* 4. Aufl., Springer 2019.

Das Standardwerk, erschöpfend und mit vielen Anwendungen – von der Beweistechnik der Inkompressibilität bis zur universellen Induktion. Für Kapitel 14 die Hauptquelle.

[15] Matiyasevich, Y.: *Hilbert's Tenth Problem.* MIT Press 1993.

Der Löser des Problems erzählt selbst – mit vollständigem Beweis und einem persönlichen Nachwort über die Zusammenarbeit mit Julia Robinson. Mathematisch anspruchsvoll, historisch ergreifend.

[16] Minsky, M. L.: *Computation: Finite and Infinite Machines.* Prentice-Hall 1967.

Ein früher Klassiker, der Automaten, Turingmaschinen und Berechenbarkeit mit erstaunlicher Anschaulichkeit verbindet. Enthält Minskys Beweis, dass zwei Zähler eine Turingmaschine simulieren.

[17] Nagel, E.; Newman, J. R.: *Gödel's Proof.* New York University Press 1958; rev. 2001.

Hundert Seiten, die den Unvollständigkeitssatz ohne Formeln erklären – und dabei nichts unterschlagen. Das Buch, das Hofstadter als Jugendlichen prägte.

[18] Odifreddi, P.: *Classical Recursion Theory.* 2 Bände, North-Holland 1989/1999.

Die enzyklopädische Darstellung – über 1200 Seiten, mit ausführlichen historischen Anmerkungen. Zum Nachschlagen, wenn Soare [26] nicht ausreicht.

[19] Papadimitriou, C. H.: *Computational Complexity.* Addison-Wesley 1994.

Beginnt mit der Berechenbarkeit und geht dann zur Komplexität über. Die Brücke zwischen diesem Band und Band 3; besonders gut bei den Reduktionen.

[20] Post, E. L.: *Recursively Enumerable Sets of Positive Integers and Their Decision Problems.* Bulletin of the AMS 50, 1944.

Der Vortrag, in dem Post sein Problem stellte. Bemerkenswert ist der Ton: Post schreibt bewusst informell und wirbt darum, das Gebiet aus der reinen Formalisierung herauszuholen.

[21] Rice, H. G.: *Classes of Recursively Enumerable Sets and Their Decision Problems.* Transactions of the AMS 74, 1953.

Sieben Seiten für einen Satz, der alle Einzelbeweise überflüssig macht. Rice promovierte damit und verließ das Gebiet danach weitgehend.

[22] Rogers, H.: *Theory of Recursive Functions and Effective Computability.* McGraw-Hill 1967; MIT Press 1987.

Der Klassiker, an dem sich alle späteren Darstellungen orientieren. Rogers' Notation und Begriffsbildung sind bis heute Standard.

[23] Schöning, U.: *Theoretische Informatik – kurz gefasst.* 5. Aufl., Spektrum 2008.

Zweihundert Seiten für den gesamten Stoff – Automaten, Berechenbarkeit, Komplexität. Für die Prüfungsvorbereitung unschlagbar knapp; zum Verstehen zu knapp.

[24] Sipser, M.: *Introduction to the Theory of Computation.* 3. Aufl., Cengage 2012.

Das beste Lehrbuch des Gebiets. Sipser schreibt ungewöhnlich klar, führt jeden Beweis erst als Idee und dann formal, und wählt die Beispiele mit großer Sorgfalt. Wenn Sie *ein* Buch zu diesem Band kaufen: dieses.

[25] Smullyan, R. M.: *Gödel's Incompleteness Theorems.* Oxford 1992.

Smullyan war Logiker und Rätselautor zugleich. Sein Zugang über selbstbezügliche Systeme ist der eleganteste, den ich kenne – und deutlich kürzer als der übliche.

[26] Soare, R. I.: *Turing Computability: Theory and Applications.* Springer 2016.

Die moderne Darstellung der Berechenbarkeitstheorie, einschließlich Prioritätsmethode und Turinggraden. Soare hat auch die Begriffswende von „rekursiv" zu „berechenbar" durchgesetzt – er begründet sie im ersten Kapitel.

[27] Turing, A. M.: *On Computable Numbers, with an Application to the Entscheidungsproblem.* Proceedings of the London Mathematical Society 42, 1936.

Sechsunddreißig Seiten, die die Informatik begründeten. Abschnitt 9 – die Analyse des rechnenden Menschen – ist der philosophisch wichtigste Text des Fachs und ohne jede Vorkenntnis lesbar.

[28] Wegener, I.: *Theoretische Informatik – eine algorithmenorientierte Einführung.* 3. Aufl., Teubner 2005.

Deutschsprachig, mit dem Blick des Algorithmikers. Wegener fragt konsequent, was die Ergebnisse für die Praxis bedeuten – eine wohltuende Ergänzung zu den rein theoretischen Darstellungen.

Wenn Sie nur drei Bücher kaufen

[24]	**Sipser**	zum Lernen – das klarste Lehrbuch des Gebiets
[6]	**Davis (Hrsg.)**	zum Nachlesen – alle Originalarbeiten in einem Band
[17]	**Nagel/Newman**	zum Verstehen – Gödel in hundert Seiten

Auf Deutsch nehmen Sie **[11] Hromkovič**; **für die Fortsetzung [26] Soare** (Berechenbarkeit) *oder* **[19] Papadimitriou** (Komplexität); **für die Informationstheorie [14] Li/Vitányi**.

Und einmal im Leben sollte man ***[27] Turing 1936*** *im Original gelesen haben* – **jedenfalls den Abschnitt 9.** *Er ist vier Seiten lang und braucht keine Mathematik.*